Technical Guide - ForkNFind

By Thomas Hazekamp (20423602) and Eoin Daly (20424366)

Supervisor: Brian Davis
Date completed: 20/04/2024

Abstract

ForkNFInd is a mobile restaurant recommendation application designed to provide personalised restaurant recommendations to suggest to users based on their reviews. Using a machine learning hybrid recommendation system which incorporates both collaborative and content techniques, these recommendations are designed to be personal to each user, and by integrating a sentiment analysis machine learning model, each user review can be judged to be either positive or negative. This application makes use of real-time location using the Google Maps API to develop a database to expand our library of restaurants. Developed using a Django backend and a React Native frontend, the application seeks to bridge the gap by providing an easily accessible, time-saving solution for finding new dining experiences whilst providing a great learning opportunity to further our understanding of both new and existing technologies.

Glossary

INTRA: INtegrated **TRA**ining**,** this is DCUs work experience name.
Sentiment Analysis: The process of computationally identifying and categorizing options expressed in a piece of text.
Roberta: An optimized method for pretraining self-supervised NLP systems
Vader: A rule-based sentiment analysis tool.
Trees: Collection of nodes connected by edges.
API: Application Programming Interface is a way in which two or more programs can communicate with each other, there are a set of protocols, commands and functions which developers can use.
Frontend: Part of the system which is accessible by the user. It is every aspect of the mobile application in which the user sees and interacts with.
Backend: Part of the system which is not accessible by the user. The backend is where most of the data is being stored and retrieved from. It is the working brain of the system.
Database: Is used for storing, maintaining and accessing data.
Expo: Is an open-source platform for making universal native apps for Android, iOS and the web with JavaScript and React.
React Native: A library which is used primarily as a frontend framework for mobile applications.
Django: A Python-based web framework often used as the backend of web applications.
SQLite: A database engine written in the C programming language. Django uses this database to store data.
Heroku: is a cloud platform service which allows its customers to run and deploy web applications on the cloud.
REST: It is an application programming interface that uses HTTP requests to access and use any data.
HTTP: Hypertext Transfer Protocol is used to load web pages and transfer data between networked devices.
GET: A HTTP request to obtain data from a source.
POST: A HTTP request to provide data to a source.
PUT: A HTTP request to send data to a server.

DELETE: A HTTP request to delete data from a source.
JSON: Is an standard text-based format for representing data.
Token: It is a long string of characters used to identify a users identity.
UI: User Interface.
Endpoint: A specific location where an API sends and receives responses.
CI/CD: Continuous Integration and Continuous Delivery.
Jest: JavaScript testing framework.
Joblib: A Python library.

1. Motivation

The motivation for this project came from our own personal experience whilst working on our INTRA internships as we found it challenging to be able to pick places to eat that would suit our individual tastes and preferences. We often found ourselves looking for applications that would be able to recommend restaurants based on our previous likes and dislikes. We also had first hand experience while travelling to a new place and being caught in ‘Tourist Trap’ restaurants that only care about providing overpriced food. Our solution to these issues was to develop ForkNFind, a restaurant recommender built to recommend restaurants to users based on their tastes and preference.

2. Research

2.1 Recommendation System

2.1.1 Collaborative filtering

Collaborative filtering is a technique designed to filter out items that a user might like on the basis of reactions by similar users, the idea is it works by searching a large group of people and finding a smaller set of users with tastes similar to a particular user [1]. It does this by looking at a user's historical preferences and interactions and comparing them to other users' historical preferences and interactions. A simplified example of the technique is if a user A rates a restaurant highly that user B has also rated highly, then user A is more likely to like another restaurant that user B has rated highly. Hence the name collaborative filtering as it requires the users information to be collaborative to generate its recommendations.

Collaborative filtering advantages and disadvantages [2],

Advantages:

- No domain knowledge necessary - Domain knowledge not necessary because the embeddings are automatically learned.
- Serendipity - The model will help users discover new interests, the model will recommend items based on similar users that are interested in them.
- Great starting point - The system won't require contextual knowledge of features to get recommendations.

Disadvantages:

- Cannot handle fresh items - If an item does not have a lot of reviews for it, that will make it significantly harder for it to be recommended as it might not have been seen during training. This is called the **cold-start problem**.
- Scalability - Collaborative filtering often suffers when working with extremely large datasets due to the fact that as the overall number of items and user's increase the general processing power needed grows exponentially.

These types of recommendation systems have many real world use cases, telling us what to buy (Amazon), which movies to watch (Netflix), whom to be friends with (Facebook), which songs to listen to (Spotify) [3]. Broadly, there are 2 types of Collaborative filtering techniques that can be used by software and applications worldwide [4], User-based collaborative filtering and Item-based collaborative filtering. In simple terms they can be described as User-based, "User's who are like you also liked" while Item-based, "Since you liked this you might also like".

2.1.1.1 Item-based collaborative filtering

It was first invented and used by Amazon in 1998, and rather than matching the user to similar customers, item-based collaborative filtering matches each of the user's purchases and rated items to similar items, then combining those similar items into a recommendation list [5]. Item-based collaborative filtering is seen as a more effective method as the average ratings being given to a specific item don't tend to fluctuate as much as the average ratings being given by users to items.

2.1.1.2 User-based collaborative filtering

User-based collaborative filtering is a technique used to predict the items that a user might like on the basis of ratings given to that item by other users who have a similar taste with that of the target user [6]. This method can be better as it can adapt dynamically to the changes in user behaviour and preferences since it is based on real time behaviour, but also relies heavily on user's interacting with the system a lot to build up a user profile to make better recommendations.

2.1.2 Content Based filtering

Content-based filtering in recommender systems leverages machine learning algorithms to predict and recommend new but similar items to the user. Recommending products based on their characteristics is only possible if there is a

clear set of features for the product and a list of the user's choices. It does this by using both keywords and attributes assigned to each item in the dataset, and comparing them against the other items in the dataset. Items then with similar keywords and attributes will be judged to be similar and therefore would be recommended. An example of this is if a user likes Restaurant A that has the attributes 'Italian', 'Pizza' and 'Pasta' and we have two other restaurants, Restaurant B with attributes 'Chinese' and 'Indian' and Restaurant C with attributes 'Italian' and 'Pizza', the Content-based filtering will recommend going to Restaurant C as it has similar attributes to Restaurant A.

Content based filtering advantages and disadvantages [7],

Advantages:

- No user knowledge necessary - The model doesn't need any data about other users, since the recommendations aren't specific to each user. Making it easier to scale to a large number of users.
- Specific user interests - The model can capture specific user interests due to the fact it is comparing against items that the user has liked.

Disadvantages:

- Domain knowledge - The features used for this method of recommendation requires a lot of domain knowledge as the specific attributes chosen to use to get the similarity scores.
- Limited Expansion - The model can only make recommendations based on existing interests of the user, which means it has limited ability to expand on the user's existing interests.

2.1.2 Hybrid Recommender

Hybrid recommenders are a special type of recommendation system which can be considered as the combination of the content and collaborative filtering methods, combining these methods may help in overcoming the shortcoming we are facing at using them separately [8]. Using the best performing collaborative filtering and content filtering models identified above we will be able to build a hybrid recommendation system.

By combining both these models we will be able to overcome some of the disadvantages we had mentioned previously, the first disadvantage being the cold start problem for collaborative filtering, which was when new restaurants that had little or no reviews wouldn't be recommended as we had no user reviews to base recommendations off. This will no longer be the issue as every restaurant will have its features meaning it can be recommended by the content filtering approach part of the hybrid recommender.

Another disadvantage previously mentioned was the fact that in a content filtering recommender that there was limited expansion as the model could only make recommendations based on existing interest of the user, leading to limited ability to expand on user's existing interests. The collaborative based approach of the hybrid recommender will be able to match users that have had similar ratings for restaurants and based on these make recommendations of restaurants those similar users had rated highly.

The combination of these separate recommendation systems into a hybrid system will allow the model to use the strengths of both systems to overcome the previously mentioned weaknesses of both individual systems.

2.2 Sentiment Analysis

When creating our project ForkNFind we decided to add sentiment analysis as part of a feature within the application. Sentiment Analysis or opinion mining is a natural language processing (NPL) technique that allows to classify specific data (in our case textual data) as positive and negative. The main goals of our sentiment analysis system is to show users the sentiment of reviews in a visual way, the other goal would be to gain insights into the customer feedback for each establishment.Allowing users to view the sentiment of reviews, allows the users to more easily decipher negative and positive reviews for any restaurant/location they are currently viewing.

Implementing sentiment analysis was a first-time experience for us and we wanted to include this type of machine learning as it serves an interesting purpose for our application. Initially, we looked at some resources online and implemented very quick and off-the-shelf solutions, these include implementing the Roberta model and Vader models to an Amazon reviews dataset. These resources allowed us to gain a quick, easy and general understanding of sentiment analysis. We wanted to understand and take a deeper look at how the whole process of sentiment analysis takes place, this is why we decided to look into text processing, vectorization types, models and working with datasets.

2.2.1 Text Processing

Text processing is an important part of any machine learning algorithm as it allows for the cleaning of the data, resulting in more accurate final results. In our case, we would need to process and clean the data for a more accurate classification of the given text, this is done as often there is a lot of noise inside text (especially reviews given by people) and many words within a piece of text do not have an impact on the general orientation of it [9].

Within the text processing of reviews, we needed to have functions for different types of cases. Below is a list of each function with a description of its use case.

- convert_emoji_to_text - Used to convert any given emoji in a text by using an emoji library, this converts the given emoji into its descriptive text
- remove_links - Used to remove any web links by identifying the general patterns of a web link
- remove_hashtags - Used to remove any text directly following hashtags
- lower_case_string - converting all text to lowercase
- remove_letter_repetitions - Used to remove letter repetitions
- remove_punctuation_repetitions - Used to remove punctuation repetitions, but making sure to keep punctuation if it exists as keeping punctuation is recommended as examples such as P.H.d [10]
- replace_contractions - Used to replace contractions with their full form, this helps to eliminate unnecessary symbols and make the text clearer [11]
- custom_tokenization - Allows for customization of the tokenization of words, the options include keeping punctuation, alphanumerical characters (i.e. letters, digits and some special characters) and keeping stopwords (e.g. not, a, the, is). Some stop words may carry a significant amount of value which is why some words need to be included when evaluating word sentiment [12]

2.2.2 Stemming

Stemming is a largely used method in NLP as it allows for the collapsing of distinct word forms. The main benefit of stemming is the reduction of vocabulary size which causes a more accurate result [13]. Since many words can be contained in one stemmed version, it lowers the number of tokens hence lowering the computation required. Below is an example of how a stemming algorithm creates the final stemmed word.

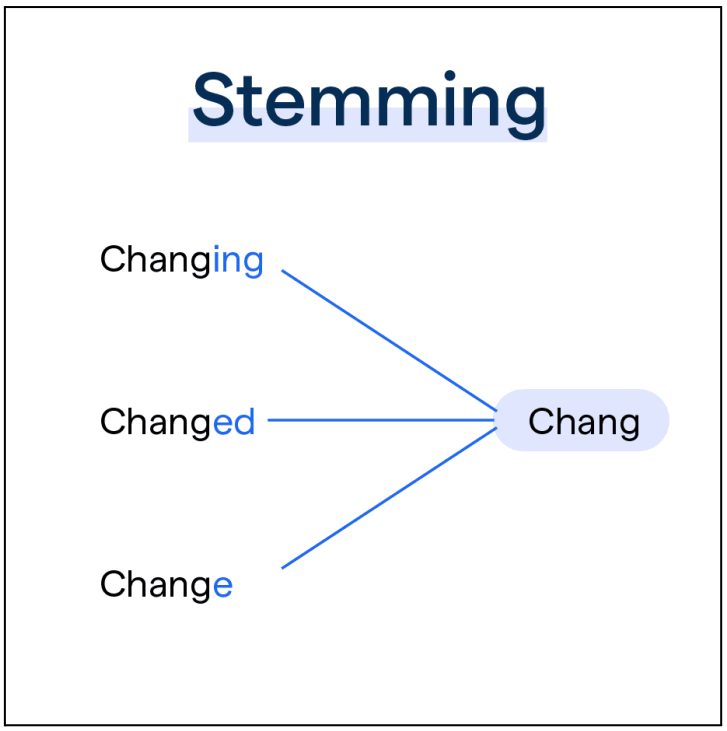

Fig. 1 - Stemming [14]

Within our Sentiment Analysis, we have the option to stem using Porter, Lancaster or Snowball stemmers. All three stemmers do a similar job but the Lancaster stemmer is the most aggressive (which could cause extra, unnecessary overstemming), meanwhile, the Snowball stemmer is an improved version of the Porter stemmer.

We decided to use the Snowball stemmer as it is an improved version of the Porter stemmer, which is a highly popular stemmer. The Snowball stemmer unlike Porter allows for better customization like ignoring stop words which may carry value in the text. Finally, the Snowball stemmer supports multiple languages which could become an important feature in the future for our Sentiment analysis.

2.2.3 Vectorization

Vectorization is another method of NLP that allows the conversion of text data into a numerical representation which then allows machine learning algorithms to understand the significance of given text and words. Vectorization methods allow the mapping of each given word into a high dimensional space, these vectors provide insightful information on the semantic relationship between words [15]. In our case using this method, helps the machine learning algorithm gain more insight into the word meaning, hence allowing for better accuracy in our final prediction of the phrases' sentiment.

The two methods of vectorization we implemented and tested are Count Vectorization and TF-IDF (Term Frequency-Inverse Document Frequency). Essentially, Count Vectorization keeps a tally of each token for each provided phrase

in the database. TF-IDF on the other hand measures the frequency of a token (similar to Count Vectorization) by converting that frequency into a number from 0 to 1 (calculated by dividing the number of times the token appears in the document by the total number of words in that document). The second step of TF-IDF is the measurement of the importance of a token across all the documents in the corpus and is calculated by taking the logarithm of the total number of documents in the corpus divided by the number of documents in which the token appears, finally, the two values are multiplied together to determine the importance of that token within the document [16].

2.2.4 Models

We looked at several models which are commonly used in Natural Language Processing projects, in this section we will specifically focus on the details of these models and how they could help us solve the issue with sentiment analysis within our project.

2.2.4.1 Logistic Regression

Logistic regression is a supervised machine learning algorithm that accomplishes binary classification of tasks by predicting the probability of an outcome. This algorithm provides possible outcomes such as yes/no, 0/1 or true/false. Logistic regression is an often used model when there is an expected binary response, for example, the probability of a rainy day [17].

We decided to use Logistic Regression as part of our project as it is a model generally suitable for linearly separable datasets, in our case, our Yelp dataset produces only two possible classes for any given row in the dataset. Logistic regression is generally seen to be a simpler Machine Learning algorithm to implement as it does not demand higher computational power when training this model, meanwhile, this cannot be said for other models [17].

Logistic regression uses a logistic function known as the sigmoid function which maps the predictions and their respective probabilities. The Sigmoid function refers to the S-shaped curve that converts real values to a range between 0 and 1. To determine which class a particular instance is from, the Sigmoid function (which is an estimated probability) will have a predefined threshold and if the estimated probability lies below that threshold, then the instance is not part of that class [17].

2.2.4.2 Naive Bayes

The Naive Bayes algorithm is a probabilistic approach that predicts based on evidence and prior probabilities, it is a popular algorithm in text classification tasks as it is simple and efficient. Naive Bayes is based on Bayes' Theorem and the concept of conditional probability. It assumes that the presence of a feature in a class is unrelated to the presence of other features in that class [18].

The Bayes' Theorem helps by finding the probability of an event occurring given the probability of another previously occurring event. There are a couple of different Naive Bayes methods to use within our sentiment analysis, for example, Gaussian, Multinominal & Bernoulli. We decided to use Multinominal Naive Bayes as it is typically used for document classification [19].

2.2.4.3 Support Vector Machine

Support Vector Machine is a Machine Learning algorithm that uses supervised learning to solve complex classification, regression and outlier detection problems by performing optimal data transformations which determine the boundaries of data points based on the classes. It is a popular algorithm when it comes to Natural Language Processing.

SVM's primary goal is to identify a hyperplane which distinguishes the data points of different classes. It tries to maximize the margin between the support vectors, which gives rise to incorrect classifications for smaller sections of data points [20]. We used the Support Vector Machine algorithm as it is flexible with the number of different types of data it can handle and it is robust as it is not affected by outliers as much as other algorithms [21].

2.2.4.4 Random Forest

Random Forest is a popular machine learning algorithm that combines the output of multiple decision trees to reach a single result. A decision tree by splitting on data based on a series of questions, after each question we get closer to the final decision. Each branch of the decision tree is a 'Yes' or 'No' and the goal is to find the best split to subset the data which is generally trained by the Classification and Regression Tree (CART) algorithm. One main issue with decision trees is that they can be prone to problem such as bias and overfitting. Random Forest is an extension of feature randomness where it generates a random subset of features, as this ensures a low correlation among the group of decision trees. To conclude, the random forest algorithm is made up of several decision trees and each tree is compromised of a data sample drawn from a training set with replacement. While Random Forest has a reduced risk of overfitting compared to the generic decision trees, the main disadvantage relates to the time-consuming process since there is a computation for each decision tree [22].

2.2.4.5 Gradient Boosting Machine

Gradient Boosing Machines are a popular machine learning algorithm, similar to random forest, GBM algorithm is also based on trees, but the main difference is that while random forests build an ensemble of deep independent trees, GBMs build an ensemble of shallow and weak successive trees which each tree learning and improving from the previous. The main idea of boosting is to add new models to the

ensemble sequentially, this means that at each iteration there is a new weak, base-learner model that is trained by keeping in mind the errors of the ensemble learnt so far [23]. One advantage of using this model relates to adaptability as this algorithm can often work with data which has not been pre-processed, although in our project we do pre-process our data [23].

2.3 Google Maps API

For our project we had planned to use an external API for getting restaurant information that could be used on our application for both the user to search and as data for the recommender system. We first began researching different external API's that we could potentially use, like Foursquare, Yelp Fusion and Zomato, and looked at both advantages and disadvantages of using them. After examining all the API's we found that the Google Maps API was the best fit due to its large number of different endpoints, large collection of data and crucially being financially viable.

Within the Google Maps API endpoints we had to do further research to determine which specific web services we would use based on our requirements, which were to get information about restaurants around the user. The Places APIs has a specific endpoint Neatby Search allowing to provide both the latitude and longitude, radius around that point and keywords for getting points of interest around a user. There are two versions of this API one from Places API and the other from Places API (New), with the key difference being able to choose which attributes are returned in the Places API (New) which can result in cheaper bills as the load is less. This allowed us to make an informed decision to use the newer API as it had everything tho old API has, with the added benefit of costing less.

Using the new Nearby Search API we will be able to configure it so it only returns the google_id of the establishments found which is the cheapest option for the API parameters, then for any unseen google_id that we have not already saved to the database we can make use of the Place Details (New) API endpoint to get all the attributes associated with the specific establishment with that google_id. The Place Details (New) API allows us to provide a google_id on the URL to get all the attributes like name, location, type and lots more information. This will result in low bills for using the API's as we will only be getting more detailed information for unseen google_ids, while all seen google_ids will already have all their attributes and information saved into our local database.

In conclusion, from doing this research it has allowed us to identify which API we will make use of in our application by identifying the advantages and disadvantages of all the external APIs currently available to us.

3. Design

3.1 System Architecture

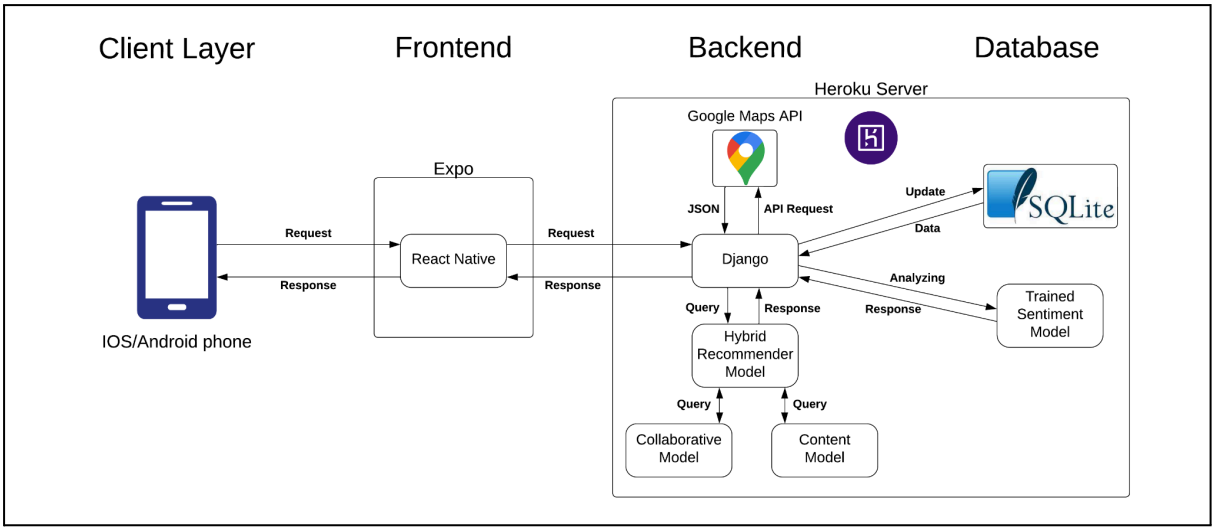

Fig. 2 - System Architecture of our application

The system architecture for our project provides a visual representation of the system components and their structure. Our system consists of four main components which work together to provide a seamless application to all users using it, these components are the client layer, frontend, backend and the database.

The client layer represents either an IOS or Android device as our mobile app has been built to be cross platform. This was accomplished using the Expo app which enables mobile devices to connect directly to a computer running the frontend without having to download any extra apps. This was important as trying to get an app published to the IOS app store isn't a straightforward process and requires a lot of time and money.

Clients using the expo app will be interacting with the React Native frontend. This frontend was chosen as programming the cross platform app for both IOS and Android is made easier as the code works on both platforms. It also has many other features such as supporting hot reloading and with it being open source means there are many libraries that are easily accessible to be used. One such library was the react-native-maps which allowed us to incorporate a map screen into our app showing establishments in the area around the user. Our application was built to have seven main screens for the user to interact with and various components built on top to further enhance the user experience. The frontend communicates with the backend using various API's.

The backend was built using Django, a python web framework, and interacts with the frontend receiving the API requests and manipulating the data based on the request. The backend also interacts with a third party API, the Google Maps API, which allows the backend to get more data about establishments in specific areas that can be displayed to the user. The hybrid recommender is also used in the backend to recommend restaurants based on the database and the restaurants being returned using the Google Maps API.

The final part of the system is the database layer which consists of the SQLite database which holds all the information relating to users, restaurants, reviews and anything else the application saves. The backend interacts with the database to gather information to be displayed to the user. The trained sentiment model is also saved as a .pkl file, allowing the backend to query the model and getting a sentiment score for a review inputted by a user. Both the backend and database layer are hosted on a heroku server.

3.2 Backend Architecture

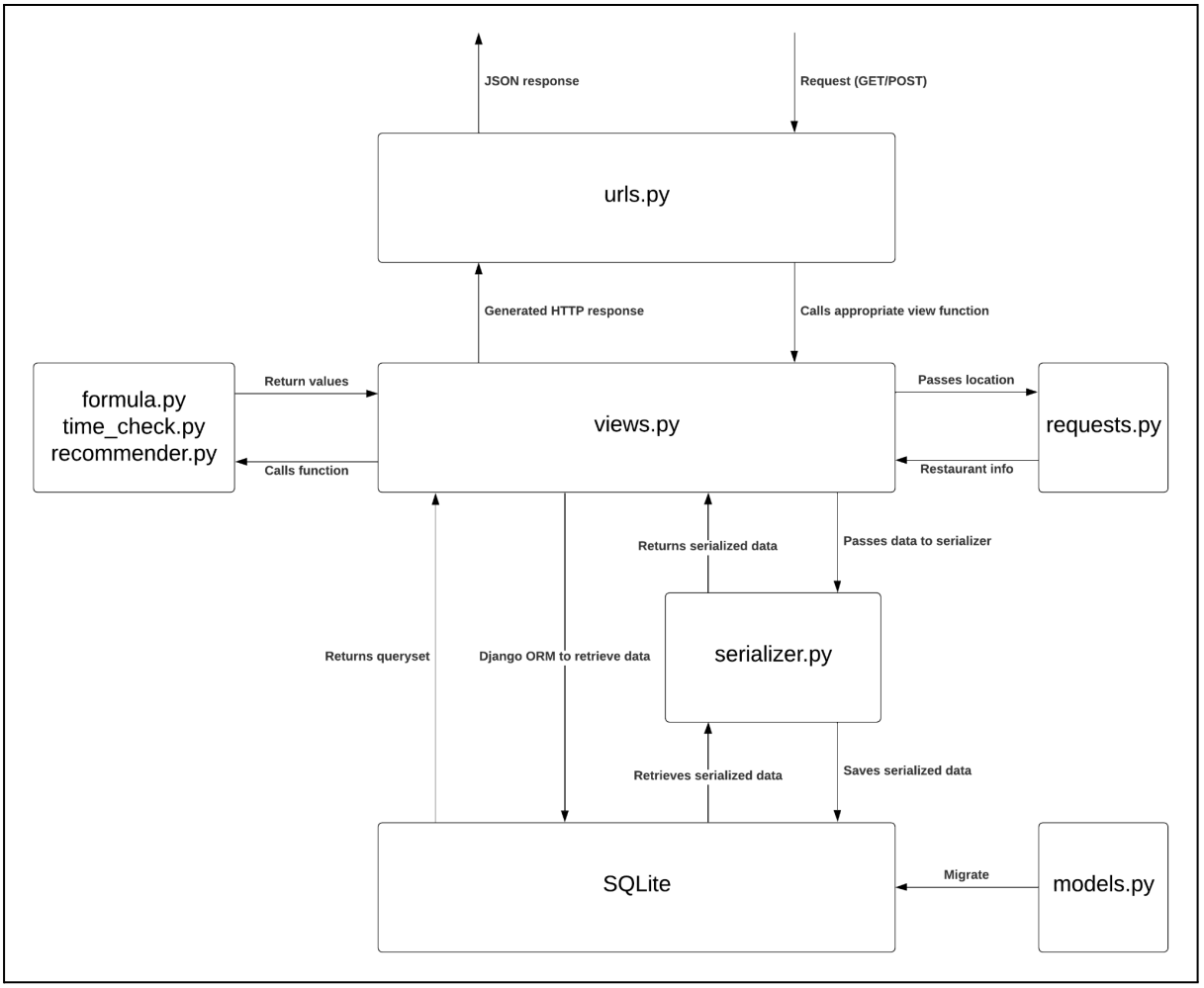

Fig. 3 - Backend Architecture

Whilst the system architecture diagram provides an overview of how the different components of the system interact with each other, the backend architecture diagram provides a clearer and more detailed overview of the inner workings of the Django backend. The Django backend is designed to work effectively with the frontend component using API's, and specifically using Representational State Transfer (REST), which provides standard HTTP methods (GET, POST, PUT, DELETE) which can be used to make the communication between frontend and backend a lot easier.

The urls.py file in Django is for defining the URLs that when searched will return data from the application, which is done by mapping the URLs to specific endpoints. Specifically in Django each URL is associated with a view class, which is defined in views.py, which means when the URL is matched it will call that view class to handle the logic related to that request. URLs also allow us to pass in route parameters which we use to search for specific information on restaurants, by using a restaurant ID we can search specifically for the restaurant attributes.

Once the urls.py file has matched the URL to a specific endpoint, the views.py is called with the request information passed in as a parameter. The view class defines the logic of what to do with the request information, which for example could be querying the database, calling a function from another file or calling a third-party API. If the views.py is interacting directly with the database it can use Django's Object-Relational Mapping, (ORM), which allows for the conversion of objects from the models.py file to rows in the relational database, this allows for increased productivity as it removes the need for writing SQL queries to interact with the database. Alternatively views.py can also call a serializer from the serializer.py which transforms the data.

Serializers are used to convert complex data types into Python data types which can be easily transferred across the network. They do this by iterating across each attribute in a class and transforming it to the format that has been defined in the serializer. This process works both ways, converting Python data types back into the more complex data types. This is essential to some view functions as this conversion makes it easier to respond to the frontend with the converted data types, and that is why we make use of them in our backend. Serializers also interact with the database by retrieving data and converting it to the Python data types.

The SQLite database uses the defined classes in the model.py file to create and manage the database tables. It does this by making migrations which are performed when new classes or attributes have been created in models.py, the migrations are a series of operations to be performed to modify the database to reflect the changes in the models.py file. This is a great feature of Django as it is a time saver not having to create the SQL queries to modify the database.

In summary, urls.py, views.py and serializer.py define how our backend interacts with data, from when it first receives a request until the response is sent, and the models.py defines how our database is designed. These files work together to control the flow of data throughout the system by using many of Django's features.

3.3 Class Diagram

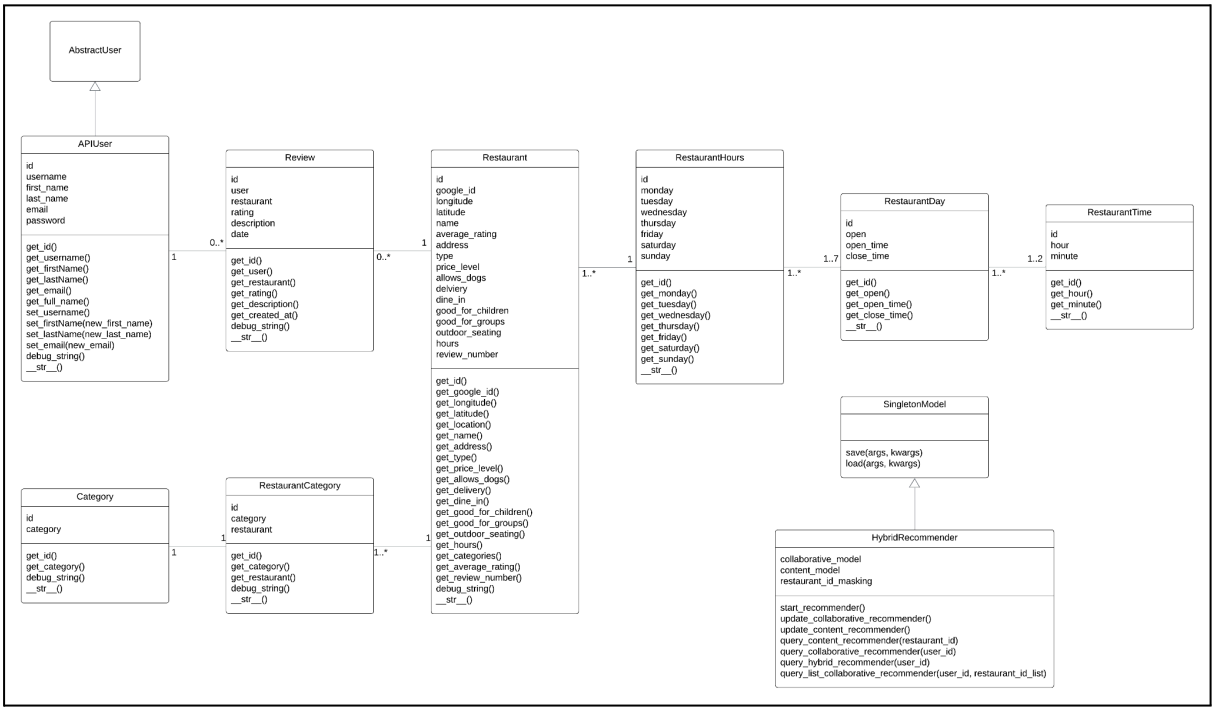

Fig. 4 - Class Diagram

The class diagram offers a visual representation of the structure of the classes in our system, it shows attributes, methods and relationships between classes. Some examples of the design of our system is the Singleton Model class which is designed to be a Singleton creational design pattern which ensures that we only have one instance of a class which has a global access point, which allows the HybridRecommender to inherit the functionality ensuring only one instance of the class is made available. This class holds both the collaborative filtering and content filtering models so that they can be used to generate the recommendations.

Similarity the APIUser inherits from AbstractUser, which is the built in class for using the Django authentication framework. Our APIUser class expands on the AbstractUser class by adding more custom methods which are used throughout the application.

3.4 Hybrid Recommendation System Architecture

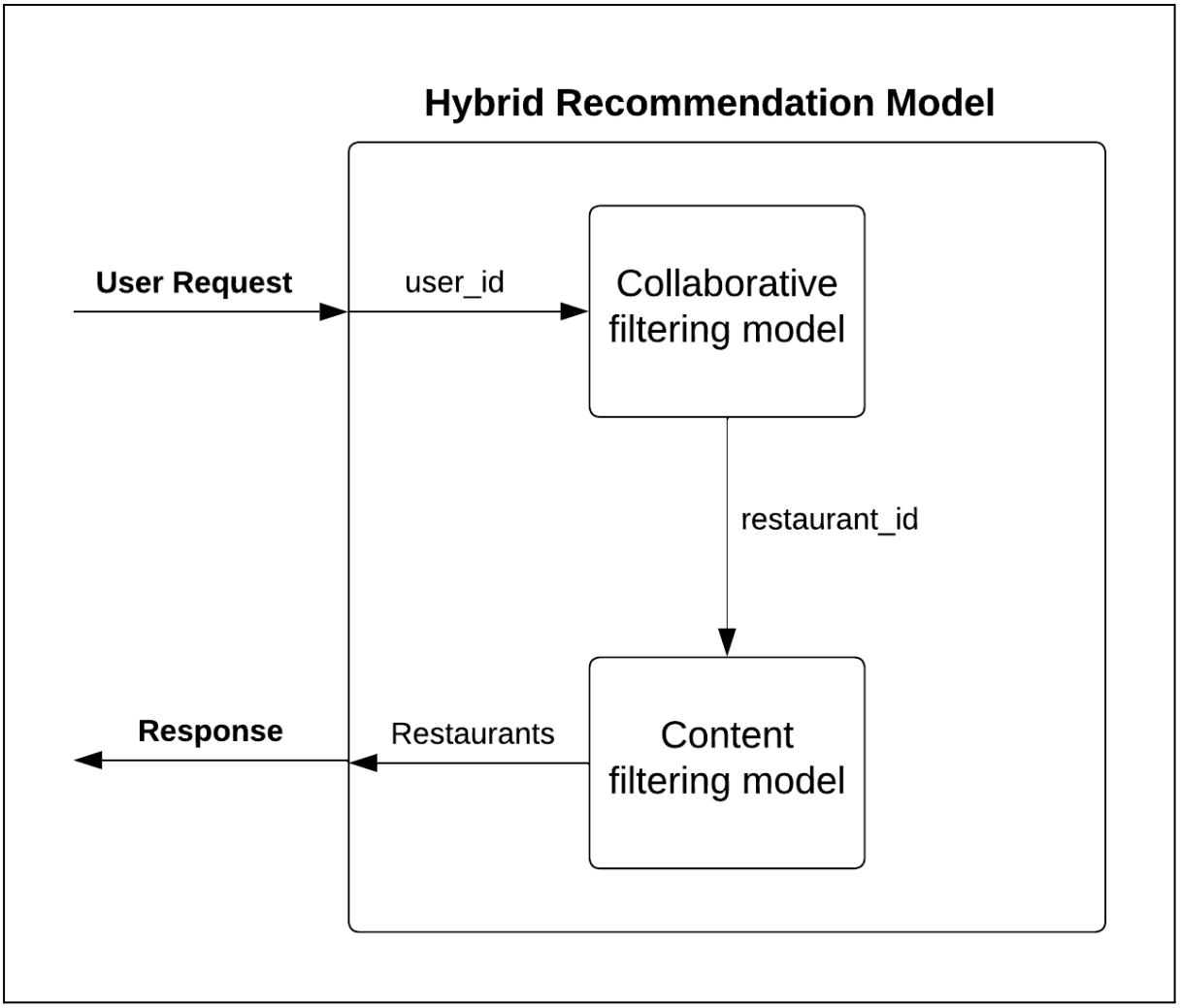

Fig. 5 - Hybrid Recommendation System Architecture Diagram

The Hybrid Recommendation system has been designed to combine both a Collaborative filtering model and a content filtering model together to help overcome both the shortcomings of these respective models. On receiving a request in the backend the model first passes the user_id into the collaborative filtering model, which will generate user specific recommendations based on reviews created by that user. With these user specific recommendations they are then passed into the content filtering model which further enhances the recommendations produced by the collaborative filtering model. Finally all these recommendations can be returned as a response. Both the Collaborative and Content models contain more complexity which will be further described in the subsequent sections.

3.4.1 Collaborative Recommendation System

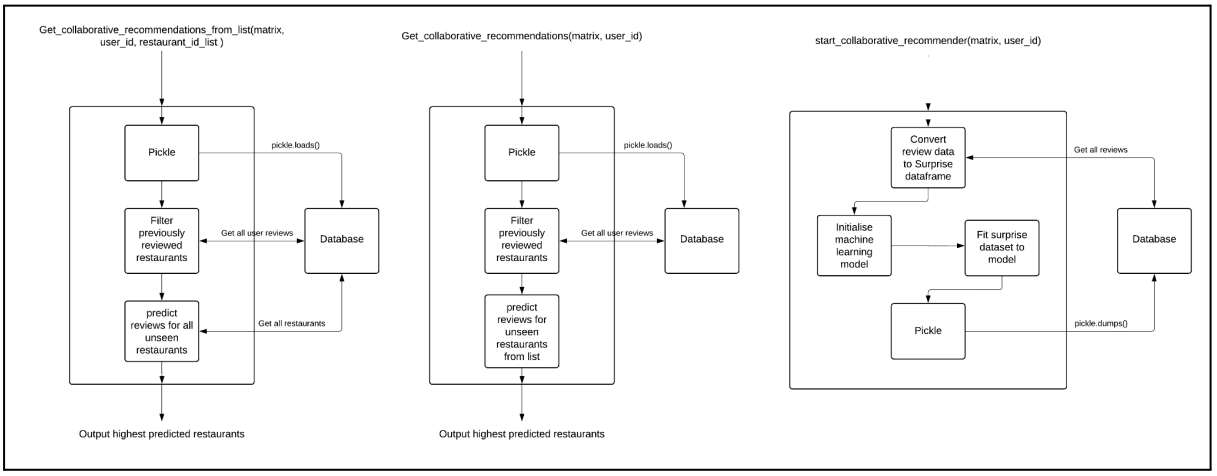

Fig. 6 - Collaborative Recommendation System Diagram

The Collaborative Recommendation System consists of three main functions which get called from the HybridRecommendation class. Each function is designed to either produce restaurant recommendations or to start/update the recommender.

start_collaborative_recommender() runs on startup of the django server and reruns anytime a new review is added to the database. The function initialises the collaborative recommender model and saves it to the database, so that it can be used to predict restaurant ratings for users. It retrieves all reviews from the database, converts them into a surprise dataframe, fits the data to a machine learning algorithm and finally, using pickle, it serialises the trained model so that it can be saved to the database.

Both get_collaborative_recommendations(matrix, user_id) and get_collaborative_recommendations(matrix, user_id, restaurant_id_list) are similar functions in that they are where the predictions are made. They both begin by using Pickle to unserialise the model from the database, they then differ in that the first function then gets all restaurants from the database to make predictions on them, while the second function only uses the restaurant ids that have been passed in as a parameter to make predictions on. Once these predictions are made the restaurants with the highest predicted ratings are returned from the functions.

These functions allow for the Collaborative Recommendation System to work effectively within the applications backend and as a component that works within the Hybrid Recommendation System.

3.4.2 Content Recommendation System

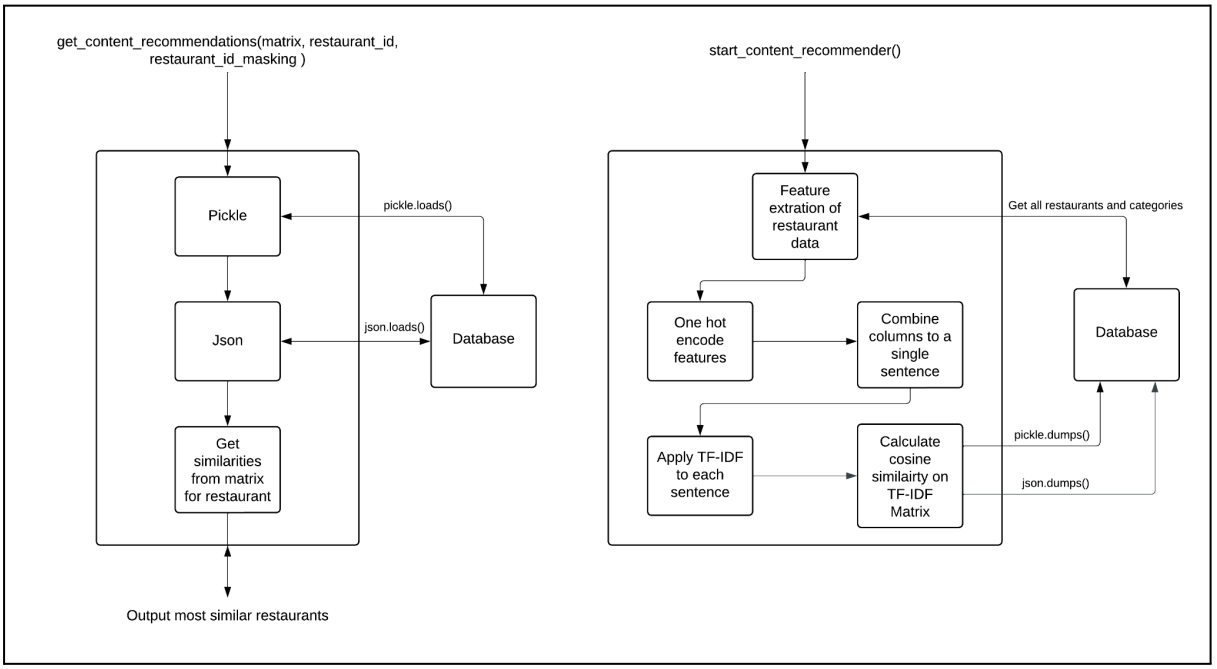

Fig. 7 - Content Recommendation System Diagram

The Content Recommendation System consists of two functions which are called from the HybridRecommendation class. Similarly to the Collaborative approach the functions either start/update the recommender or produce recommendations.

start_content_recommender() is called on startup of the django server and is rerun anytime a new restaurant is added to the database. It's purpose is to initialise both a similarity matrix and a restaurant mapping dictionary which are both stored to the database. This is completed first by getting all the restaurants in the database and extracting their features, both attributes and categories, then by one hot encoding these values it allows us to use the categorical variables in our models. Every hot column for each restaurant is then combined into a new column "sentence" which has TF-IDF applied to capture the importance of features in the dataset. Finally cosine similarity is then used on the TF-IDF matrix to calculate the similarity between each vector representing a restaurant. Once completed the similarity matrix can be saved to the database, using pickle, with all the values allowing quick access for when we want to query the model.

get_content_recommendations(matrix, restaurant_id, restaurant_id_mapping) is the function used for querying the previously created model. The function takes an input of the similarity matrix, a specific restaurant id and the restaurant id mappings. It first uses both pickle and json to convert the matrix and restaurant_id_mapping to python objects, and then passes in the masked restaurant id to the matrix to get all the similarity scores for that restaurant. By taking the top 3 restaurants from the matrix they can be returned by the function as the most similar restaurants.

These functions allow for the Content Recommendation System to work directly with the Collaborative Recommendation System as the restaurant id's outputted from the Collaborative system can be directly fed into the Content system to enhance the recommendations to the user.

3.5 Sentiment Analysis System Architecture

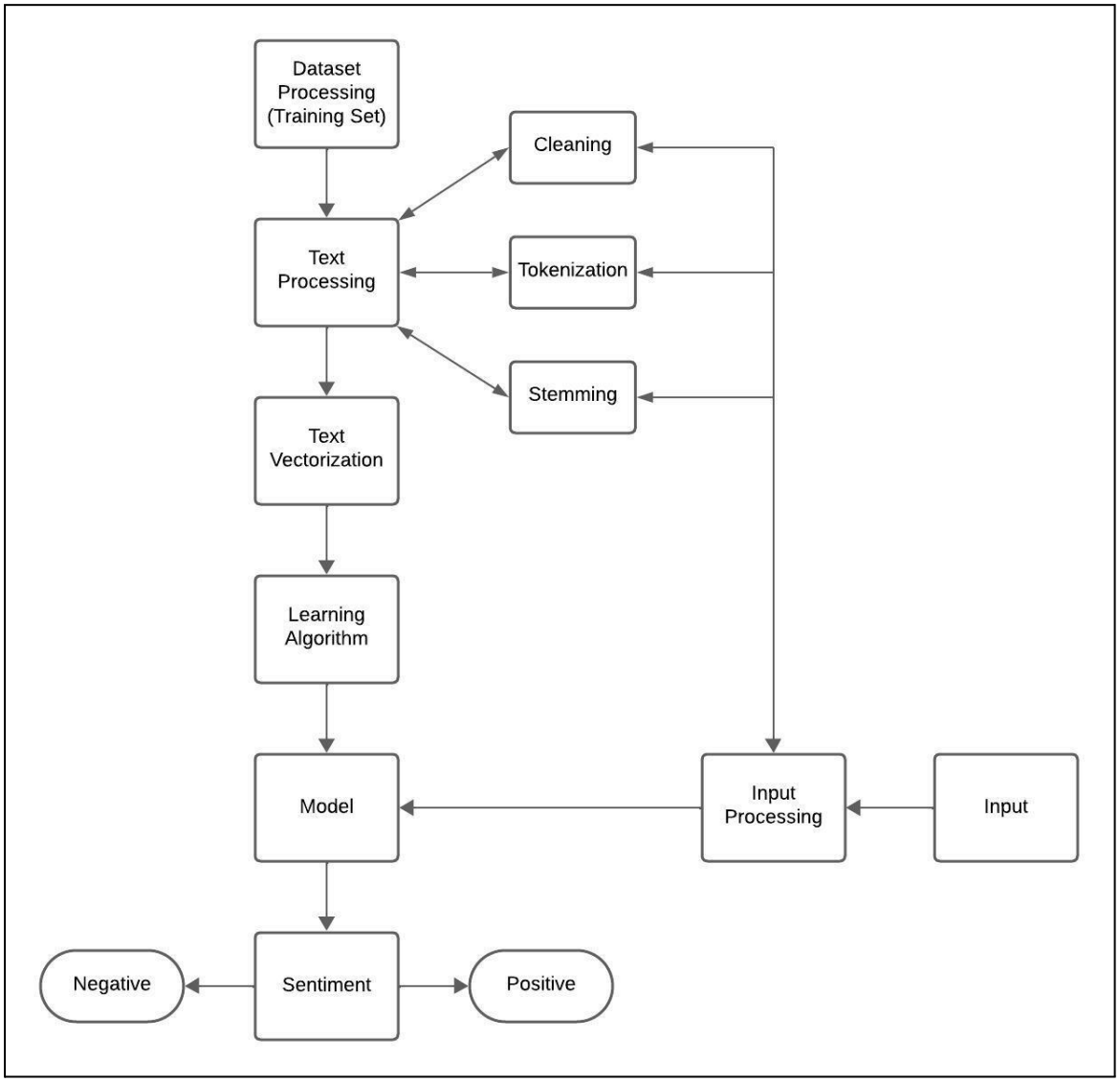

Fig. 8 - Sentiment Analysis System Architecture Diagram

The sentiment analysis starts by processing the dataset, which includes modifying specific columns in the dataset and converting them into lists. The next step is to process each piece of text within the data given, the cleaning of data (converting emojis, captitilization, repetition of letters, etc.), tokenizing and stemming the data. Vectorization occurs before the data is ran through the algorithm which then creates a trained model. When there is a specific input given, the input text is processed and

then ran through the trained model which will then provide a positive or negative sentiment of the text.

3.6 Map View

3.6.1 Sequence Diagram

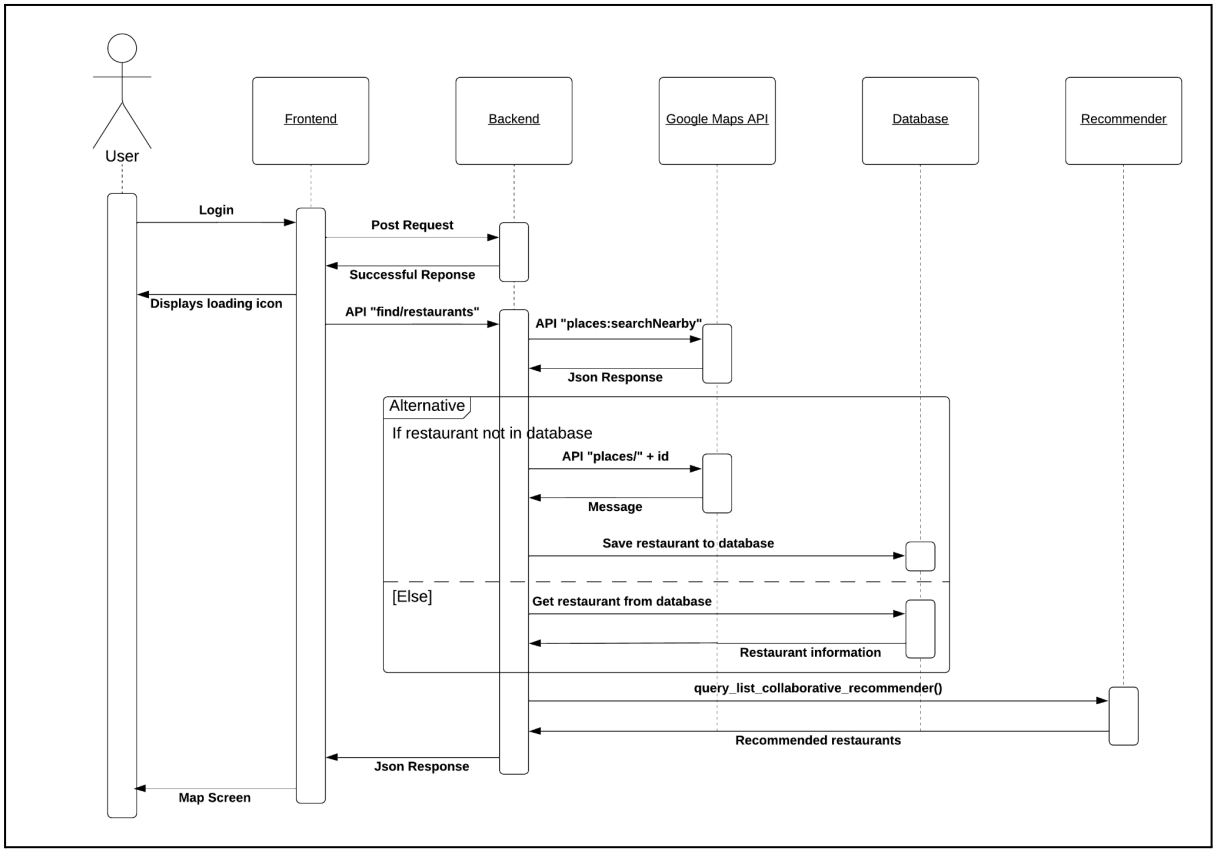

Fig. 9 - Sequence Diagram Map View

This sequence diagram shows the sequence of messages between the components of the system for generating the map view when users first login to the system. Once a user logs in they are redirected to the map screen which will show a loading icon. Whilst the loading icon is being displayed the frontend communicates with the backend through the “find/restaurants” API which will return all the restaurants nearby to the user.

To return all the restaurants to the user the logic must first be applied to get the correct restaurants, this is done by first calling the Google Maps API to check to google database for any restaurants in the area and the restaurants returned will be checked to see if they are in our local database and if not, the restaurants will be added to the database by using another Google Maps API to get specific more detailed information about each restaurant. Once all the restaurants have been found within a certain area of the user they are passed into the recommender system

to predict which of those restaurants the user would be likely to rate the highest. The top ten percent have an attribute added to their JSON to have recommended as true. Finally the backend responds to the frontend with all the nearby restaurants in JSON format which can be displayed on the map screen.

3.6.2 User Wireflow Diagram

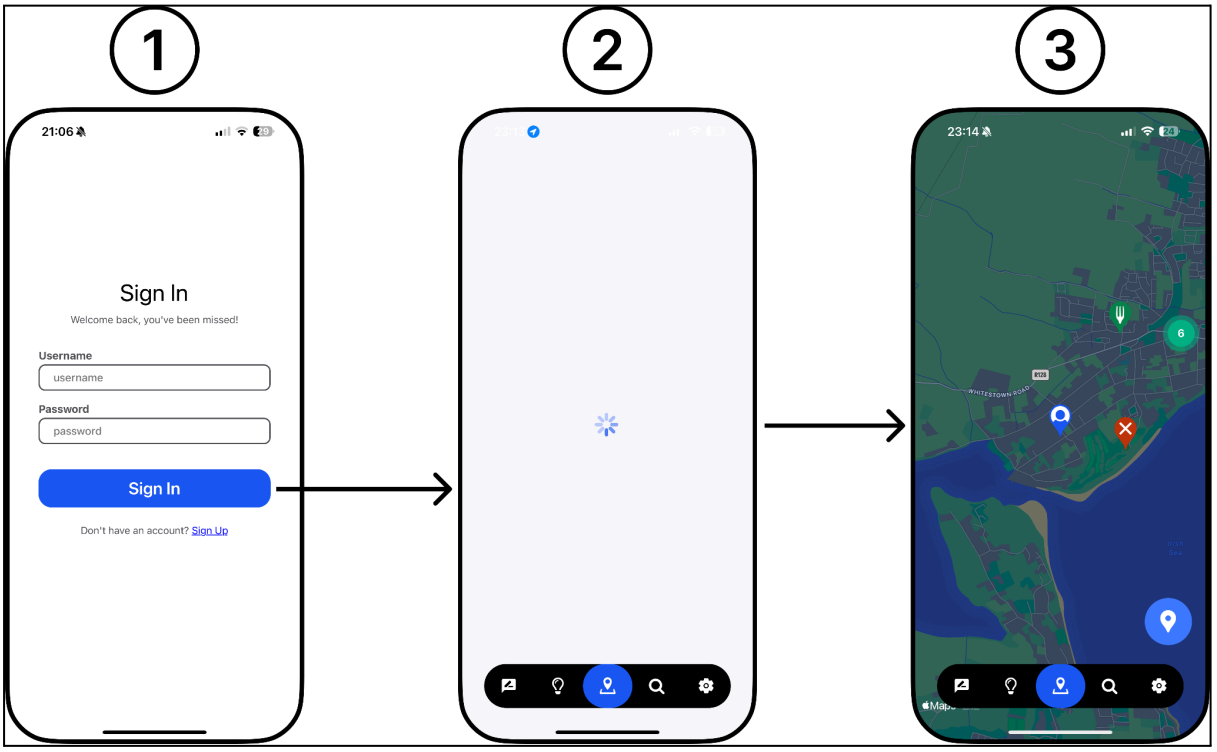

Fig. 10 - User Wireflow Diagram

This wireflow diagram provides a visual representation of how a user interacting with the application would get the map view visually displayed to them. Screen one is the login screen and a user would put both their username and password into the provided text boxes, once completed they can click the “Sign In” button. This will move the user to screen two which displays a loading icon, which will be displayed while the application communicates with the backend. Once the restaurants have been obtained the screen will update to screen three showcasing the user location and nearby restaurants, which were fetched from the backend using the API described in the sequence diagram. This process isn't instant as there are a lot of background processes with getting the user location and communicating with the Google Maps API.

3.7 Creating a Review

3.7.1 Sequence Diagram

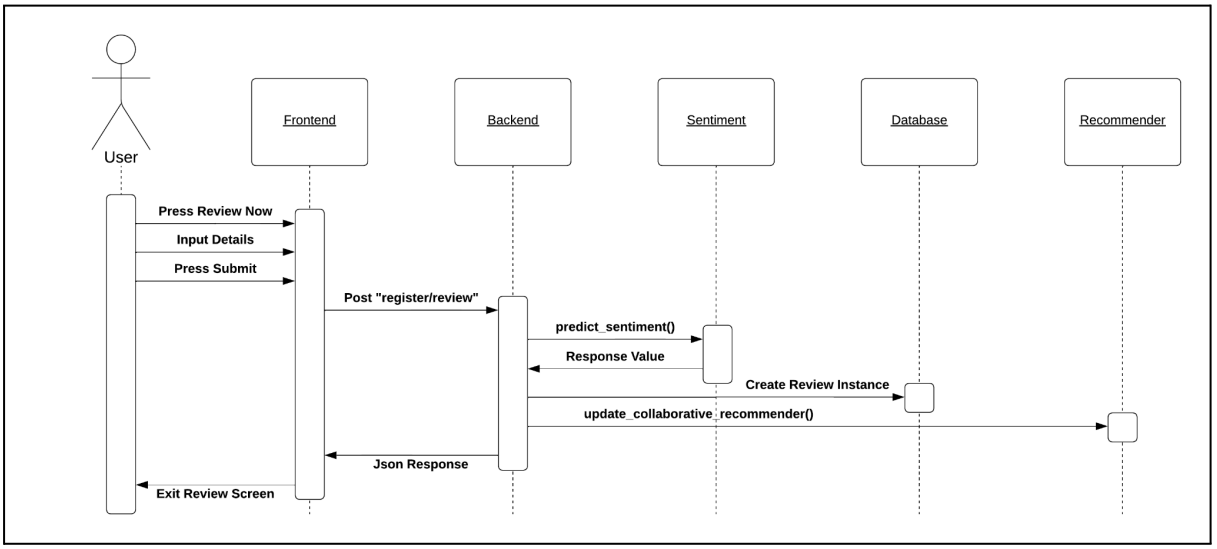

Fig. 11 - Sequence Diagram Creating a Review

This sequence diagram shows the sequence of messages between the components of the system for a user creating a review for a restaurant. A user can submit a review for a restaurant, which using the “register/review” API will create a review for that restaurant on the backend. The text provided with the review is first analysed by the sentiment model to determine whether the review was positive or negative, and then the review can be saved to the database. Once the review has been saved the recommender is notified to recalculate recommendations with this new information. The backend then responds to the frontend with a json response and the user interface is updated.

3.7.2 User Wireflow Diagram

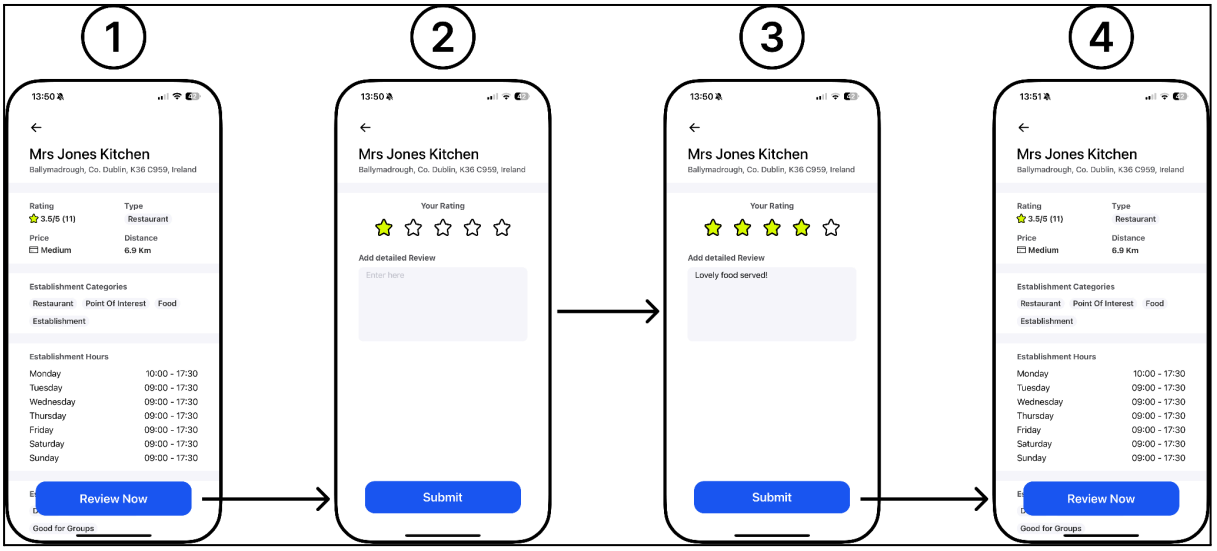

Fig. 12 - User Wireflow Diagram

This wireflow diagram provides a visual representation of how a user interacting with the application would create a review for establishment. Screen one is an individual restaurant page and if a user would like to create a review for that restaurant they can click the “Review Now” button. This brings up screen two, a review form where a user can enter the number of stars for the restaurant and a text review, once a user enters this information the screen would look like screen three. Finally for a user to submit the review from screen three they can click the “Submit it” button which will send an API request to the backend with the information and once the backend responds the review screen will be closed and will go back to the individual restaurant screen, as seen in screen four.

3.8 UI Design

3.8.1 Buttons

An important part of our project was the design of our buttons. Buttons are an important part of any application as they are the method in which users can interact with the system to navigate, provide input and trigger actions. For our button design we had many various factors we wanted to take into consideration like position, clear messaging and colouring.

The first factor we worked on was the positioning of the buttons. The positioning of buttons is very important in a mobile application as users shouldn’t have to constantly adjust their grip on the phone every time they are required to interact with buttons. Based on this reasoning we planned to put our important buttons at the bottom of the application and after doing research on the Gutenberg Principle which outlines how users naturally scan the screen starting from the top left and ending at

the bottom right, it made sense to have to buttons at the bottom of the screen as the users eyes would naturally end up falling on this position.

Another factor considered when designing buttons was having clear messaging. Clear messaging on buttons is important as it gives the users a clear path through the application as they will easily be able to understand what the button's purpose is. Some examples in the figure 13, is the login screen having the button “Sign In”, which clearly communicates the action the user will be performing when clicking the button and on the review screen the button “Submit”, which communicates that the user will be submitting a review. Clear messaging is very important as it allows new users to begin using the application with no learning curve on what each action the buttons are performing.

Finally the last major factor taken into consideration was the colouring of the application. The colour selection of buttons is very important as the visibility and contrast of the button will allow it to be easily noticeable to the users and the psychological representation a colour button can stand for. For our design we used a blue button for all positive actions that the user could pick, from figure 13 these are the “Sign In”, “Review Now”, “Submit” and “Apply”, blue was chosen as the colour provokes positive emotions within people. Similarly we used a red button for all negative actions that the user could press, some examples are the “Reset” and “Back” buttons which both have the red colouring. Red is a colour that provokes negative emotions within people, so that is why this colour was chosen for the negative buttons.

Overall taking into account all these design decisions we feel that the design for buttons do a good job at enhancing the overall user experience whilst using the application.

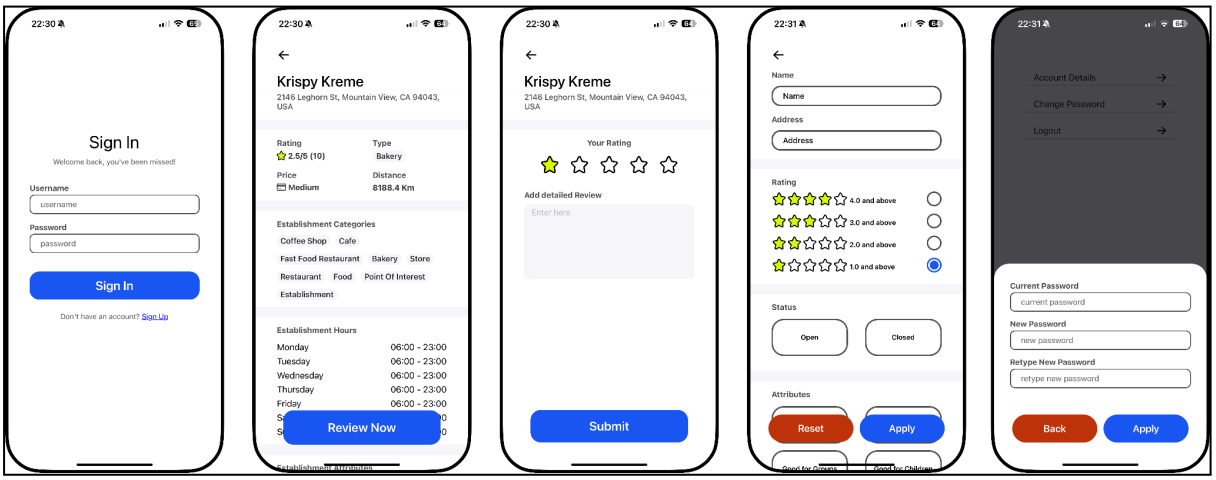

Fig. 13 - ForkNFind Application screenshot

3.8.2 Typography

Typography is the technique of displaying text so that it is both readable and appealing to view, in UI this is extremely important as having an application that is easily readable is essential to the application. For the typography of our application we focused on the factors of font, having a hierarchy and contrasting colours.

Our font choice for the application was to use React Native's native choices, which for IOS is San Francisco and for Android is Roboto. Both these fonts have been specifically designed for their respective platforms and will provide both a consistent and familiar look and feel with their native operating systems.

Establishing a hierarchy is important as it is a method of visually structuring the content of the UI based on the importance of certain elements. Figure 14 showcases on a restaurant page the various different types of text we can have to develop a hierarchy. The most important text we have is the heading, which in this case is the restaurant's name, which is both the biggest font size, the highest contrasting colour to the background and has medium bold applied, this has been purposefully designed this way to make the heading stand out compared to the other content. The sub heading directly follows the heading and the difference in text can be noted as the font is much smaller and a different colour, again demonstrating the overall hierarchy of the typography. This can also be noted between sub section headings and the sub section content that there is a difference in the font colouring to showcase the hierarchy.

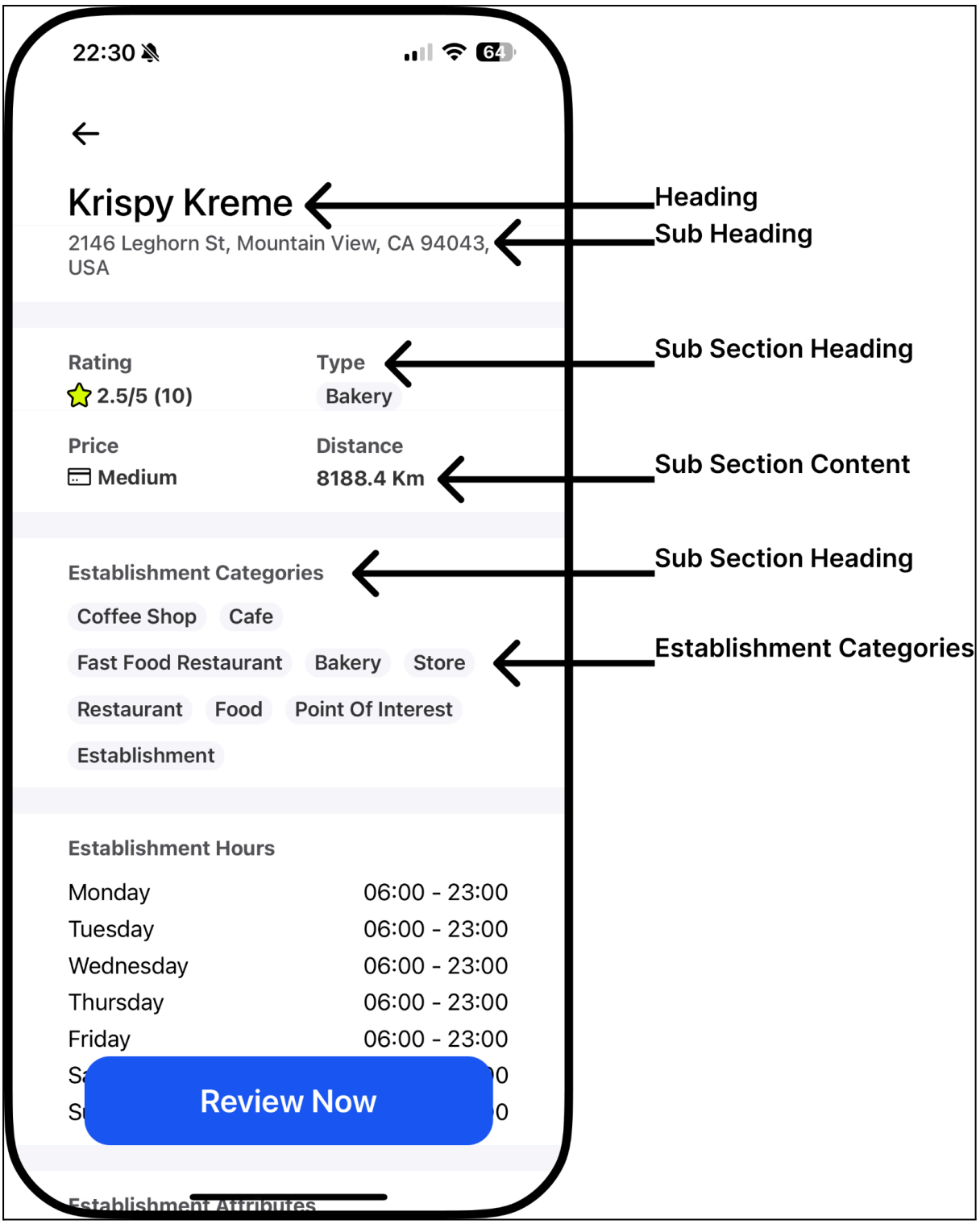

Fig. 14 - Restaurant screenshot

3.8.3 Iconography

Iconnography is the technique of identification and interpretation of symbols, in our UI we wanted to make sure that the viewable objects represent a general standard of understanding as this would lead to a simpler, cleaner and more efficiency user experience. The choice of icons is extremely important as they communicate the

core idea and intent of an action, apart from the increased aesthetic appeal of the application, using well placed icons can bring benefits such as saving screen real estate [24]. Saving screen real estate especially in a mobile application where screen space is already limited gives us a big reason to use well placed icons. We decided to use some universal icons such as stars to provide review ratings (figure 15), location symbol for the map (figure 15), a magnifying glass for the search page (figure 15), a cog for the settings page (figure 15), a box with a pen for the reviews page (figure 15). When displaying sentiment of a given review, we initially only had a light shade of green or red (depending on the sentiment) around the review box, this provided major issues with colour blindness, so to fix this issue we added smiley face (positive sentiment) and sad face (negative sentiment) icons (figure 16). Other icons used in our application include, x's and ticks for closed and open restaurants, credit cards for the general price of locations and a user figure to pin point the users location.

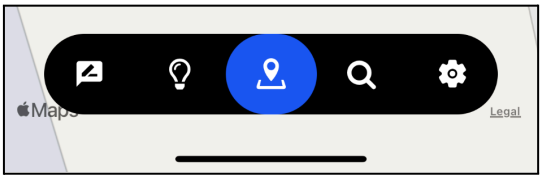

Fig. 15 - Navigation Bar

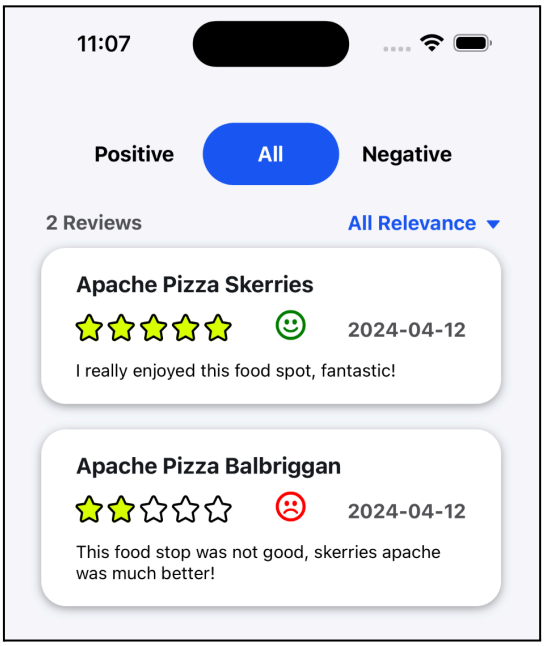

Fig. 16 - Stars & Sentiment Icons

4. Implementation/Sample Code

4.1 User Authentication

The user authentication for our application was designed to use the 'rest_framework_simplejwt' framework, which offers a JSON Web Token (JWT) for

handling the user authentication. Using the API endpoint ‘/api/token/’ and sending a post request of both a username and a valid password, a JWT token gets created which can be used on future API requests to validate the user is authenticated and who they are. The urls are defined in ‘src/backend/backend/urls.py’ and the code can be found as figure 17.

```python
from django.contrib import admin
from django.urls import path, include
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns = [
    path('', include('forknfind.urls')),
    path('admin/', admin.site.urls),
    path('api_auth/', include('rest_framework.urls')), # Apit auth token urls
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),

]
```

Fig. 17 - Python urls Code

The API endpoint ‘/api/token/’ gets called from the frontend in file ‘src/mobile_react_native/forknfind-mobile/app/navigation/requests/LoginAPIRequest.js’ and the code can be found as figure 18. This request includes the username and password as the body of the request and if the login was a success the returned access token is saved in a global variable, which can be used in future API requests requiring user authentication. This also updates a usestate to be true to signal the user has been logged in which will update what screens are accessible to the user. Error messages are set up to verify that both the username and password have been entered and also that the account credentials are correct.

```
import ErrorMessage from '../components/ErrorMessage';

// Function LoginAPIRequest
// setIsLoggedIn - use state passed in that will be updated if the login is successful
// username - the username inputted by the user
// password - the password inputted by the user
const LoginAPIRequest = ({setIsLoggedIn, username, password}) => {

    console.log(JSON.stringify({username: username, password: password}))

    // Fetch the API with this information
    fetch("http://192.168.1.82:8000/api/token/",
    {
        method: 'POST',
        headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json',
        },
        body: JSON.stringify({username: username, password: password})
    }).then(response=>response.json())
    .then(data=>{

        // Error checks in the form.
        if (data['username'] == "This field may not be blank.") {
            ErrorMessage(content="Username field may not be blank.")
        } else if (data['password'] == "This field may not be blank.") {
            ErrorMessage(content="Password field may not be blank.")
        } else if (data['detail'] == "No active account found with the given credentials") {
            ErrorMessage(content="No active account found with the given credentials.")
        }

        if (data["access"] == null) {
            return
        }

        // Save the information returned
        username_global = data["username"]
        access_global = data["access"]
        setIsLoggedIn(true)
    })
    .catch(error => {
        console.error('Network request failed:', error);
      })
    ;
}
```

Fig. 18 - API Request Code

The LoginAPIRequest function is called from the ‘src/mobile_react_native/forknfind-mobile/app/navigation/screens/LoginScreen.js’ which displays the user interface to the user through the frontend. This interface has a form that can be used to enter both the username and password of an account to login to the application. The UI can be seen in figure 19.

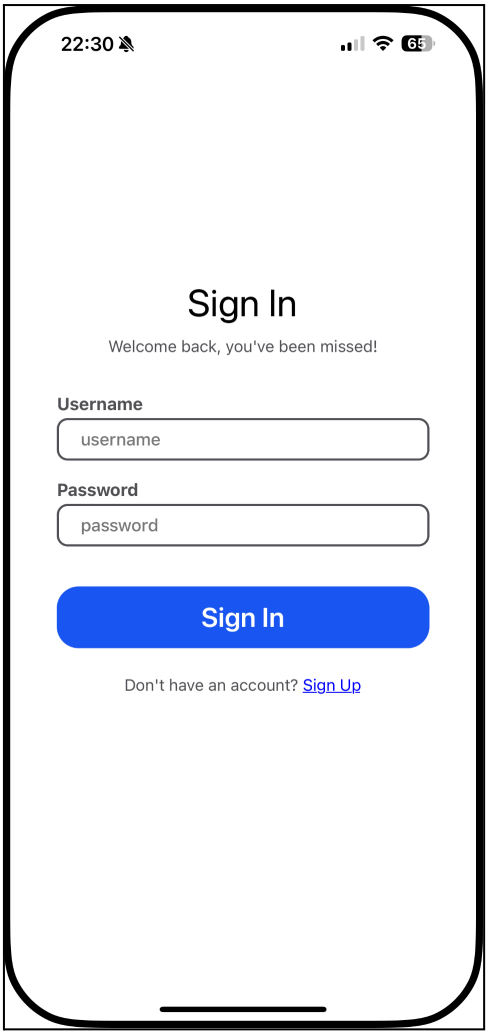

Fig. 19 - Sign in screenshot

4.2 Searching Restaurants

Being able to search for restaurants was an important aspect of our project as having a search system that allows for filtering would be excellent for narrowing down restaurants to visit. The API for searching restaurants is '/search/restaurant/<latitude>/<longitude>/', where both latitude and longitude are floats that have been provided as part of the URL. Once the API endpoint has been hit it calls the SearchRestaurantAPIView() class. The class extracts the latitude and longitude values from the URL, calls a SearchRestaurantFilter filterset class which will perform the filtering for the search and finally calls a serializer for converting the filterset to JSON. The class can be seen as figure 20.

```python
# SearchRestaurantAPIView using filtersets for the search functionality
class SearchRestaurantAPIView(generics.ListAPIView):
    serializer_class = RestaurantSearchSerializer
    permission_classes = [AllowAny]
    filter_backends = [DjangoFilterBackend]
    filterset_class = SearchRestaurantFilter

    # queryset of restaurants
    def get_queryset(self):
        return Restaurant.objects.all()

    # context for serializer
    def get_serializer_context(self):
        context = super().get_serializer_context()
        context['latitude'] = self.kwargs.get('latitude')
        context['longitude'] = self.kwargs.get('longitude')
        return context
```

Fig. 20 - SearchRestaurant API Code

The filterset class can be seen as figure 21, and defines various filter fields to be used with the parameter ‘lookup_expr’, with the majority using the value icontains, which will filter any instances that don't contain the parameter value. A custom filter method is used for the categories as a specific restaurant instance can have many categories assigned to it, so an external filter method is needed to loop through all the restaurant categories to verify if the inputted category is present. These filters will all be applied when an API request is received from the frontend.

```python
# SearchRestaurantFilter for the search lookup
class SearchRestaurantFilter(filters.FilterSet):

    # filters that can be used when searching
    name = filters.CharFilter(lookup_expr='icontains')
    address = filters.CharFilter(lookup_expr='icontains')
    type = filters.CharFilter(lookup_expr='icontains')
    price_level = filters.CharFilter(lookup_expr='icontains')
    allows_dogs = filters.BooleanFilter()
    delivery = filters.BooleanFilter()
    dine_in = filters.BooleanFilter()
    good_for_children = filters.BooleanFilter()
    good_for_groups = filters.BooleanFilter()
    outdoor_seating = filters.BooleanFilter()
    average_rating = filters.NumberFilter(lookup_expr='gte')
    open_or_close = filters.CharFilter(method='filter_open_or_close', label='Open or Close')
    categories = filters.CharFilter(method='filter_categories', label='Categories')

    class Meta:
        model = Restaurant
        fields = ['name','address','type','price_level','allows_dogs','delivery','dine_in',
                  'good_for_children','good_for_groups','outdoor_seating','average_rating','open_or_close','categories'] # Show these fields

    def filter_open_or_close(self, queryset, _, value):
        filtered_ids = [obj.pk for obj in queryset if self.get_open_or_close(obj).lower().find(value.lower()) != -1]
        return Restaurant.objects.filter(pk__in=filtered_ids)

    def filter_categories(self, queryset, _, value):
        # Get all the category instances
        categories_name = [category.strip() for category in value.split(',')]
        categories = Category.objects.filter(category__in=categories_name)

        # Filter to get all unique restaurant ids that have any of the categories appearing
        filtered_ids = RestaurantCategory.objects.filter(category__in=categories).values_list('restaurant', flat=True).distinct()

        return Restaurant.objects.filter(pk__in=filtered_ids)

    def get_open_or_close(self, obj):
        return get_open_or_close(obj)
```

Fig. 21 - SearchRestaurantFilter Code

The API endpoint ‘/search/restaurant/<latitude>/<longitude>/’ gets called from the frontend in file ‘src/mobile_react_native/forknfind-mobile/app/navigation/requests/SearchAPIRequest.js’, the function SearchAPIRequest handles the GET method by inputting the users latitude and longitude coordinate and also for creating a search string based on the parameters the user has selected. This is done by looping through an array of objects that have all the different possible search parameters with a value, which can be empty or a value inputted by the user. Once the search string is combined it could look like “/search/restaurant/53/-6/?” this would search the database for all restaurants with no filters or the search string could be “/search/restaurant/53/-6/?name=pizza&dine_in=true&open_or_close=open&” which would return all restaurants with the name pizza, dine in attribute true and is currently open. The request code can be seen as figure 22.

```
Function CreateAccountAPIRequest
// searchQueryRef - search query information
// setData - for setting the data
// setOriginalData - for setting the data as a backup
// location - location information for user
const SearchAPIRequest = (searchQueryRef, setData, setOriginalData, location) => {

    var searchString = "?";

    // loop through the search query information combing it together to maek 1 long query
    for (const [key,value] of Object.entries(searchQueryRef.current)){

        if (!(value == "" || value == false)) {
            searchString += key + "=" + value + "&"
        }
        console.log(key, value);
        console.log(searchString);
    }

    // add it to the end of the url
    fetch("https://lionfish-dear-roughy.ngrok-free.app/search/restaurant/" + location.latitude + "/" + location.longitude + "/" + searchString,
    {
        method: 'GET',
        headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json',
    },
    }).then(response=>response.json())
    .then(data=>{

        // set the data once its returned
        setData(data)
        setOriginalData(data)
    })
    .catch(error => {
        console.error('Network request failed:', error);
      })
    ;
}

export default SearchAPIRequest
```

Fig. 22 - API request Code

The SearchAPIRequest function is called from ‘src/mobile_react_native/forknfind-mobile/app/navigation/screens/SearchScreen.js’ which can be seen as figure 23 as this file displays the frontend for the search functionality. The first screen showcases the search in its simplest format, for searching only using the name, both icons on the search bar are used with the left activating the search and the right entering the filter screen. The filter screen allows for the search using all the different filters the address, rating, status, attributes and categories can all be used to expand the search to the users benefit. Overall this

implementation of the search functionality has allowed for a hugely customizable search with all the available filters.

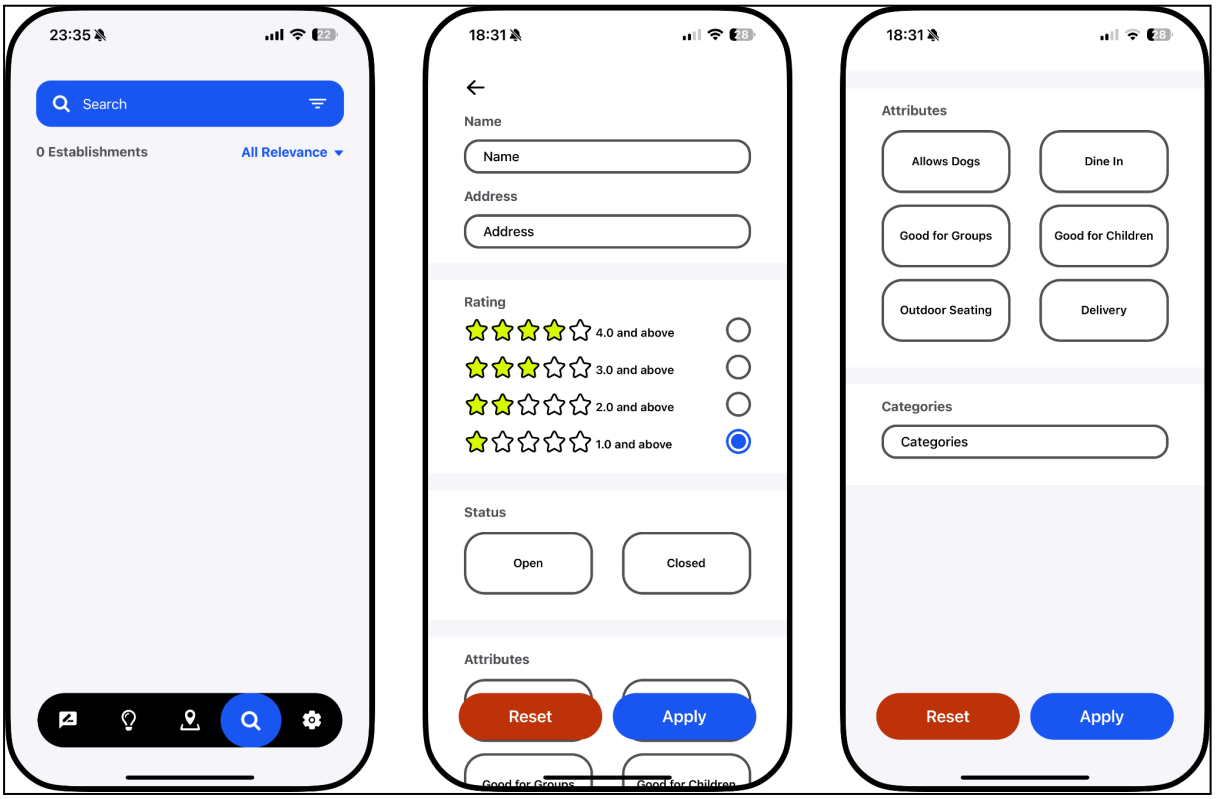

Fig. 23 - Application screenshots

4.3 Map Screen

The map screen is an important feature of the application as it allows the user to get a view of all the restaurants that are located around them. The API for getting all the restaurants around the user is '/find/restaurant/', which the frontend sends a post request containing their current latitude and longitude location. When the API endpoint is hit it calls the class RestaurantsAroundUserAPIView() which handles the logic behind getting the information.

Figure 24 shows the code for the POST method in the class. Using the Google Maps API the closest restaurants to the user in Google's databases will be returned and if they are unseen they are then saved to the database. Using the users longitude and latitude, the code loops through all the restaurants in ForkNFind's database to locate the restaurants within a 10 kilometre radius of the user, these restaurants within the catchment area will be returned as a response element to the frontend.

```python
# RestaurantsAroundUserAPIView
class RestaurantsAroundUserAPIView(APIView):

    def post(self, request):

        # get the longitude and latitude from the post
        longitude = request.data.get("longitude")
        latitude = request.data.get("latitude")

        # pass it to the google api
        google_api_functions(longitude, latitude)

        # get all restaurants
        queryset = Restaurant.objects.all()

        within_distance = {}
        restaurant_ids = []
        # loop through using haversine forumla to see if they are within distance, in this case its 10km
        for item in queryset:
            distance = haversine((float(longitude), float(latitude)), item.get_location())
            if distance < 10:
                restaurant_ids.append(item.get_id())
                within_distance[item.get_id()] = {'location':item.get_location(),
                                                  'id': item.get_id(),
                                                  'name': item.get_name(),
                                                  'type': item.get_type(),
                                                  'price_level': item.get_price_level(),
                                                  'average_rating': item.get_average_rating(),
                                                  'distance_from_user': distance,
                                                  'open_or_close': get_open_or_close(item),
                                                  'recommend': False,
                                                  'review_number': item.get_review_number(),
                                                  }

        # reload hybrid recommender as new restaurants added
        model = HybridRecommender.load()
        model.update_content_recommender()

        results = model.query_list_collaborative_recommender(request.user.id, restaurant_ids)

        for item in results:
            within_distance[item]['recommend'] = True

        # return the closest restaurants
        return Response(within_distance, status=status.HTTP_200_OK)
```

Fig. 24 - Map Code

The Google Maps API request code can be seen in figure 25. The code makes a request to the ‘v1/places:searchNearby/’ API endpoint and specifically requests that the type of information returned must have “restaurant” in them, twenty restaurants maximum, are ranked using distance (closest restaurants are returned) and are within a radius of ten kilometres. This function returns all the restaurants retrieved from the Google Maps API where the unseen instances are then added to the database.

```python
# searching google maps nearby to return restaurants
def google_maps_nearby_search(longitude, latitude):

    # url to search
    url = "https://places.googleapis.com/v1/places:searchNearby"

    # headers to include
    headers = {
        "Content-Type": "application/json",
        "X-Goog-Api-Key": "AIzaSyB-I-mzQNM2A5l2uInqEwxkgOaV1bkmn2o",
        "X-Goog-FieldMask": "places.displayName,places.id",
    }

    # data to send, has to be a restaurant, ranked via distance and longitude and latitude coordinates passed in
    data = {
        "includedTypes": ["restaurant"],
        "maxResultCount": 20,
        "rankPreference": "DISTANCE",
        "locationRestriction": {
            "circle": {
                "center": {"longitude": longitude, "latitude": latitude},
                "radius": 10000.0,
            }
        }
    }

    # sent the request with the information
    response = requests.post(url, json=data, headers=headers)

    # if it worked then get the json and return it
    if response.status_code == 200:

        restaurant_info = response.json()
        return restaurant_info
    # if it broke then it has not been seen yet what it sent as we haven't broke it
    else:
        print(f"I have no idea what is the issue in this case: {response.status_code} -- {response.text}")
```

Fig. 25 - Google Maps Search Code

The API endpoint '/find/restaurant/' gets called from the frontend in file 'src/mobile_react_native/forknfind-mobile/app/navigation/requests/NearbyRestaurantsAPIRequest.js', which can be seen as figure 26. This code sends a POST request to the backend and includes JSON data of the users location in longitude and latitude coordinates which are used in the backend. The returned data is saved to a use state to be used in the frontend to display the icons of restaurants.

```javascript
// Function NearbyRestaurantsAPIRequest
// setData - use state passed in that will be updated based on the data
// location - the device location
const NearbyRestaurantsAPIRequest = (setData, location, setOriginalData) => {

    fetch("https://192.168.1.82:8000/find/restaurants/",
    {
        method: 'POST',
        headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json',
    }, body: JSON.stringify({"longitude": location["coords"]["longitude"].toString(), "latitude": location["coords"]["latitude"].toString()}),
    }).then(response=>response.json())
    .then(data=>{

        console.log(location["coords"]["longitude"].toString(), location["coords"]["latitude"].toString())
        // set the data
        setData(data);
        setOriginalData(data);
    })
    .catch(error => {
        console.error('Network request failed:', error);
      })
    ;
}

export default NearbyRestaurantsAPIRequest;
```

Fig. 26 - Nearby Restaurant API Code

The code for implementing the map screen is located in the file ‘src/mobile_react_native/forknfind-mobile/app/navigation/screens/LocationScreen.js’ and figure 27 is the code for specifically displaying the icons on the map. Looping through the returned data from the POST request, and using the longitude and latitude from the restaurants we can render the icons onto the map. There are three variations of icons and they each use a different .png file for being displayed; the different icons are for open, closed and recommended and then a callout component is used to display information about that specific marker when it has been clicked, which we have used to display more information about the restaurant.

```
{/* Mark the restaurant locations by looping through data */}
{Object.keys(data).map(key => (
<Marker key={key} coordinate={{ latitude: data[key]["location"][1], longitude: data[key]["location"][0]}} >
    { data[key]["open_or_close"] == "Open" ?
    ( data[key]["recommend"] == true ?
        <Image
            source={require('../../../image/recommend_marker.png')}
            style={{width: 33, height:48, marginBottom: 40}}
        />
        :
        <Image
            source={require('../../../image/open_marker.png')}
            style={{width: 33, height:48, marginBottom: 40}}
        />
    )
    :
    ( data[key]["recommend"] == true ?
        <Image
            source={require('../../../image/recommend_marker.png')}
            style={{width: 33, height:48, marginBottom: 40}}
        />
    :
        <Image
            source={require('../../../image/closed_marker.png')}
            style={{width: 33, height:48, marginBottom: 40}}
        />
    )
    }
    <Callout tooltip={true}>
        <RestaurantCardMap restaurantData={data[key]} />
    </Callout>
</Marker> ))}
```

Fig. 27 - Looping Code

The user interface for the map screen can be seen as figure 28. The map gets displayed using Apple Maps on IOS and Google Maps on Android, this is because we are using the react library ‘react-native-maps’, which makes use of the native map software on the devices. This screen is highly interactive as it allows the user to move, scroll and zoom the map interface to view other restaurants in the vicinity and a filter is also provided allowing the users to filter to only one type of restaurant, i.e. open, closed or recommended.

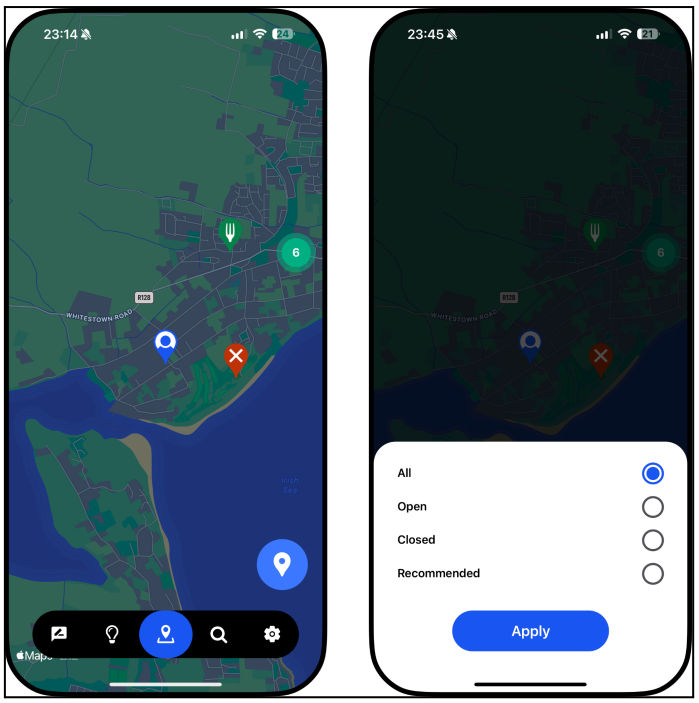

Fig. 28 - Map View

4.4 Recommendations Screen

The recommendations screen is an important aspect of the mobile application as it gives user personalised recommendations using the machine learning recommendations system. The API for getting the recommendations for the user is ‘recommend/hybrid/<latitude>/<longitude>’, when this API endpoint is hit it calls the class RecommendRestaurantHybridAPIView() which when it receives a GET request it will return all the recommended restaurants for the user.

Figure 29 shows the code for the GET method in the class. The implementation first loads the HybridRecommender instance which can be done easily due to it being designed to incorporate the Singleton creational design pattern, which once loaded will get the hybrid restaurant recommendations by passing the user id and then the method returns the restaurant recommendations. Looping through each recommendation the response JSON is built to include all the restaurants information and both the distance from the user and whether the restaurant is currently open or closed. Once the loop ends the API sends the response with all the restaurant information.

```python
def get(self, request, longitude, latitude):

    # load the recommender and query the hybrid recommender
    model = HybridRecommender.load()
    restaurants = model.query_hybrid_recommender(request.user.id)

    # loop through the id's getting the restaurant information
    response_data = []
    unique_list = []
    for item in restaurants:

        # hybrid could have duplicate id's so make sure nothing can be returned twice
        if item not in unique_list:
            unique_list.append(item)

            # get the restaurant info
            restaurant = Restaurant.objects.get(id=item)

            # append it tp the response data
            response_data.append({
                "id": restaurant.get_id(),
                "name": restaurant.get_name(),
                "type": restaurant.get_type(),
                "price_level": restaurant.get_price_level(),
                "average_rating": restaurant.get_average_rating(),
                "distance_from_user": haversine((float(longitude), float(latitude)), restaurant.get_location()),
                "open_or_close": get_open_or_close(restaurant),
                'review_number': restaurant.get_review_number(),
            })

    # return the recommended restaurants
    return Response(response_data, status=status.HTTP_200_OK)
```

Fig. 29 - HybridRecommender Looping Code

The method called on the instance of HybridRecommender can be seen in figure 30, this method first generates three collaborative recommendations for the user using the collaborative recommender model. Each of these collaborative recommendations are then passed into the content recommender to further enhance the amount of recommendations generated. Once completed all the recommended restaurant id's are returned from the function.

```python
# Query the hybrid recommender, inputting a user_id and outputting a list of similar restaurants
def query_hybrid_recommender(self, user_id):

    # First query the collab recommender, using the user_id and get a list of restaurants
    collab_recommended_restaurants = get_collaborative_recommendations(self.collaborative_model, user_id)

    hybrid_recommendations = []

    # Loop through the list of recommendations and input to content recommender to further generate recommendations
    for id in collab_recommended_restaurants:

        hybrid_recommendations.append(id)
        hybrid_recommendations.extend(get_content_recommendations(self.content_model, id, self.restaurant_id_masking))

    # Return the hybrid recommendations list
    return hybrid_recommendations
```

Fig. 30 - Query Hybrid Recommender Code

The API endpoint 'recommend/hybrid/<latitude>/<longitude>' gets called from the frontend in file 'src/mobile_react_native/forknfind-mobile/app/navigation/requests/HybridRecommendatiosnAPIRequest.js', which can be seen as figure 31. The code sends a GET

request to the backend and both the longitude and latitude of the user are included in the URL. The response from the API is saved to a use state which will update the user interface with the response data.

```
// Function LoginAPIRequest
// setData - use state for saving data
// setOriginalData - backup for saving data
const HybridRecommendationsAPIRequest = (setData, setOriginalData) => {

    fetch("http://192.168.1.82:8000/recommend/hybrid/53.580041/-6.107879/",
    {
        method: 'GET',
        headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json',
        'Authorization': 'Bearer ' + access_global,
    },
    }).then(response=>response.json())
    .then(data=>{

        // set the data
        setData(data);
        setOriginalData(data);
    })
    .catch(error => {
        console.error('Network request failed:', error);
      })
    ;
}

export default HybridRecommendationsAPIRequest;
```

Fig. 31 - Hybrid Recommender API Code

The user interface for the recommendations screen can be seen as figure 32. The first screen shows the loading screen that shows while the API is communicating with the backend, which once the response data has been received the screen updates with the recommended restaurants. The restaurants are displayed in a card design with key pieces of information like name, rating, distance from user, etc. The restaurants can also be sorted based on a variety of factors which can be selected by the user, these are displayed on the third screen.

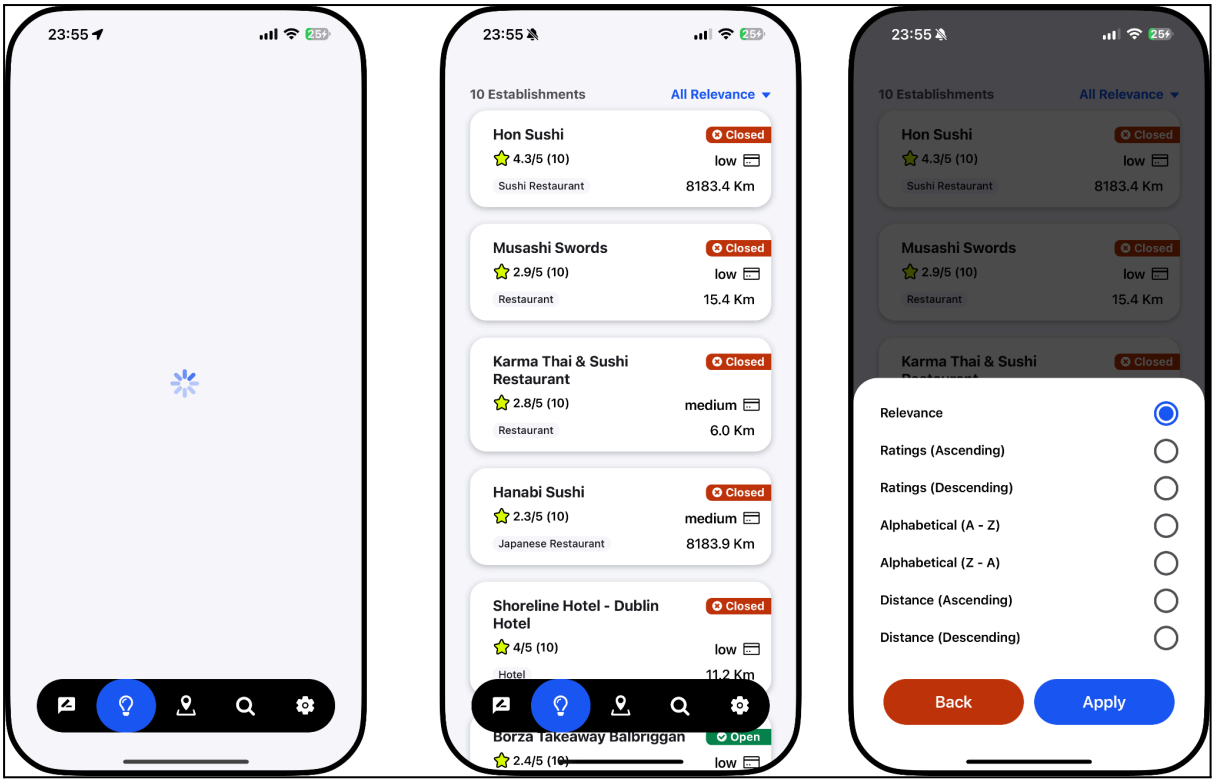

Fig. 32 - Recommendation View

4.5 Sentiment Analysis

In the following section, we will outline the steps and process of how our sentiment analysis system comes to a result. Firstly the function which initiates the process is called predict_sentiment, this function is situated in the runner_sent_analysis.py file. This predict_sentiment accepts the text, the type of vectorization and model wanting to be used. The first step of this function is to check whether a saved models file exists (this is to save time and not retrain the model continuously) the other steps include the processing of the given text, the transformation of the text and finally the sentiment prediction of the text. If the sentiment is positive, then the returned value will be ‘1’, if its negative the returned value will be ‘0’ (figure 33).

```
processed_text = process_text(text) # process the text - remove links, convert emojis to text, re
new_text = text_trans.transform([processed_text]) # transform the processed text - using the text
prediction = model.predict(new_text) # predict the sentiment of the new text - using the model

print(f'Current time at end: {datetime.now()}')

# Checking if the prediction is positive or negative with an error case
if prediction == 1:
    return '1'  # positive sentiment
elif prediction == 0:
    return '0'  # negative sentiment
else:
    return 'NA - error'  # not available / error
```

Fig. 33 - Code snippet of runner function

Suppose this predict_sentiment function cannot find a saved trained model. In that case, we will enter a run_model function which allows us to process all the text and data in the dataset, use a chosen vectorization type (CV or TF-IDF) and finally train a specified model on the train part of the already processed dataset. This new model will then be saved into a .pkl file for future use.

When processing the text from the run_model function we read in a dataset, apply column modifications and convert the dataset to lists. The file where all this dataset preprocessing occurs is called dataset_worker.py, in here we can specify how much of the dataset we want to work on, convert each of the rows' text into cleaned and processed text, convert a column with textual sentiment into a number system (0 and 1) and finally combine all the tokens of processed text together to create a whole corpus (Figure 34).

```
def apply_dataset_column_modifications(df): # applies the new processed text to the dataset.
    # apply the process_text function to the tweet_text column
    df["tokens"] = df["text"].apply(process_text)

    # change sentiment to 1 or 0
    df["text_sentiment"] = df["sentiment"].apply(lambda x: 1 if x == "positive" else 0)

    return df

# Convert the dataset to lists
def dataset_tolist(df):
    x  = df["tokens"].tolist()
    y = df["text_sentiment"].tolist()

    return x, y
```

Fig. 34 - Code snippet for dataset changes

5. Problems Solved

5.1 Google Maps API

Initially when setting up the Google Maps API we ran into various issues. This was a new API system we were using and due to the extensive library on offer from Google Maps API it was difficult to understand all the steps involved to set up an account, set up payment and to create the API key.

As this was a new area by looking up tutorials on how to set all of this helped massively and it allowed us to solve all the problems we had encountered and by using these tutorials we also learnt about the credits that Google gives monthly which allows us to have a certain amount of free API calls each month.

5.2 GPS User Location

An issue that we encountered while developing this system was related to getting the GPS location of the user. React Native has a library called ‘location’ which offers the ability to get the users longitude and latitude, but the issue is this library is designed to be called asynchronously, so once called the code won't wait for the response but instead will continue executing, which is an issue as we need the code to be executed synchronously so once it gets the user location we can send that to the API endpoint on the backend to get restaurant locations for the user.

The solution to this issue was to use the await keyword which means that getting the user location information will no longer be done asynchronously but instead synchronously as it waits for the function to return the information. This fixed our issue of the API’s being called too early and sending GET requests with the incorrect data.

5.3 Dataset

When we initially started working on the sentiment analysis we were using an example tweet dataset which allowed us to gain the necessary information to create a version of our sentiment analysis system, one of the big issues with the tweet dataset was in regards to misalignment of the data as we ideally wanted to implement a dataset with restaurant reviews, as this data would more accurately align with our problem space. We initially found it hard to find the right dataset that had the sentiment attached to each of the data, but ended up going with a large yelp dataset curated by Xiang Zhang, Jumbo Zhao, Yann LeCunn. To solve our issue we modified the columns of the new yelp dataset to be most similar to our previous tweet dataset, this helped with the continuity of our system.

6. Results

6.1 Unit Testing

6.1.1 Django & Recommendation System

Unit testing is important as it has helped us identify problem code areas, improve our code quality and encouraged us to build more maintainable code. For our Django backend we incorporated Django’s TestCase library to run our unit tests. These unit tests consisted of testing the class methods for all the classes and for testing important functions. Using the python library coverage.py we were able to achieve a one hundred percent coverage of the models.py, recommender.py and formula.py which are all the files which are covered in the unit tests. Some examples of unit tests which are being run can be found below.

This unit test first creates some mock data using the setUpTestData() function and then it tests methods on the APIUser class. Using assertEqual() we can verify that both methods are working as intended and are providing the correct result. Both these unit tests are relatively simple but if we were to accidentally change the functionality of either of these methods we would be alerted as these unit tests would fail

```python
# Unit tests for the APIUser class
class APIUserMethodTests(TestCase):

    # Initial set up of data
    # Creates 1 instance of the class APIUser
    def setUpTestData():

        APIUser.objects.create(
            username='dalye54',
            first_name='Eoin',
            last_name='Daly',
            email='eoin.daly54@mail.dcu.ie',
            password='password'
        )

    # Test to validate the get_id() method
    # Expected result is that the id 1 is returned from the method
    def test_get_id(self):

        user = APIUser.objects.get(id=1)
        id = user.get_id()
        correct_id = 1
        self.assertEqual(id, correct_id)

    # Test to validate the get_username() method
    # Expected result is that the username 'dalye54' is returned from the method
    def test_get_username(self):

        user = APIUser.objects.get(id=1)
        username = user.get_username()
        correct_username = "dalye54"
        self.assertEqual(username, correct_username)
```

Fig. 35 - Model Testing Code

The unit tests for the formula.py file are designed to test the haversine() function which is for determining the distance between two points on a sphere. We use this formula to get the distance between the user and a restaurant using the longitude and latitude of both and the sphere is Earth. To test this function we put in two sets of points and validate that the returned value is what is expected, in the event these tests fail we would know to examine what has been changed in the file.

```python
# Formula file testing
class FormulaTests(TestCase):

    # Test to validate the haversine function is working correctly
    # Expected result is to have 1568.520556798576 returned
    def test_haversine_formula(self):

        distance = haversine((0,0),(10,10))

        self.assertEqual(1568.520556798576, distance)

    # Expected result is to have 0 returned
    def test_haversine_formula_0_edge_case(self):

        distance = haversine((0,0),(0,0))

        self.assertEqual(0, distance)
```

Fig. 36 - Formula Testing Code

These unit tests are extremely important to the project as they have alerted us to when we have made changes to models that would break their original functionality. Without these unit tests we would have encountered more issues in developing the project.

6.1.2 Sentiment Analysis

Our main method of testing the functionality of the sentiment analysis system is by using the unit test library in Python to create assert test cases, these test cases are run in the CI/CD pipeline after every time there is an update on our Gitlab repo, this ensures that no new unexpected issues arise. The unit tests include tests such as:

- Given a piece of text, provide the expected sentiment - e.g. “I love this restaurant” is clearly a positive sentiment and should therefore return 1
- Given a piece of text with emojis, provide the expected sentiment - e.g. smiley face emoji should return 1
- Given a piece of text containing special symbols, provide an expected sentiment

Finally, for each model and vectorization type, we run unit tests to check that given a piece of text they will return a sentiment score. Below you will find an example of the unit test.

```python
def test_cv_logistic_regression(self):
    self.assertIn(predict_sentiment("I love this restaurant", 'cv', 'logistic_regression'), ['1', '0'])

def test_cv_random_forest(self):
    self.assertIn(predict_sentiment("I hate this restaurant", 'cv', 'random_forest'), ['1', '0'])

def test_cv_naive_bayes(self):
    self.assertIn(predict_sentiment("I love this restaurant", 'cv', 'naive_bayes'), ['1', '0'])

def test_cv_support_vector_machine(self):
    self.assertIn(predict_sentiment("I hate this restaurant", 'cv', 'support_vector_machine'), ['1', '0'])

def test_cv_gradient_boosting_machine(self):
    self.assertIn(predict_sentiment("I love this restaurant", 'cv', 'gradient_boosting_machine'), ['1', '0'])
```

Fig. 37 - Unit tests for Sentiment Analysis

6.1.3 Frontend

Another important location to have our unit tests is in the frontend (mobile application), as this allows us to know exactly when and where specific issues within the application are occurring. Having unit tests for the frontend is a crucial component as it allows the customer to have a consistent and reliable system to use.

The testing library which we decided to use is the react native testing library and using Jest as our testing framework. All the unit tests are grouped inside folders called __tests__, these folders are included in the screens and components directories. Within these __tests__ folders we have all the unit tests and snapshots of the unit tests inside folders called __snapshots__. From our research with testing react native application, this file and folder layout seemed to be most common layout. When creating these unit tests we relied on resources such as the react native testing documentation and [25].

We have added a number of different types of unit tests within our frontend unit testing, these include:

- Snapshot testing
- Specific render testing
- Mock testing

```javascript
it('Should have a snapshot', () => {
    // check that the screen matches the snapshot
    const tree = create(<MapFilter />).toJSON();
    expect(tree).toMatchSnapshot();
});
```

Fig. 38 - Sample unit test frontend Code

Above is an example of a unit testing for a snapshot for the CreateAccountScreen screen. Snapshot testing has been implemented as it allows us to to make sure that our UI changes are deterministic and that we are aware of new changes [26].

```
it('Should have 3 children', () => {
    // check that the screen has 3 children
    const tree = create(<CreateReviewScreen restaurantData={mockRestaurantData} />).toJSON();
    expect(tree.children.length).toBe(3);
});
```

Fig. 39 - Sample unit test frontend Code

Above is an example of a unit test checking the number of children in the component, there is also some mock data being passed to this component. Children are the number of objects enclosed with opening and closing tags when invoking components [27].

```
it('Should have name', () => {
    // render the review screen with a given data
    const { getByText } = render(<ReviewCardRestaurant data={mockData} />);
    // check that the screen has rendered
    expect(getByText(mockData.restaurant.name)).toBeTruthy();
});
```

Fig. 40 - Sample unit test frontend Code

Above is an example of a unit test regarding the rendering of the text ‘Relevance’, this test checks that the specific string has been rendered correctly.

```
// mock the review data, needed for the screen
const mockData = {
    id: '1',
    name: 'Test Restaurant',
    type: 'Italian',
    price_level: '3',
    average_rating: 4.3,
    distance_from_user: 2,
    open_or_close: 'Open',
    review_number: 1,
  };
```

Fig. 41 - Mock data used for unit tests

```
// mock the review data, needed for the screen
const mockReviewData = {
    name: 'Test',
    description: 'Nice',
    rating: 4,
    restaurant: {
        name: 'Test Restaurant'
    },
};
```

Fig. 42 - Mock data used for unit tests

Above are some examples of mock data used for specific components, this mock data is used as for certain components there is a need for provided mock data. Below we can see some variables which are necessary for the testing of other components, these variables determine the operating system and placeholder authenticating tokens.

```
global.Platform = 'ios' // used to mock the platform
global.access_global = 'token_test' // used to mock the access token
```

Fig. 43 - Variables for unit tests

6.2 API Integration Testing

API Integration testing is another crucial form of testing as it verifies that the API endpoints are performing as expected and returning the correct results. If the API endpoints were to not be performing correctly or if we had made a change that broke a specific endpoint, the API Integration testing would catch this error.

In the backend using rest_framework.test we make use of both APIClient and APITestCase which are both designed to test Django REST framework APIs. APIClient allows us to simulate making HTTP requests to the Djangos API endpoints without the server actually running and APITestCase provides extra functionality for specifically testing REST APIs. These test cases have any set up data defined in a setUpTestData() function and an instance of APIClient is initialised in a setUp() function.

Some of the code that gets run in these API integration tests use functions that make external API requests to the Google Maps API, in an effort to keep the costs down as these unit tests get run quite a lot we made the decision to mock the functions that make these external requests. We have mocked them so that they still behave the same way and return information in an identical format to how they would if they were not mocked. An example of such a function can be seen below.

```
# Mocked the external API call to the Google services
def google_maps_nearby_search_mock(tmp, tmp2):
    return {"places": [
            {"id": "google_id_583589498278432", "displayName": {"text": "A Pizza Place", "languageCode": "en"}},
            {"id": "google_id_583589498478432", "displayName": {"text": "A Pasta Place", "languageCode": "en"}},
            {"id": "google_id_583583498278432", "displayName": {"text": "A Chinese Place", "languageCode": "en"}},
            {"id": "google_id_583589198278432", "displayName": {"text": "Apache Pizza", "languageCode": "en"}},
            {"id": "google_id_5835986198278432", "displayName": {"text": "Apache Pizza", "languageCode": "en"}},
        ]}
```

Fig. 44 - Mock data for API

The function would usually make a request to the Google Maps API to get restaurants around the user, but we have mocked it so that it returns the same five restaurants to the test case in a format that is identical to how the original function would return information.

Below is an example of an API integration test broken up into two parts the first demonstrating the set up of the data. This API integration test is for testing the API endpoint “/register/review/” which is for registering reviews for a restaurant. The setup data consists of an instance of APIUser which we will be logging into and a Restaurant instance which the user will be reviewing. There is also an instance of RestaurantHours and RestaurantDay created as an instance of Restaurant requires these.

```python
# Unit tests for the Register Review API
class APIRegisterReviewTests(APITestCase):

    # Initial set up of data
    # Creates 1 instances of the class APIUser
    # Creates 1 instances of the class Restaurant
    # Creates 1 instance of the class RestaurantDay
    # Creates 1 instance of the class RestaurantHours
    def setUpTestData():

        APIUser.objects.create(
            username='dalye54',
            first_name='Eoin',
            last_name='Daly',
            email='eoin.daly54@mail.dcu.ie',
            password='password'
        )

        RestaurantDay.objects.create(
            open=False
        )

        restaurant_day = RestaurantDay.objects.get(id=1)

        RestaurantHours.objects.create(
            monday=restaurant_day,
            tuesday=restaurant_day,
            wednesday=restaurant_day,
            thursday=restaurant_day,
            friday=restaurant_day,
            saturday=restaurant_day,
            sunday=restaurant_day,
        )

        restaurant_hours = RestaurantHours.objects.get(id=1)

        Restaurant.objects.create(
            google_id='google_id_583589498278432',
            longitude=43.78,
            latitude=17.35,
            name='A Pizza Place',
            address='Dublin',
            type='pizza',
            price_level='PRICE_LEVEL_INEXPENSIVE',
            allows_dogs=True,
            delivery=False,
            dine_in=True,
            good_for_children=True,
            good_for_groups=False,
            outdoor_seating=False,
            average_rating=0,
            hours=restaurant_hours
        )
```

Fig. 45 - API Registration Test Code

To begin the test an instance of APIClient is started which will allow the HTTP request to be made to the application. Once this is completed the individual test case begins, the test uses force_authenticate() to bypass the authentication process and to set itself as the created user. Using .post() a review is sent into the API endpoint in

json format and we verify using the response that a review has been created. If this succeeds then the review is retrieved from the backend and using various assertEqual() it is checked to make sure that the data in the backend is identical to the data that was sent.

```
# Set up data for each test
# Creates 1 instances of the APIClient
def setUp(self):

    self.client = APIClient()

def test_post_api_register_review(self):

    # get the user we will be logging into
    user = APIUser.objects.get(id=1)

    # data to create new review
    self.client.force_authenticate(user=user)

    # data to create new review
    data = {"restaurant": "1", "rating": "4", "description": "Was a really good time and i enjoyed my food a lot."}

    # post the data to the url in json format
    response = self.client.post("/register/review/", data=data, format="json")

    # verify the response data has been created
    self.assertEqual(response.status_code, status.HTTP_201_CREATED)

    # get the created user
    review = Review.objects.get(id=1)

    # verify the data in the review is the same as what we sent
    self.assertEqual(review.get_user(), user)
    self.assertEqual(review.get_restaurant(), Restaurant.objects.get(id=1))
    self.assertEqual(review.get_rating(), int(data['rating']))
    self.assertEqual(review.get_description(), data['description'])
```

Fig. 46 - Unit tests for backend

Overall these API integrations tests have been extremely important to our application as when making changes to the views.py or serializers.py and then running these tests we have been alerted to issues in the code not performing in the desired format and been able to make the necessary fixes before merging with master.

6.3 CI/CD Pipeline

We have set up a CI/CD pipeline that has two major components, continuous integration and continuous deployment. Setting up a CI/CD pipeline is a crucial part of any project as it helps us avoid bugs, failures and the maintainability of our code while keeping a continuous cycle of software development and updates [28].

The continuous integration of our application has the responsibility of running all the unit tests and integration tests when code has been committed and pushed to the gitlab repository. This is accomplished by setting up a python image to version 3.10.12 as that is what we have programmed with and installing all the required packages that we have used as part of our development which can be found in requirements.txt. Once this has been set up the three test jobs are run. These test jobs run all the unit tests and integration tests that we have designed and developed as part of testing the application.

If these test jobs are run successfully the pipeline moves to the continuous deployment component, which will only run if the branch is master. The continuous deployment, deploys the Django backend to a Heroku server and this is done by first using an image of ruby 3.3 and downloading ‘dpl’ which is used for simplifying the deployment process. Once this is completed the Django backend is deployed using both a HEROKU_APP_NAME and HEROKU_API_KEY which are saved variables and hidden or security. Once this is finished the Django backend will be deployed on our Heroku server.

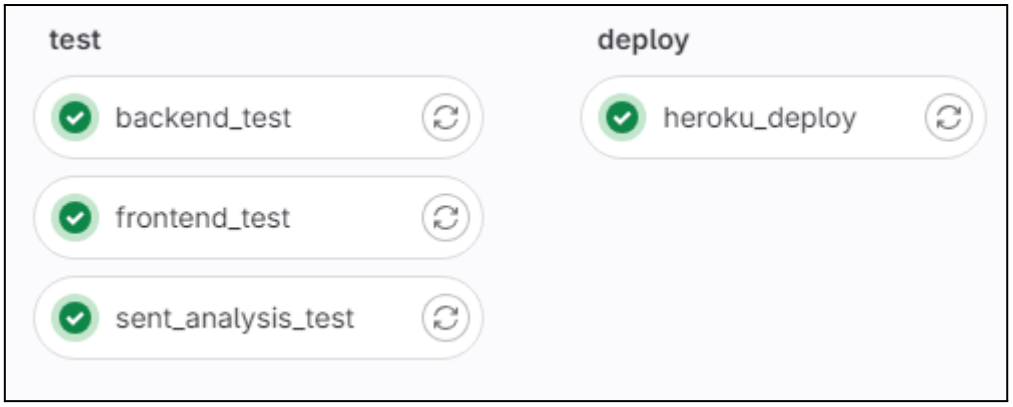

Fig. 47 - Jobs in Gitlab Pipeline

Above we can see all the specific jobs within our CI/CD pipeline, we have backend tests, frontend tests, sentiment analysis tests and we have a job to deploy the backend to Heroku.

```
sent_analysis_test: # job name - for sentiment analysis tests
  stage: test
  image: python:3.10.12
  before_script:
    - cd src/sentiment_analysis/
    - pip install -r requirements.txt
  script:
    - python3 tests_sentiment_analysis.py
```

Fig. 48 - Example script of job

Above we see an example of the sentiment analysis code for its job, it has an image for python, before the test are ran it will move into the sentiment analysis folder within the code base and install all necessary requirements to perform the sentiment analysis, finally the script for running the tests is ran.

6.4 Sentiment Analysis

6.4.1 Data

6.4.1.1 Dataset Description

The initial dataset which was being used for the testing and training of the sentiment analysis model was a tweet dataset which held the textID, the tweet text and its respective tweet sentiment (‘positive’ or ‘negative’). Due to the nature of our application and its field, we felt that using a tweet dataset [29] would not be the most appropriate dataset when relating to food location reviews, therefore we pivoted and found a dataset which uses a yelp dataset modified by Xiang Zhang to provide the sentiment of each review. This Yelp dataset contained two .CSV files, ‘test’ and ‘train’. The test and train csv files both have the same format which is two columns, the first column holds values of ‘1’ or ‘2’ (to signify the sentiment of the review), ‘1’ indicates a negative sentiment meanwhile ‘2’ indicates a positive sentiment, meanwhile the second column holds the text review. Test.csv holds 37999 unique values and the train.csv holds 559999 unique values.

6.4.1.2 Ethical Considerations

–

6.4.1.3 Data Cleaning & Preprocessing

6.4.1.3.1 Unused Files

We have decided to only implement the train.csv file from this Yelp dataset as it contains over 500000 unique values. To make use of all data accessible to us, we could combine the training dataset and the test dataset, as in our testing we are using an 80/20 split of the chosen dataset.

6.4.1.3.2 Dataset Transformation

Our Yelp dataset did initially include specific column titles and was using the first rows attributes as the column titles, to make this more readable we decided to use the column name ‘sentiment’ for the dataset values ‘1’ or ‘2’ and the ‘text’ column for the text reviews

Secondly, we decided to transform the sentiment data from ‘1’ or ‘2’ to a text-based description for better recognition and understanding of the dataset. We achieved this by converting all initial ‘1’ to ‘negative’ text descriptions and all initial ‘2’ to ‘positive’ text descriptions.

Lastly, as we were using a tweet dataset initially for the sentiment analysis, we decided to rename the column to have a standardised naming convention, in this case, we changed the tweet_text column name to text.

6.4.1.3.3 Understanding the dataset

This Yelp dataset has a near-even split with positive and negative reviews, this means that this Yelp dataset has been balanced and will help prevent the sentiment analysis model from being biased [30]. Shown below is a bar plot of the negative and positive number of reviews.

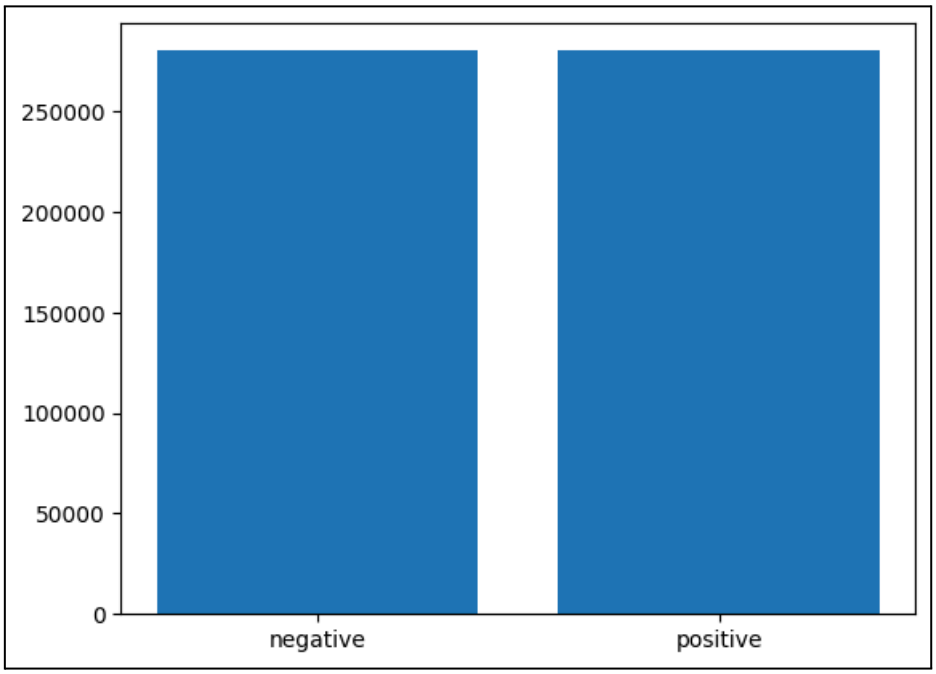

Fig. 49 - Bar plot for Yelp dataset

sentiment	text
negative	279999
positive	280000

Fig. 50 - Displaying dataset data in number format

An interesting way to visualize the positive and negative text data is through a word cloud, below we can see the most common words for reviews which have been given a positive and negative sentiment. From the below figures, we can see in the large text that there are some common words which occur in both types of sentiment, followed by smaller text which varies more.

Fig. 51 - Positive word cloud

Fig. 52 - Negative word cloud

6.4.1 Train Test Split

When training our model we decided to use an 80/20 (training/testing) split as these percentages are most commonly used [31]. Using this train test split procedure allows us to validate and simulate how the model would perform on any unseen data. Using the same data for training and testing would not simulate how the model would perform on any new data given [32]. The python notebook for all the testing and validation relating to our final sentiment analysis system can be found in:

```
src/
     sentiment_analysis/
          ipynb_documentation_testing_cleaning_files/
               yelp_testing_doc_sentiment_analysis.ipynb
```

6.4.2 Vectorization Testing

In our sentiment analysis, we decided to use TF-IDF as our default vectorization method as from the research and testing of the methods we came to the conclusion that TF-IDF provided more accurate final results, below is a graph comparing all the models being used with the two vectorization methods.

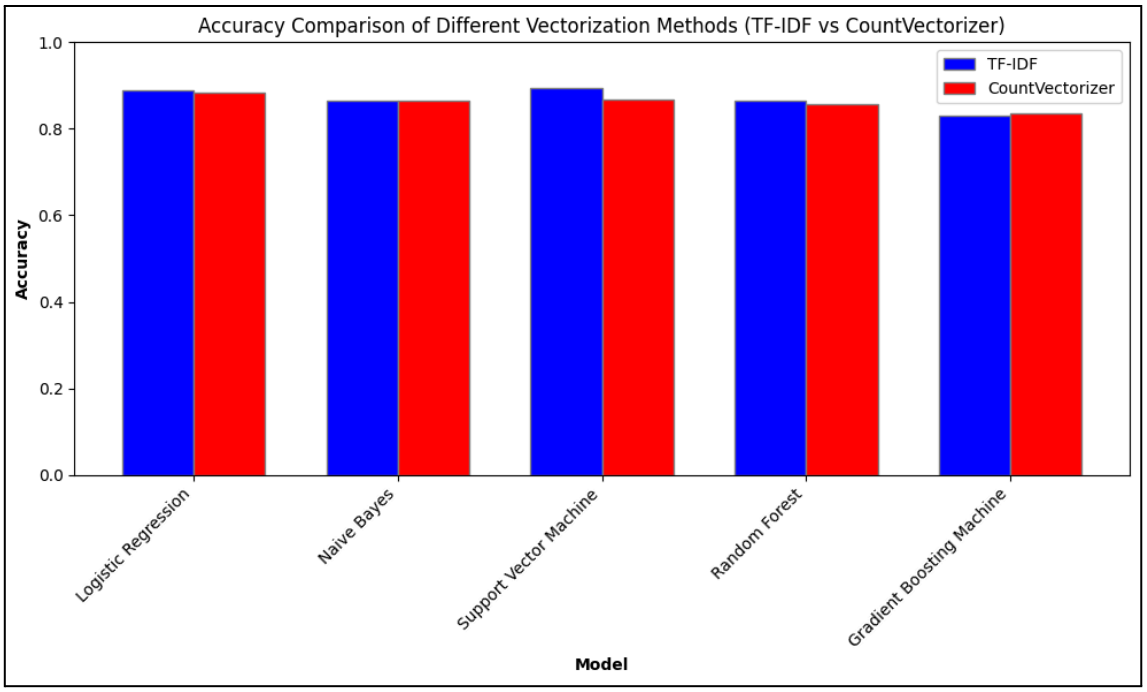

Fig. 53 - Bar chart comparing the accuracies of vectorization types

As we can see above the large majority of models allow for a higher accuracy when using TF-IDF and not Count Vectorization as the vectorization method.

6.4.3 Model Testing

For testing the performance and accuracy of our models we compared the accuracy, confusion matrices, precision, recall, F1 scores and ROC AUC using the same data for each of the models and metrics. To display these results we used bar charts and

confusion matrices to display the number of predicted sentiments for each of its squares. We decided to use the top 10000 rows as this would generate a good-sized dataset, while still allowing us to complete the testing within a reasonable time frame.

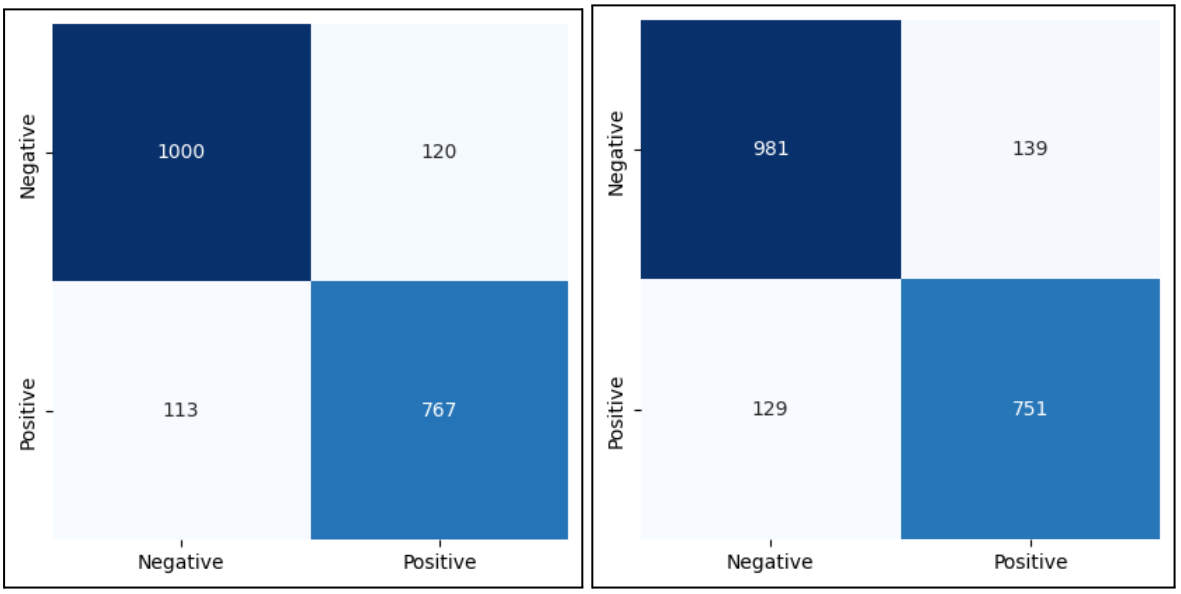

Fig. 54 - Conf Matrix CV Logistic Regression & CV Naive Bayes

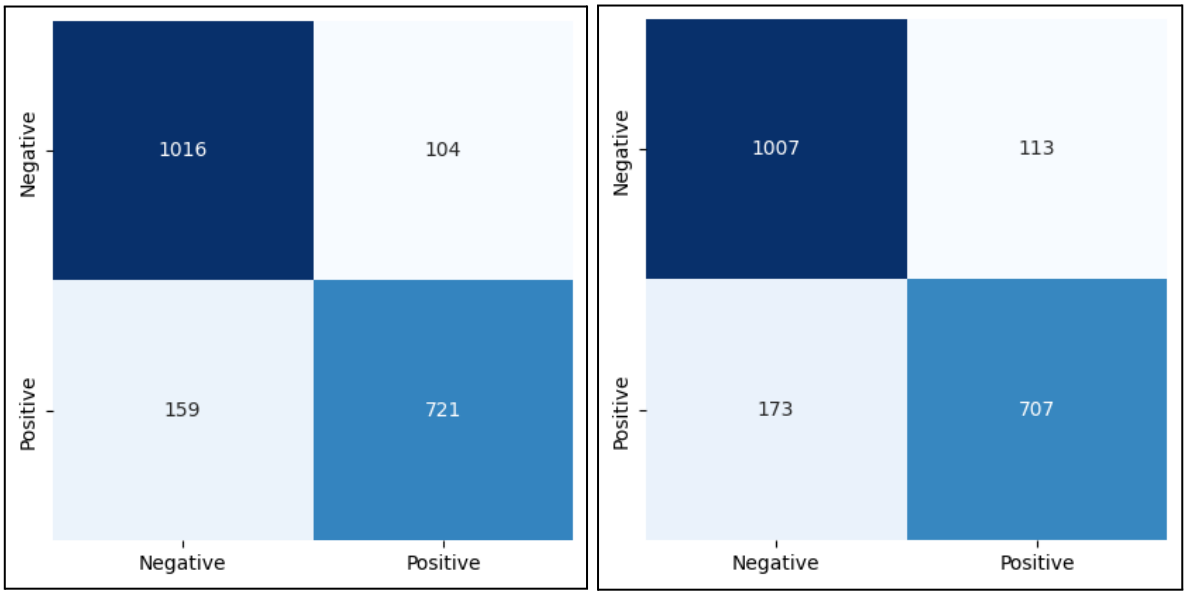

Fig. 55 - Conf. Matrix CV Support Vector Machine & CV Random Forest

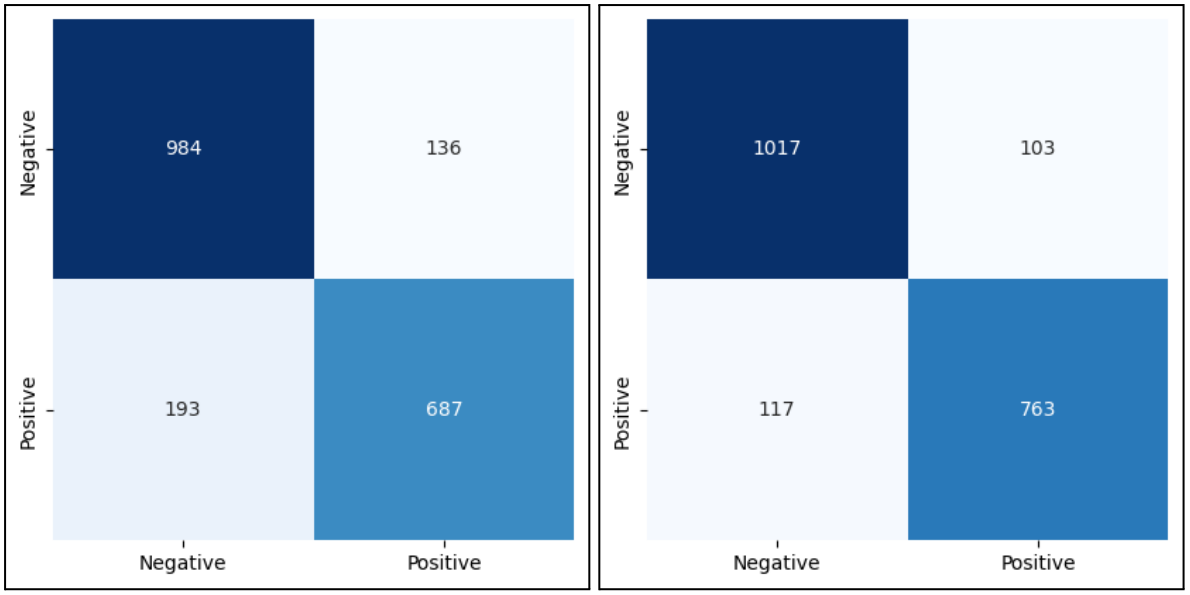

Fig. 56 - Conf. Matrix CV Gradient Boosing Machine & TF Logisic Regression

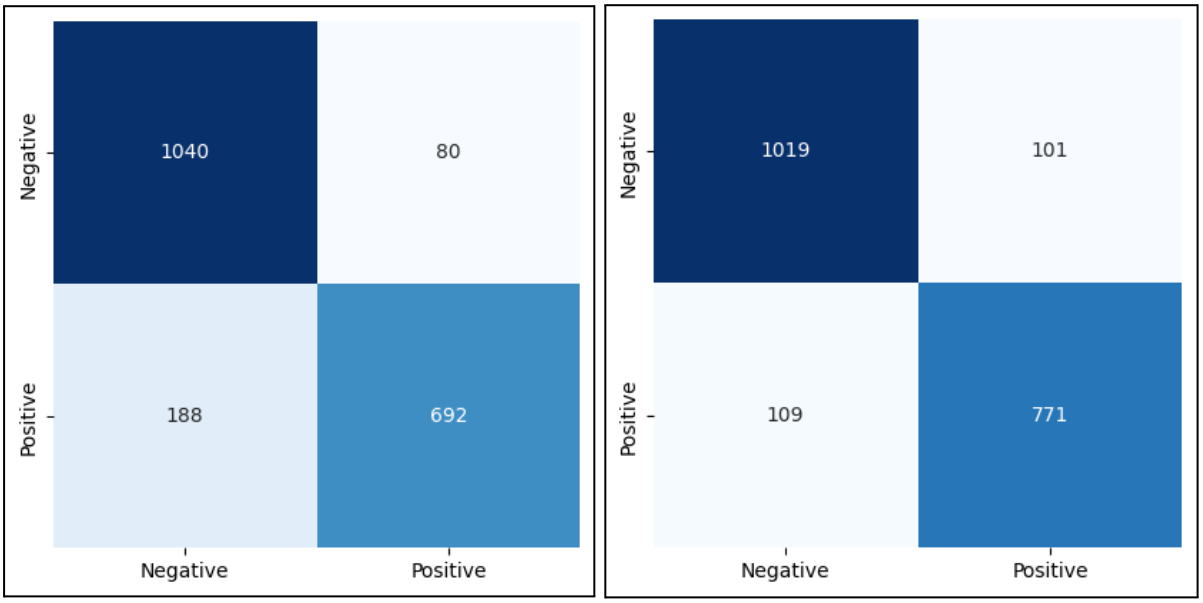

Fig. 57 - Conf. Matrix TF Naive Bayes & TF Support Vector Machine

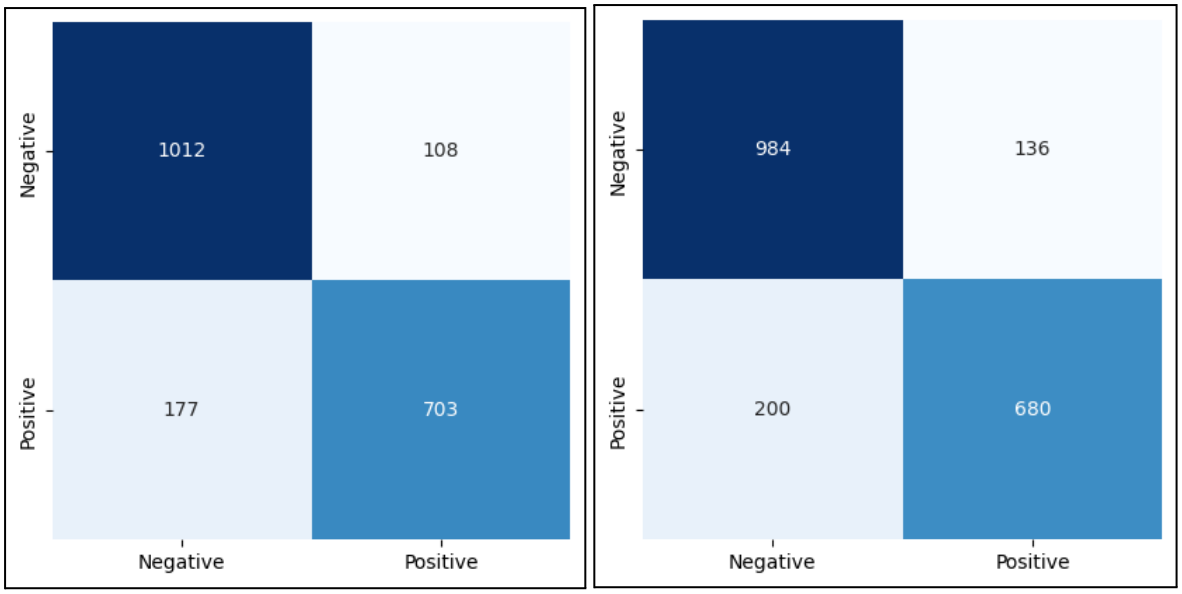

Fig. 58 - Conf. Matrix TF Random Forest & TF Gradient Boosting Machine

```
--- Count Vectorization ---
Logistic Regression:
 [[1000  120]
 [ 113  767]]
Total false postives and false negatives:  233
Total true postives and true negatives:  1767
Naive Bayes:
 [[981 139]
 [129 751]]
Total false postives and false negatives:  268
Total true postives and true negatives:  1732
Support Vector Machine:
 [[1016  104]
 [ 159  721]]
Total false postives and false negatives:  263
Total true postives and true negatives:  1737
Random Forest:
 [[1007  113]
 [ 173  707]]
Total false postives and false negatives:  286
Total true postives and true negatives:  1714
Gradient Boosting Machine:
 [[984 136]
 [193 687]]
Total false postives and false negatives:  329
Total true postives and true negatives:  1671
```

Fig. 59 - CV results on all models

```
--- Tfidf Vectorization ---
Logistic Regression:
 [[1017  103]
 [ 117  763]]
Total false postives and false negatives:  220
Total true postives and true negatives:  1780
Naive Bayes:
 [[1040   80]
 [ 188  692]]
Total false postives and false negatives:  268
Total true postives and true negatives:  1732
Support Vector Machine:
 [[1019  101]
 [ 109  771]]
Total false postives and false negatives:  210
Total true postives and true negatives:  1790
Random Forest:
 [[1012  108]
 [ 177  703]]
Total false postives and false negatives:  285
Total true postives and true negatives:  1715
Gradient Boosting Machine:
 [[984 136]
 [200 680]]
Total false postives and false negatives:  336
Total true postives and true negatives:  1664
```

Fig. 60 - TF-IDF results on all models

The above images show us all the different models and vectorization types results when showing the data of positives, negatives, false positives and false negatives. Looking at the numbers we see:

- Model and Vectorization type with highest number of false positives and false negatives: Gradient Boosing Machine using Count Vectorization
- Model and Vectorization type with lowest number of false positives and false negatives: Support Vector Machine using TF-IDF
- Model and Vectorization type with the lowest number of true positives and true negatives: Gradient Boosting Machine using TF-IDF
- Model and Vectorization type with the highest number of true positives and true negatives: Support Vector Machine using TF-IDF

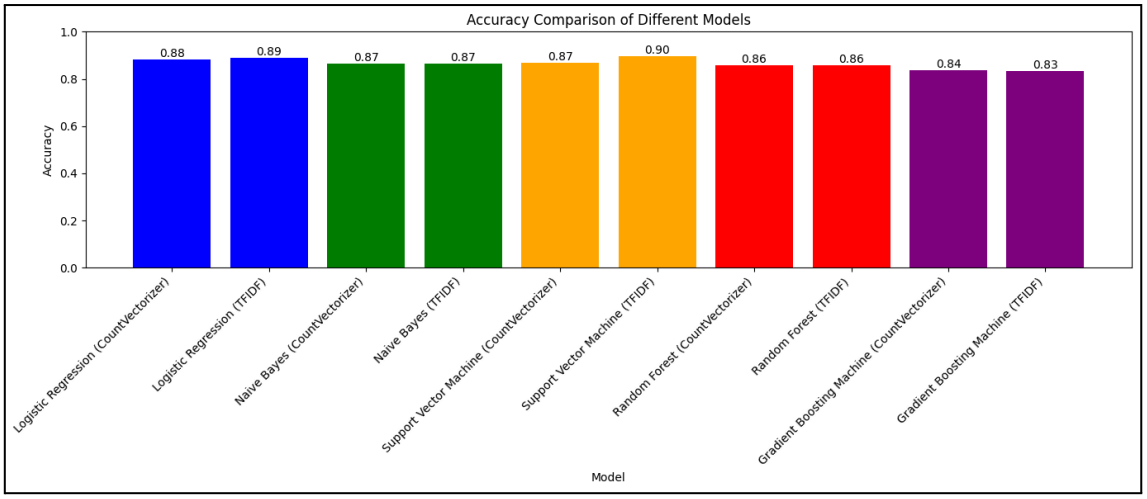

Fig. 61 - Accuracy of models

Looking at this data, we see an interesting pattern as for both vectorization types the TF-IDF method outperformed the Count Vectorization in all models bar the Gradient Boosting Machine. When looking at the models we see that the best performing model is the Support Vector Machine using TF-IDF and the Logistic Regression model when using Count Vectorization. To conclude all models performed at a good accuracy rate of 83%+ and the top model performed at 90% accuracy.

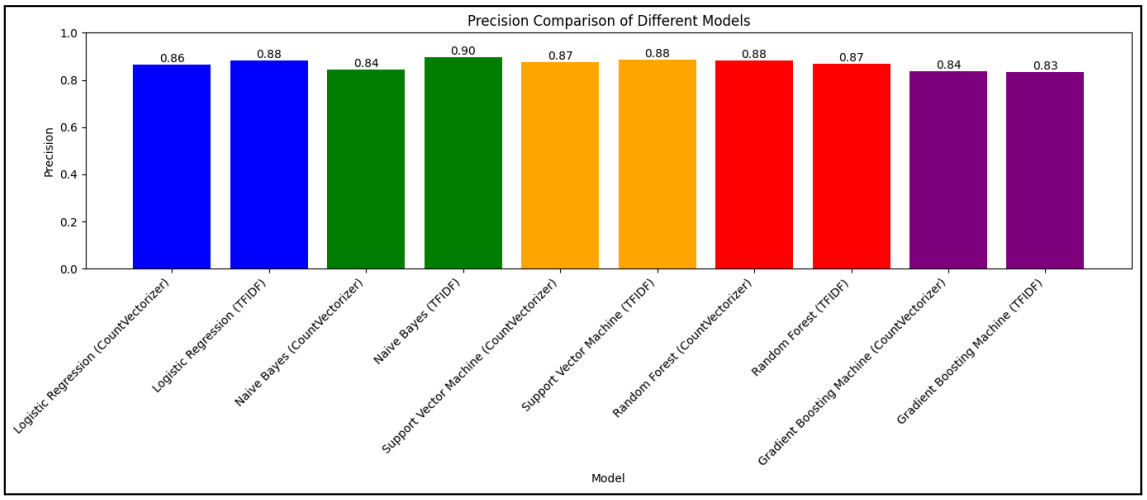

Fig. 62 - Precision of models

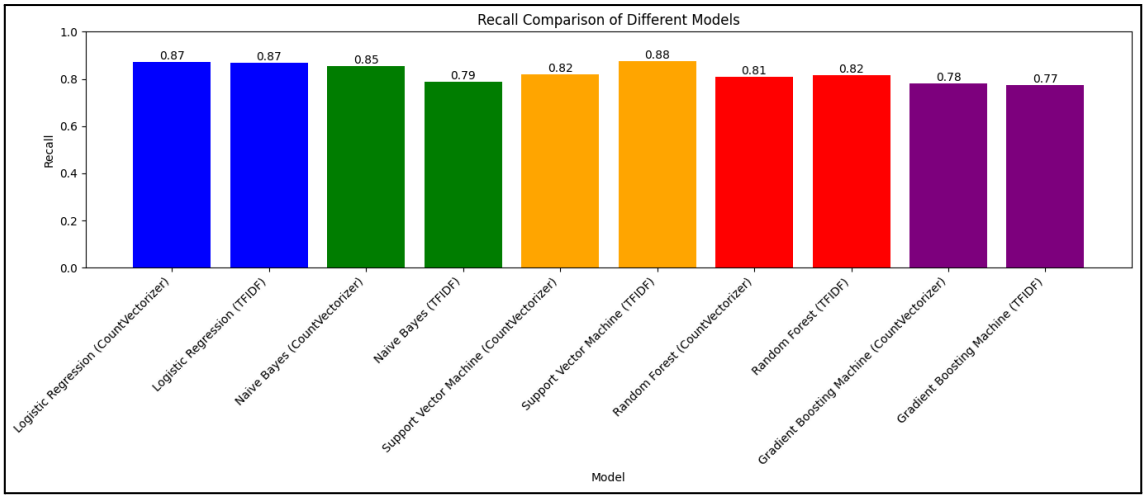

Fig. 63 - Recall of models

Looking at the precision and recall, generally speaking, the models have higher precision compared to each of their recall scores. Logistic Regression stands out as the model which is most consistent with precision and recall, meanwhile, Naive Bayes is the model which has the most fluctuation between precision and recall.

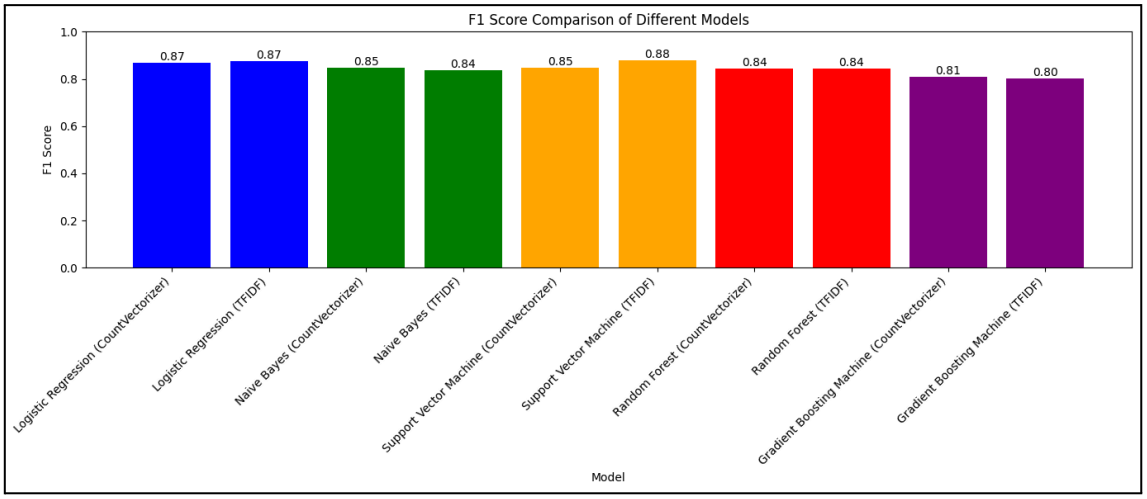

Fig. 64 - F1 Score of model

When looking at the F1 scores which are essentially using the best of both worlds of precision and recall, we see that yet again, similar to using accuracy, the best performance comes from the Support Vector Machine model using TFIDF as its vectorization method.

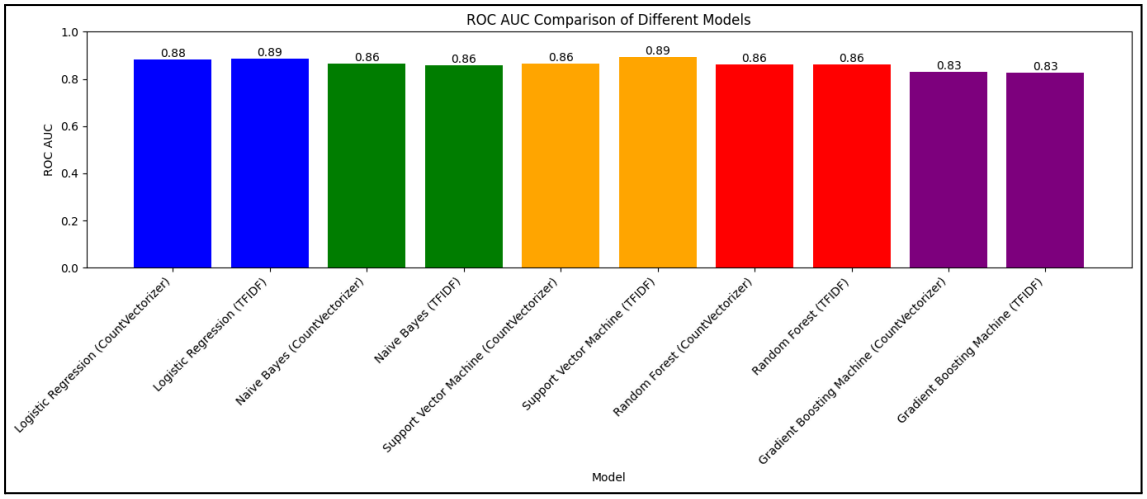

Fig. 65 - ROC AUC of models

Using the ROC AUC metric we see that both Logistic Regression and Support Vector Machine models perform identically and are by far the best performers. To conclude the default model we are using within our sentiment analysis is the Support Vector Machine (TFIDF) as it has shown us it has been the best-performing model when using the different metrics.

Regarding the time consumption of running the training of the model we decided to keep the trained model in a file using the Joblib library which allows us to save any models which are run, this speeds up the process immensely when running the sentiment analysis on any given unit test and text.

6.4.4 Reasoning behind our evaluation metric choices

When researching the possible evaluation metric to be used on the machine learning models we implemented, we noticed the most popular metrics/methods used are accuracy, confusion matrices, ROC-AUC (Receiver Operating Characteristics Area Under Curve) and F1 scores. Using accuracy can allow for a great way to see how the models have performed, it works well only if the dataset is balanced, this is because accuracy is essentially the mean. In our case, the dataset chosen has a near 50/50 split of positive and negative reviews which allows us to use and rely on the accuracy scores of our models [33].

Confusion matrices allowed us to gain a different type of perspective on the models' performance, showing us the number of items correctly predicted in a true positive, true negative, false positive and false negative way. Confusion matrices are the foundation of many other evaluation metrics [33].

Area Under the Curve (AUC) Receiver Operating Characteristics (ROC) is a widely used metric associated with binary classification problems. It plots a True Positive Rate (the proportion of positive data points correctly considered positive), True Negative Rate (the proportion of negative data points correctly considered negative)

and the False Positive Rate (the proportion of negative data points that are considered positive). As we can see this metric uses the same metrics as the confusion matrices [33].

F1 score is another popular metric for the evaluation of models as it combines and “takes the best” of both, precision and recall [33]. Precision is the number of correct positive results over the number of total predicted positive results, Recall is the number of correct positive results over the total number of all relevant samples. The F1 score gives us a value to determine how robust and precise our model is [33]. Using all these models allows us to gain a far better holistic view of all the models' performance.

6.5 Recommender Systems

The goal is to enhance the overall experience of using our mobile application, we believe this will be achieved by generating personalised recommendations for each individual user, that will be tailored to their specific needs, it will encourage them to interact more with the app. The higher the accuracy of the recommendations the more likely a user is to continue to use our mobile application. With the recommendation system in the mobile application, users won’t need to spend an outrageous amount of time trying to research and locate the perfect restaurant, as this research and calculations will be performed by the recommender system.

To verify that we have successfully completed our goal of enhancing the overall user experience of using our mobile application by incorporating a recommendation system we will use evaluation metrics to evaluate the effectiveness of our model. These evaluation metrics will be able to accurately tell us how our model is performing.

6.5.1 Data

The dataset being used for the training and testing of this recommendation system is a Yelp dataset, the link can be found at [34]. Yelp has made this dataset public specifically for the development of academic projects as this dataset is available solely for academic research purposes. The dataset contains 5 json files which consists of 150,346 businesses, 1,987,897 users, 6,990,280 reviews, 908,915 tips and 131,930 check-ins. A description of the dataset summary can be found in Table 1.

The restaurant file and user file both have unique identifiers for each instance which will allow us to merge them later on into a single file which would tell us which users have left reviews for specific restaurants.

Table 1. Dataset Summary

Data Source	Size [MB]	Number of FIles	File Type	Number of instances	Number of Attributes
yelp_academic_dataset_business.json	118.9	1	JSON	150,346	14
yelp_academic_dataset_checkin.json	287.0	1	JSON	131,930	2
yelp_academic_dataset_review.json	5,300	1	JSON	6,990,280	9
yelp_academic_dataset_tip.json	180.6	1	JSON	908,915	5
yelp_academic_dataset_user.json	3,400	1	JSON	1,987,897	22

6.5.2 Data Cleaning

6.5.2.1 Unused Files

Three of the files in the dataset will be left unused and these files will be the ‘yelp_academic_dataset_user.json’, ‘yelp_academic_dataset_checkin.json’ and ‘yelp_academic_dataset_tip.json’. These files won't be needed as part of the training and testing of the model as they are redundant in the information that they offer.

6.5.2.2 Business File

6.5.2.2.1 Data Preprocessing

The original business dataset file can be found as “clean_dataset/business.csv”. The data cleaning for the business file started by reducing the dataset due to its size and so that the dataset is more manageable in volume and to improve efficiency for testing. To do this the focus of the recommender was set to only one geographical city. The focus of the training and testing will be on the city ‘Philadelphia’ which has 14569 businesses.

An issue in the business file is that Yelp is not just a website for restaurants and has other businesses such as Health and fitness, Local retailers and Home services. Using the category attribute in the dataset we can filter so that any business without ‘Restaurants’ in their categories will be deleted and won’t be used as part of the recommender. We are now left with 5852 restaurants in the dataset after completing the preprocessing.

6.5.2.2.2 Data Transformation

An issue with the format of the business ids in this dataset were there format, an example output of a random 5 business ids can be seen as Figure 66. This format of ids will cause issues later when trying to fit to some of the models as they require ids to be in an integer format.

By transforming this column we will convert all the business id's to a simple integer id. This will reassign each business's id to an integer between one and 5852 as there are only 5852 unique businesses in the dataset. An example output of a random 5 business ids of the new format can be seen as Figure 67. The new format will work with all the machine learning algorithms.

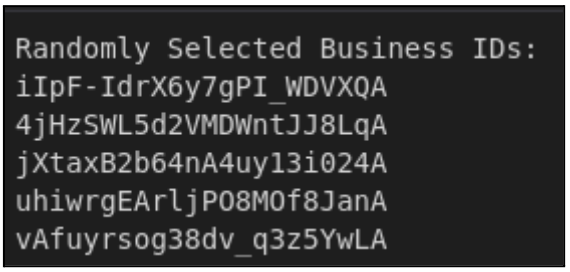

Fig. 66 - Original business id format

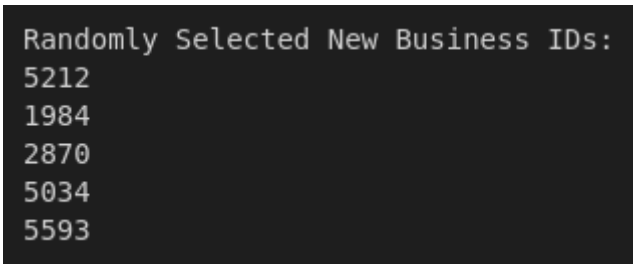

Fig. 67 - New business id format

Another issue with the format of the business dataset is the categories column in the dataset which is saved as a string and contains all the categories which can be used to describe the dataset. The issue in this current format is where comparing the categories of restaurants it will only be able to compare the whole string such as, Restaurant A has "Chinese, Takeaway, Delivery" and Restaurant B has "Chinese, Takeaway, Delivery, Table Service", comparing both Restaurant A and B we can identify how they have similar categories but as they are not identical they wouldn't be considered similar.

To overcome this issue we will make use of One-hot Encoding, which is the conversion of categorical information into a format that may be fed into machine learning algorithms to improve prediction accuracy [35]. In our case it will turn each unique category into its own column so if a restaurant has a category such as Chinese, in the Chinese column it will now have a '1' to identify that it is relevant to

that restaurant and all the categories that don't appear will have a value of ‘0’. This change will also make the dataset more compatible with many machine learning algorithms that do not work with categorical data.

Similarly the attribute column will also be one-hot encoded for the same reasons outlined above, but the attributes column has more information as it is saved as a dictionary such that the key represents an attribute and the value can be either “True” or “False”. One hot encoding this column will just make it such that if the attribute has the value of true in that column it will have ‘1’ and if the value is false or the attribute doesn’t appear it will have the value of ‘0’.

6.5.2.2.3 Cleaned File

The cleaned business file can be seen as Table 2, the number of instances has been reduced from 150,346 to 5852, the size in megabytes has been reduced from 118.9 to 1.1 and the number of attributes has been increased from 14 to 73.

Table 2. Business dataset Summary

Data Source	Size [MB]	Number of FIles	File Type	Number of instances	Number of Attributes
processed_dataset/business.csv	1.1	1	CSV	5852	73

6.5.2.3 Review File

6.5.2.3.1 Data Preprocessing

The original business dataset file can be found as “clean_dataset/review.csv”. The data cleaning for the review files starts by loading in the cleaned business file to get all the unique ‘old_business_id’, as all the reviews for businesses that are not in the cleaned business file can be discarded and dropped from the reviews dataset. This is done by checking each review’s business_id and if it is not one of the unique ids then the row can be dropped.

An issue that was encountered was that users can have more then one review for a specific business. This is a concern as it can impact the bias of the recommender if a user can continuously create reviews for the same business which could skew the overall performance. To counter this we will only be taking into consideration the users most recent review, when they have more then one for a specific business.

6.5.2.3.2 Data Transformation

Each instance of a review has a foreign key of a business_id, and as we have updated all the business_ids to a new format we will also update the business_id on each instance of a review. This is important as without completing this we would not be able to link a review to a specific business.

The unique review_id's are also in a format similar to the original business_id and just as we converted them to integers we also convert the the review_id to the new format of just having integers going from 1 to 665,775

6.5.2.3.2 Cleaned File

The cleaned review file can be seen below as table 3, the number of instances has been reduced from 6,990,280 to 665,775, the size in megabytes has been reduced from 5,300 to 26.7 and the number of attributes has decreased from 9 to 7.

Table 3. Review dataset Summary

Data Source	Size [MB]	Number of FIles	File Type	Number of instances	Number of Attributes
processed_dataset/clean_review.csv	26.7	1	CSV	665,755	4

6.5.3 Collaborative Filtering Models

This section will outline the models that were developed as part of testing and designing the collaborative filtering approach. We made the decision to investigate and build different models for different python libraries in an attempt to better understand and develop a better overall recommender model. The 4 libraries that we investigated and designed models for were Fastai, Sklearn, LightFM, Surprise. All 4 models will be built such that they all have an identical required input and output outlined below. It is important that they all have the same input and output as this will be required late when creating the hybrid recommendation system. For testing these models we used training and testing using an eighty percent split, such that the data is trained on eighty percent and tested on the remaining twenty percent.

Input - This model will take an input of a user id, as it is required to see which user we are predicting which restaurants they would be interested in

Output - The output of this model will be our prediction of which restaurant we have calculated that the input user will rate highest.

6.5.3.1 Baseline

6.5.3.1.1 Description

As part of our comparison between different libraries and models in an effort to get the overall best performing model we need a sufficient baseline to compare all models against. A baseline model will be a very basic algorithm to serve as a base point that all models should be able to efficiently perform better than. This will allow us to assess the more advanced machine learning models.

For our baseline we will train it on a dataset so that it learns the frequency of each rating, i.e. 5's appearing 50% of the time. Once the model is trained on the frequencies it will then use these frequencies to try to predict on a test set, for the example given before it will predict that half of the ratings are 5. This model is making its frequencies on the observed frequencies.

6.5.3.1.2 Model

The model first splits the data into training and testing. With the split data we want to loop through the training data and count how often each value is appearing, adding the occurrence to a dictionary with the rating as the key and count as the value. Once completed we will have the amount of times each value occurs in the data.

With these values we then get the total number of rows in the training dataset. Using the total number of rows we divide each of the ratings occurrences by the number of rows to get a percentage of how often the rating occurs. Then using the random module of python we use the choices function using the percentages as the weights and predict values for each row in the test dataset. Once completed we save these values to a list.

Finally we have both the real ratings and the predicted values using our baseline model. Using an evaluation metric we can compare the overall performance of this model and have that value as a baseline for all the future models that we will be testing. Every future model should be performing significantly better than our frequency baseline.

6.5.3.2 FastAI

6.5.3.2.1 Description

Fastai is a deep learning library which provides practitioners with high-level components that can quickly and easily provide state-of-the-area results in standard deep learning domains, and provide researchers with low-level components which can be mixed and matched to build new approaches [36]. The library is built on top of another well known python library PyTorch which is a machine learning framework. Fastai provides a simplistic process to training and this is due to its design of being built using different components based on whatever you're trying to develop.

While Fastai has many different components which can be used for all types of machine learning, we are only focusing on its Collaborative filtering functionality. The library offers a streamlined process to setting up the recommender through its use of CollabDataLoaders class, which enables us to easily assign the dataset to an instance of this class and make the training, testing and evaluation easier.

The CollabDataLoader instance then works with the collab_learner function which creates an instance of the collaborative filtering model. This is where the dataset is passed in as a parameter followed by other parameters, n_factors, range. This instance of the collaborative filtering model is then trained on the data and can be evaluated.

Overall Fastai is a powerful deep learning library which builds on PyTorch and creates an easy to use, simple, interface that enables us to build a complex machine learning model with relative ease.

6.5.3.2.2 Model

The model consists of two separate parts, the initial model and the hyperparameter tuning. We initially begin with creating a data loader, which is the type of data that is required to use the Fastai library. Using the inbuilt function CollabDataLoaders and passing in the training dataset the training dataset is converted into the correct format to be used for the library. The bs value in the function represents batch size, which is the number of training interactions analysed for each iteration of the model. The higher the value of the batch size the more memory and computation needed to run the model.

Using the previously defined data loader we can pass this as a parameter into the collab_learner, taking the training dataset as an input to the machine learning function. Two other parameters, n_factors and y_range are also passed as y_range is the range of values we would expect the model to predict, which for the model will be between 1 and 5, n_factors is the amount of factors used to represent the relationship between users and items.

The fit_one_cycle method used on the collab_learner instance begins the learning process of the model by specifying parameters for the learning process. The first parameter 5 represents the number of times the model will go through the dataset for the training, 0.005 is the learning rate of the model and wd=0.1 is the weight decay which prevents the model from overfitting specifically to the training data and making it a poor performer to new unseen data.

Once the model is trained we can move onto the evaluation stage of the model. For this we call the CollabDataLoaders function again, but this time for the test data. With the test data in the correct format we can call the get_preds method on the trained model and pass in the test data to get the model's predictions for each of those values. The predicted values are stored and then compared against the actual value and various evaluation methods and functions are then used to compare the performance of the model.

6.5.3.2.3 Hyperparameters

Hyperparameters are parameters whose values control the learning process and determine the values of model parameters that a learning algorithm ends up learning [37]. Changing the values of the hyperparameters can have a significant impact on the performance of a given model making it better or worse compared to a model using the standard hyperparameters. To test our model with different hyperparameters we used the grid search algorithm. Grid search is the process of an exhaustive search through a manually specified subset of the hyperparameters space of a learning algorithm [38]. For our implementation we created a grid search function and passed in the parameters that we wanted to test. The parameters are described below and their values.

- n_factor_values: The amount of factors used to represent a relationship between users and items, values = 50, 100, 200
- lr_values: Learning rate of the model, values = 0.001, 0.005, 0.01
- wd_values: Weight decay of the model, values = 0.1, 0.01, 0.001

Running these alternative models takes a lot of time due to the grid search being an exhaustive search method leading us to run 3 * 3 * 3, different models with all the chosen parameters, which is 27 different models. The outcome of this testing is that we get models both with improved performance and models with worse performance than the baseline model with no hyperparameter tuning.

6.5.3.3 LightFM

6.5.3.3.1 Description

LightFM is a Python implementation of a number of popular recommendation algorithms for both implicit and explicit feedback, it makes use of both item and user metadata into the traditional matrix factorization algorithms [39]. This library is specifically designed for building hybrid models that would combine both collaborative filtering and content based filtering to one model. LightFM has various functions and features that make using it that much easier.

LightFM offers a Dataset class which takes care of the building interactions between users' items and features for both users and items. Using a pandas dataframe you can easily fit the data to an instance of the Dataset class, which will generate all the interactions between items and users. This class is designed to prepare your data so that it is in the correct format for the library.

LightFM also offers different loss functions which can be tested on the dataset. A loss function is what is used to measure the difference between what the model is predicting on the training set compared to the training set's actual values. Each of the loss functions will attempt to optimise its performance such that the model is

performing better. An advantage of LightFM is that by having different loss functions they can all be tested to see which is performing best.

Overall LightFM is a great library to use for building collaborative recommenders due to its simplistic setup by offering ease of use for formatting the data into the correct format and also by having many different loss functions to test using.

6.5.3.3.2 Model

The model consists of two separate parts, the initial models and the hyperparameter tuning of the best performing model. This differs from the Fastai approach as because we have the ability to use more loss functions in LightFM we will initially run each with their basic parameters and whichever performs best we will seek to further improve using hyperparameter optimisation.

For this implementation we first define the interactions between users, restaurants and the ratings they gave. We create this by building a sparse matrix, a matrix that has a large number of zero elements, where the user_id's and restaurant_id's are the rows and columns with the ratings being the values. This matrix is the format the data needs to be in to be used by the model. The data is split into the relevant train and test datasets split by the appropriate value of 80/20.

The model can then be fitted using the training data and the parameters for the number of epochs is set to 30, the model then begins to be trained using these parameters. Once this has been completed we can begin the evaluation of the model, which is done by calling the trained model and getting it to predict all the values in the training dataset. With these predictions we can then compare against the actual values in the training dataset getting a value for how the model is performing.

Because we are completing this for 3 separate models, whichever model performs the best at this stage we will attempt to further improve by using hyperparameters on that model. We have only chosen to do this with the best performing model as completing this for all 3 would be very computationally expensive and take a large amount of time.

6.5.3.3.3 Hyperparameters

From the evaluation of the 3 models we found that the model using the WARP loss function was performing best, so this was the model that we will attempt to optimise the hyperparameters for. Once again we defined a grid search function that we could pass in the hyperparameters as a parameter too and run the model with those chosen hyperparameters so it could identify the best performing model. The parameters are described below and their values.

- no_component_values: The amount of components used to represent a relationship between users and items, values = 30, 50, 64
- learning_rate_values: Learning rate of the model, values = 0.01, 0.05, 0.1
- item_alpha_values: Controls the regularisation of items, values = 0.0001, 0.001, 0.01
- user_alpha_values: Controls the regularisation of users, values = 0.0001, 0.001, 0.01
- epochs_values: The number of times the model runs over the training dataset, values = 20, 30, 50

Once each variation of the model has finished running we can analyse and see which model and hyperparameters performed the best overall for the LightFM library, this model will then be compared against the other models that we will be looking at.

6.5.3.4 Sklearn

6.5.3.4.1 Description

Sklearn is a machine learning library for Python, which features various classification, regression and clustering algorithms like random forests, gradient boosting and k-means [40]. Sklearn was built on top of three other python libraries, NumPy, SciPy and matplotlib, which enables Sklearn to use many of the features these libraries support natively. Sklearn is great to use due to its versatility and how it has a large number of machine learning algorithms for a variety of different uses.

Due to Sklearn being such a widely used library and arguably being one of the most well known python libraries it has a large amount of tutorials and examples. The documentation for this library is very well maintained and has many tutorials that can be used to get a better understanding of the concepts. This helps make using this library more accessible as without all the documentation and tutorials it would make using the library more challenging.

While Sklearn is an expansive, excellent library it isn't specifically designed for collaborative filtering, which means its algorithms aren't built for the sole purpose of collaborative filtering, they can still be used but this may affect the results slightly due to it not being purpose built for collaborative filtering.

6.5.3.4.2 Model

The model consists of two separate parts, the initial baseline model and the hyperparameter tuning of the model. We begin by splitting up the dataset such that the interactions, user_ids and business_ids, are put into one dataset and the ratings are put in another dataset. They are then both split into training and testing at the appropriate rate 80/20.

For the model we use KNeighborsRegressor which is a model that is a model apart of the K-nearest neighbours approach. We initialise the model with 5 neighbours and fit both the training datasets, the interactions dataset and the ratings dataset. Once this has been completed we can begin to predict using the test dataset. We only pass in the interactions test dataset and then it predicts the values.

Finally we can compare the predicted vs the actual ratings using the ratings test dataset and we can get a value for how the model is performing. Once this has been completed we can move onto optimising the model.

6.5.3.4.3 Hyperparameters

To optimise the performance of the model we will change the values of the hyperparameters. As we are using the Sklearn library we can use the inbuilt function GridSearchCV, which will perform the grid search for us. By passing the a model, a parameter grid, which holds the hyperparameters we want to change and the values we want to test with, a scoring function and the cv value for cross validation, which splits the dataset into 5 folds and runs the model 5 times with each fold being the test set for a run. The parameters are described below and their values

- n_neighbors: The number of neighbours used for regression, values = 3, 5,
- weights: Determines the weights of the neighbours, values = uniform, distance
- algorithm: The algorithm used to calculate the neighbours, values = auto, ball_tree, kd_tree, brute
- leaf_size: Maximum number of points stored in a tree node, values = 10, 20, 30
- metric: Distance metric to calculate the distance between neighbours, value = euclidean, manhattan

Once all the possible models have been calculated the best performing model and its hyperparameters will be returned, so that we will be able to compare it against the other python library models.

6.5.3.5 Surprise

6.5.3.5.1 Description

Surprise is a Python scikit for building analysing recommender systems that deal with explicit rating data, which provides various ready-to-use prediction algorithms such as baseline algorithms, neighbourhood methods, matrix factorization-based and many others [41]. The surprise library is mainly focused on building collaborative recommenders and has many different machine learning algorithms that can all be used to build models.

Surprise is built on top of the previous library we looked at Sklearn, which means all of the natively supported functions and features that Sklearn used can also be used

in the Surprise library workflow. Surprise also has a wide range of documentation and examples with built in datasets you can use to start learning how the library works, which makes the learning curve easier for this library.

Surprise is a good choice for a collaborative filtering model due to the sheer number of machine learning algorithms it has on offer to build models while also having a vibrant community that actively seeks to share their knowledge through tutorials and documentation.

6.5.3.5.2 Model

This model differs from the rest of the models due to the large number of machine learning algorithms we will be testing, but we will follow a similar idea of only optimising the hyperparameters of the model that performs the best, as attempting to optimise all the algorithms would take a considerable amount of time and be a waste of computational resources.

We initially start by loading in the dataset and then initialising an instance of the Surprise Reader class, this is so that we can set the range of values that the model would be predicting. With both the dataset and the range they can be fitted to an instance of the Surprise Dataset class, which we use as it has better native support with the Surprise library and will make the whole model run better.

With the dataset set up we can begin evaluating each of the machine learning algorithms. By passing each of the algorithms into a function and having that function train the models on the training set and then evaluate each of the models we can find out which model performs best. Each model's result is saved into a list until all the models have been then we can get the model which performs the best.

From our testing we found the model using the algorithm BaselineOnly performed the best, so we will attempt to optimise this model by trying different hyperparameters to improve the performance.

6.5.3.5.3 Hyperparameters

We can once again use Sklearns GridSearchCV function to perform a grid search for us. By passing the algorithm, a parameter grid, a scoring function and a value for the number of cross validations, we can begin evaluating each model. The parameters are described below and their values.

- method: Method to compute the baseline, values = als, sgd
- ref: Regularisation to prevent overfitting, values = 0.05, 0.01, 0.02, 0.03
- learning_rate: Learning rate of the model, values = 0.005, 0.01, 0.02
- n_epochs: The number of times the model runs over the training set, values = 10, 15, 20, 30, 40

Once all the possible models have been evaluated we can get the best performing model with the specific hyperparameters that were used. We can then compare this model against the models from other libraries.

6.5.4 Collaborative Filtering Evaluation

6.5.4,1 Root Mean Square Error (RMSE)

The root mean square error (RMSE) measures the average difference between a statistical model's predictive values and the actual values [42]. To evaluate a model using RMSE, the closer the RMSE value is to 0 the better the model is performing, this is because it would show the average difference between the prediction and actual are low.

Specifically we are using this evaluation metric as it will show how close our model is at predicting the ratings that the user would rate for a restaurant. This is what makes it useful for comparing against other models as we will be able to see how accurate the models are at successfully predicting what a user would give to a restaurant for its rating.

6.5.4.2 Baseline Comparison

Our initial evaluation of all our models is to compare against the baseline model. For this we used the best performing model from each of the libraries, before we performed the hyperparameter optimization of each model and compared it against the baseline model. Figure 68 displays a graph of each library's best performing model compared to the baseline model.

We can observe that even the worst performing machine learning mode, LightFM, performs better than the simple baseline. This showcases that each of the models are effective at improving the recommendations compared to the baseline model, but also displays how the Surprise libraries model is performing better than Sklearn, Fastai and Lighftm by a considerable margin.

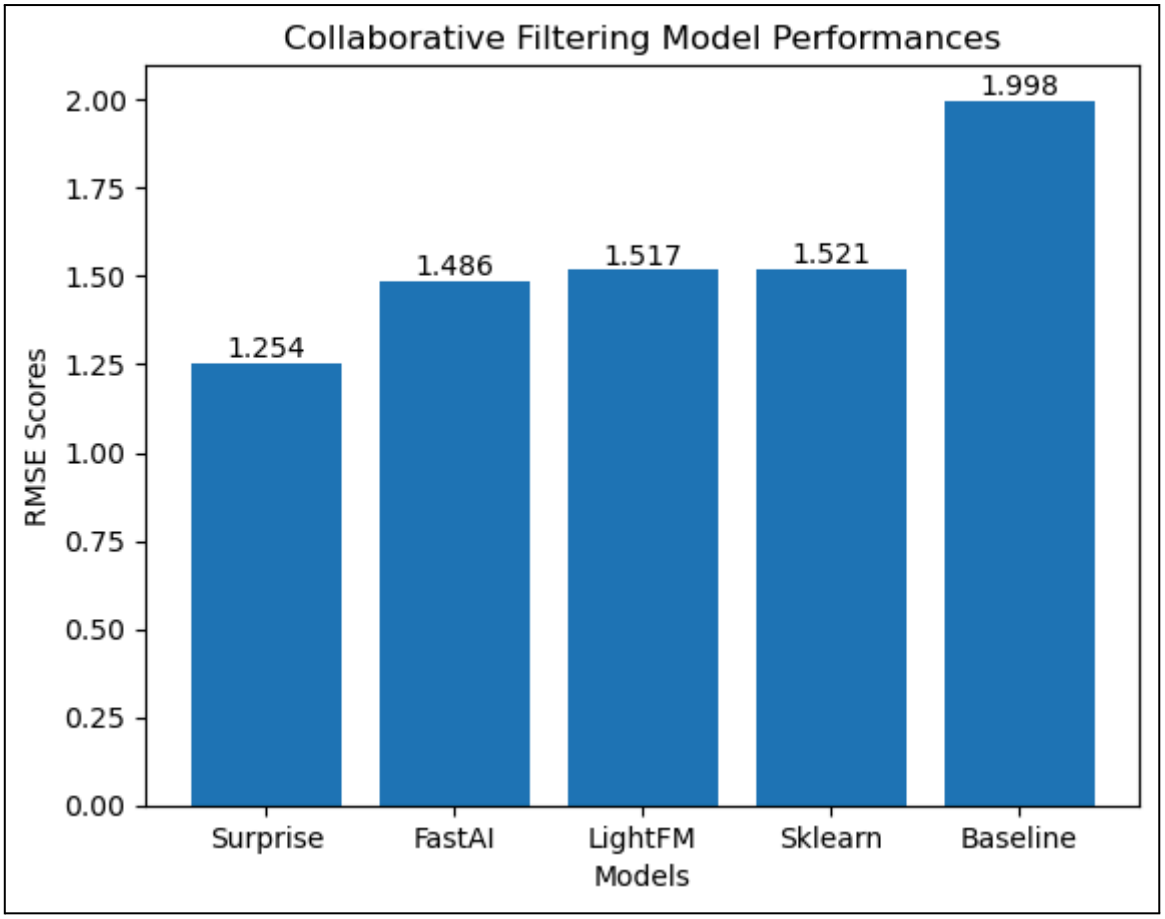

Fig. 68 - Graph comparing models to baseline

6.5.4.3 Hyperparameter Optimisation

Hyerparameter optimization is an important part of improving the performance of the machine learning models. Figure 69 displays a graph of each library's initial model, in blue and the improved optimised model in orange. This graph allows us to visually explore the performance increase we have obtained from experimenting and testing with each of the machine learning models.

The Surprise model showed the least significant improvement going from achieving a RMSE score of 1.254 to 1.24. This would suggest that the initial hyperparameters that the model has initially set are effective at training on this dataset. Whilst Surprise had a minimal improvement it was already outperforming the other models so the performance is still excellent compared to them.

The LightFM model was the 2nd worst performing model and made a significant improvement to remain the 2nd worst performing machine learning model. The model went from achieving an RMSE score of 1.517 in its predictions to a score of 1.415, which is a 6.7% improvement compared to the standard model. LightFM responded well to the hyperparameter tuning and this is in no small part due to the large number of parameters and values that can be tested. The overall improvement doesn’t move the model up the overall ranking.

The Sklearn model made a slight improvement upon optimising the hyperparameters, with the initial score being 1.521 which was improved to 1.505. While this is a good improvement, it is completely overshadowed by the improvement of the LightFM model. This model is the 3rd best performing model overall.

Finally Fastai made a significant improvement after the hyperparameter optimisation, the initial score was 1.486 and after the optimisation this dropped to 1.401. Whilst Fastai utilises deep learning for its model it can lead to the hyperparameters being difficult to tune correctly. But nonetheless this model was the 2nd best performing machine learning model.

Overall each model made some sort of an improvement with some being more observable then others. This showcases the importance of hyperparameters as in one case it grants a 6.7% improvement in performance for the LightFM model.

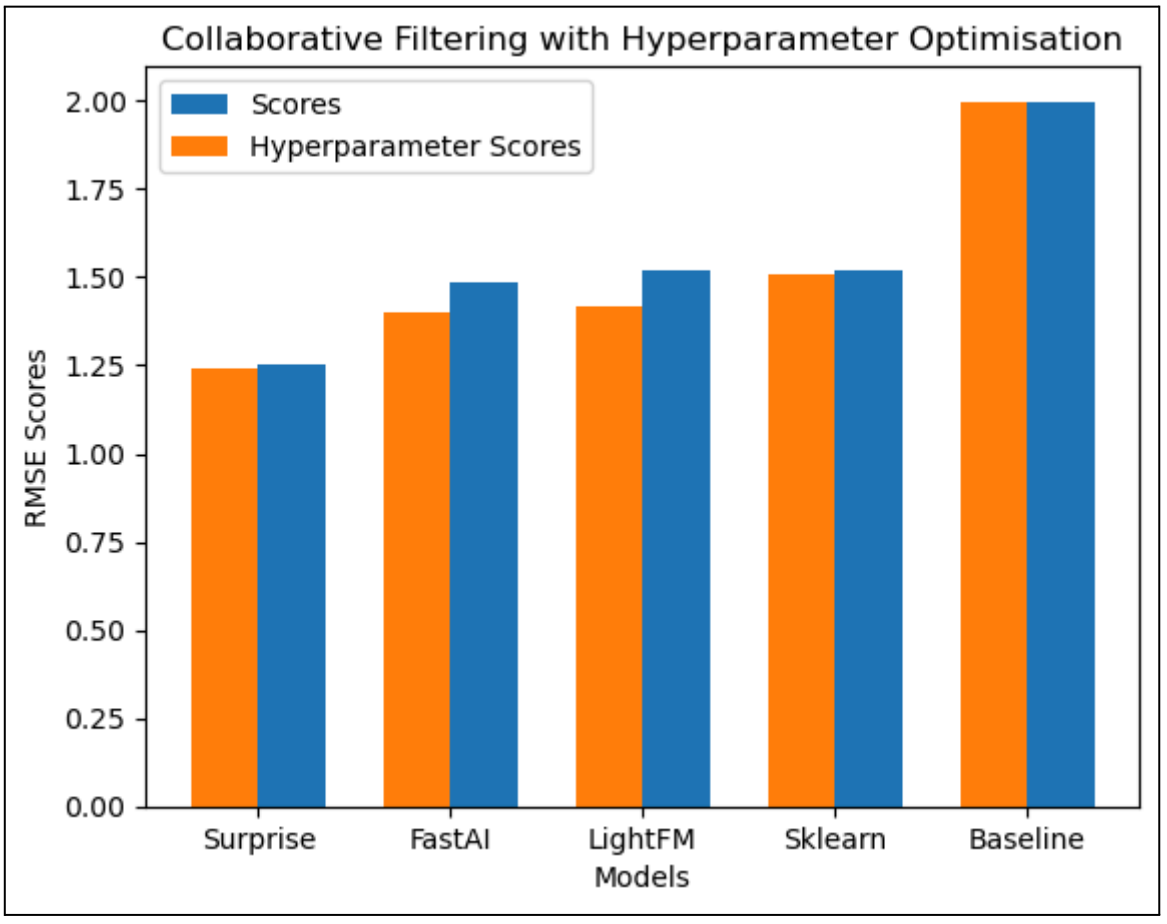

Fig. 69 - Graph comparing models to optimised models

6.5.5 Content Based Filtering Datasets

Compared to the collaborative based approach which only used one dataset for testing its models which consisted of the user_id, stars and restaurant_id, the content based approach will be tested on three different datasets which are, categories, attributes and categories and attributes. In this Yelp dataset restaurants have a lot of content associated with them and by breaking the content up into different datasets we will be able to identify which works best and generate recommendations. An outline of each of the datasets can be found below.

6.5.5.1 Categories

The first dataset that we will be using to test our models is the categories column from the Yelp business dataset. Categories are assigned to each restaurant based on what kinds of food they are serving at their restaurant, some examples are the likes of ‘Mexican’, ‘Burgers’ and ‘Gastropubs’. This has some issues as the categories were originally saved in a list and because of this it wouldn’t be compatible with many machine learning algorithms so that is why during data transformation we transformed the category column to be one hot encoded so that it would be in the correct format.

This was done so that it was easier to compare the similarity of two restaurants based on these new transformed columns as if the category appears for that restaurant the unique column for that category would be set to 1 and for the categories that don’t appear their values would be set to 0. We choose to only one hot encode the 40 most frequent categories that appear and all the rest would be disregarded.

This dataset will allow us to only compare the similarity of restaurants based on their categories and nothing else about the restaurants information. Using these categories we hope to identify patterns and relationships between restaurants.

6.5.5.2 Attributes

The second dataset we will be using to test our models is the attributes column for the Yelp business dataset. Unlike the categories column, which is saved in a list, the attributes are saved in a dictionary and cover a wider range of features about that restaurant. Some examples of what can appear are “RestaurantTakeOut” : “True”, “RestaurantsPriceRange2” : “3”, “WiFi” : “no” and “OutdoorSeating” : “False”. From these examples we can identify that the format at which these attributes can appear are not as straightforward as the categories column.

Similarly to the categories column we wanted to one hot encode the attributes making them easier to work with for the various algorithms, but the issue in this case was the fact that each attribute has a key and a value, unlike the categories which just appeared as values. To fix this issue we simply combined all the keys and values

that had “True” in them so they would appear like “RestaurantTakeOut_True” and made them into columns. So if a restaurant had an attribute with the key “RestaurantTakeOut” and value “True” it would get a 1 for that column and if not it would get a 0. This approach worked well as we could ignore all the occasions in which a value and key pair appeared and the key value was “False” or “no”.

This dataset will allow us to compare how similar the restaurants are based on the attributes associated with each restaurant and using this information generate recommendations of similar restaurants. Comparing this dataset with the previous one will give us vital information to which is performing better.

6.5.5.2 Categories and Attributes

The final dataset we will be using to test our models is a combination of the above mentioned datasets. This dataset consists of both the attributes mentioned above and the categories mentioned above. All the information will be in the same format of being one hot encoded just will all the attributes and categories.

This dataset will allow us to see if having a more detailed overview of each of the restaurants will allow us to identify relationships and trends generating improved recommendations compared to the other datasets. In total this dataset contains 67 distance features that a restaurant could potentially have, 40 being the categories and 27 being the attributes. With this amount of information we should be able to find similarly rated restaurants.

6.5.6 Content Based Filtering Models

This section will outline the models that were developed as part of testing and designing the content based filtering approach. We made the decision to investigate and build different models in an attempt to better understand and develop a better overall recommender model. Comparatively each of the models will be run on each of the above mentioned datasets to identify which features are important to generating content based recommendations. All the models will be built such that they all have the exact same input and output and these are outlined below.

Input - This model will take an input of a restaurant id, as the model will be using this restaurant to make its calculations of which restaurants are similar.

Output - The output of this model will be the restaurants that have been calculated to be most similar to the inputted restaurant.

6.5.6.1 Baseline

6.5.6.1.1 Description

Similarly to our collaborative filtering models, we once again have designed a baseline model to be the bare minimum that each of our models should be able to sufficiently beat. Using this as a base point allows us to get a better understanding of how the more advanced algorithms are performing.

The same approach will be adopted as the previous baseline model as we train on the datasets to learn the frequency of each rating that appears, i.e. 4's appearing 25% of the time. Once the model has been trained using these frequencies it will make its predictions for the test set. The model is making its predictions on the observed frequencies.

6.5.6.1.2 Model

The model first splits the data up into two separate datasets one containing the 'user_id' and 'restaurant_id' and the second containing just the 'stars'. The data is then split into training and testing sets, but crucially the first dataset 'information' is disregarded as this information won't be needed as part of the baseline model. With the training set of stars we want to use these to create the frequencies of how often they appear. Looping through we can keep track of the amount of occurrences.

With these values we then get the total number of rows in the training dataset. Using the total number of rows we divide each of the ratings occurrences by the number of rows to get a percentage of how often the rating occurs. Using the random module in python we can make use of these percentages to make predictions for each restaurant in the test set using the percentages we have previously defined. Once this has been completed we can save our predicted ratings.

Finally we have both the real ratings and the predicted values using our baseline model. Using an evaluation metric we can compare the overall performance of this model and have that value as a baseline for all the future models that we will be testing. Every future model should be performing significantly better than our frequency baseline.

6.5.6.2 Cosine Similarity

6.5.6.2.1 Description

Our first model will use cosine similarity which is a measure of similarity between two non-zero vectors using the cosine of the angle between the vectors. Using this formula we will be able to measure the similarity between the restaurants. Using the Sklearn library we can use their inbuilt function 'cosine_similarity' to measure the angle between two vectors which in our specific case will be the one hot encoded features of the restaurants.

Once all the distances have been calculated we will use a K-Nearest neighbour approach for prediction. This will revolve around using the cosine distances calculated to get the closest K, where K is a predefined number, restaurants and using them to predict the average rating of the unseen restaurant based on their values.

6.5.6.2.2 Model

The model consists of four separate functions, the first of which begins by splitting the rating data into separate training and testing datasets. The column headings for the original dataset are saved so they can be used later to split up the dataset based on if we are only using categories, attributes or categories and attributes. After this we call the second function which has three variations.

The second function is used to divide up the dataset so that we only use the features that are required for that model, the three variations are the categories, attributes and categories and attributes. Each variation of the function simply sets different split points of the original dataset so all the unnecessary features are discarded.

Once completed the third function is called “compute_similarity_and_loop” which calculates the cosine similarity matrix of the chosen features. The matrix is formed comparing each restaurant to all the other restaurants in the dataset. With this similarity matrix we can evaluate the model's performance. By looping through the test dataset and getting each individual business_id we pass them to the 4th and final function.

The “get_similar_items” is the final function is used and uses the same logic as the K-Nearest neighbour algorithm, it will loop through the most similar restaurants to the one passed in and once it gets the K, in our case 7, most similar restaurants which are in the train dataset it returns the average rating of all 7 of them to use as our prediction for the restaurant passed in.

Once this has been completed for each restaurant in the test dataset we can compare the predicted vs the actual ratings to get a value for how the model is performing. Once this has been completed for each of the three datasets, categories, attributes and categories and attributes we will have three values for this model.

6.5.6.3 TF-IDF

6.5.6.3.1 Description

This model is similar to the previously described model using cosine similarity to measure similarity but the key difference being the use of the TF-IDF. TF-IDF will measure the importance of a word to a document in a collection, adjusted for the fact that some words appear more frequently in general. Making use of this algorithm as

part of our content based recommender will adjust the values assigned to each feature and appropriately weight the features that are more important to restaurants as they appear less times across the whole dataset.

The rest of the model will bear a resemblance to the previously described method as it will also make use of the cosine similarity and K-Nearest neighbour, to both generate the similarities and make predictions based on those similarities.

6.5.6.3.2 Model

Similarly to the description of the Cosine Similarity model, four separate functions are used for the design of this model. The only difference between this model and the previously mentioned model is in the function “compute_similiarty_and_loop”, as in the previous model this only consisted of calculating the cosine similarity and looping through the test dataset.

In this solution this function contains some more complex functionally for forming the TF-IDF matrix. This is first performed by looping through the dataset converting the one hot encoded column into a new format where if the value was 1 to replace it with the name of the column followed by a space and if the value was 0 to replace it with an empty string. Once this has been completed for the whole row then a new column is created ‘sentence’ which contains the names of all the features that were present in the restaurant. This new column will be used as the input for the TF-IDF formula.

Using this new column, an initialised TfidfVectorizer will create a TF-IDF matrix for each of the restaurants and the terms that appear across the whole dataset with relevant weighting for the more frequently appearing terms. Using this new matrix we can once again calculate the cosine similarity between the matrices to calculate the new distances between each restaurant with their adjusted TF-IDF weights for their features.

K-Nearest neighbour with a value of 7, is used to get the most similar restaurants and their ratings for the comparison and once this has been used for the whole test dataset we can compare the predicted ratings vs the actual ratings of the restaurants to see how the model is performing. This is also performed for each of the three datasets, categories, attributes and categories and attributes we will have three values for this model.

6.5.7 Content Based Filtering Evaluation

6.5.7.1 Root Mean Square Error (RMSE)

The root mean square error (RMSE) will be used again to evaluate the performance of the models, where the closer the number is too 0 indicates the average difference between the real ratings and the predictions are low.

We will specifically be using this evaluation metric to measure how accurately we are predicting the average rating of the restaurant, using the restaurants we have computed to be similar.

6.5.7.1 Baseline Comparison

Initially we want to compare our models against the baseline model as at the very least we would hope the models are performing better than the baseline. Figure 70 displays a graph of the models RMSE values and the baselines RMSE value. We can see that our models are performing better than the baseline, with each model's results being in the range of 0.825 to 0.843, whilst the baseline model only achieved 1.105. This means each model has a 24% better RMSE score compared to the baseline model using the frequencies.

Further analysing the models we can compare the TF-IDF models against the Cosine similarity models. The worst performing TF-IDF model is achieving a better result compared to the best performing Cosine model, even if only by a small amount, 0.835 compared to 0.836. This gap is only extended as we compare the best performing TF-IDF model which had a score of 0.825 compared to the 0.836.

These scores highlight the importance of using the TF-IDF algorithm as part of the content based recommender, Its valuable normalisation of which features appear a lot across the restaurants compared to the features that appear less across the restaurants has clearly aided in its ability to identify which features are better for calculating similar restaurants.

Comparing the results based on the datasets being used showcases that in both models, TF-IDF and Cosine similarity, the best performing models used the “Categories and Attributes” dataset. This doesn’t come as a surprise as combining both the categories and attributes of the restaurants allow us to have a more comprehensive description of each specific restaurant, which in turn means the recommender has more features to compare all the restaurants against. This also highlights the fact that the both categories and attributes compliment each other at describing the details of the restaurants.

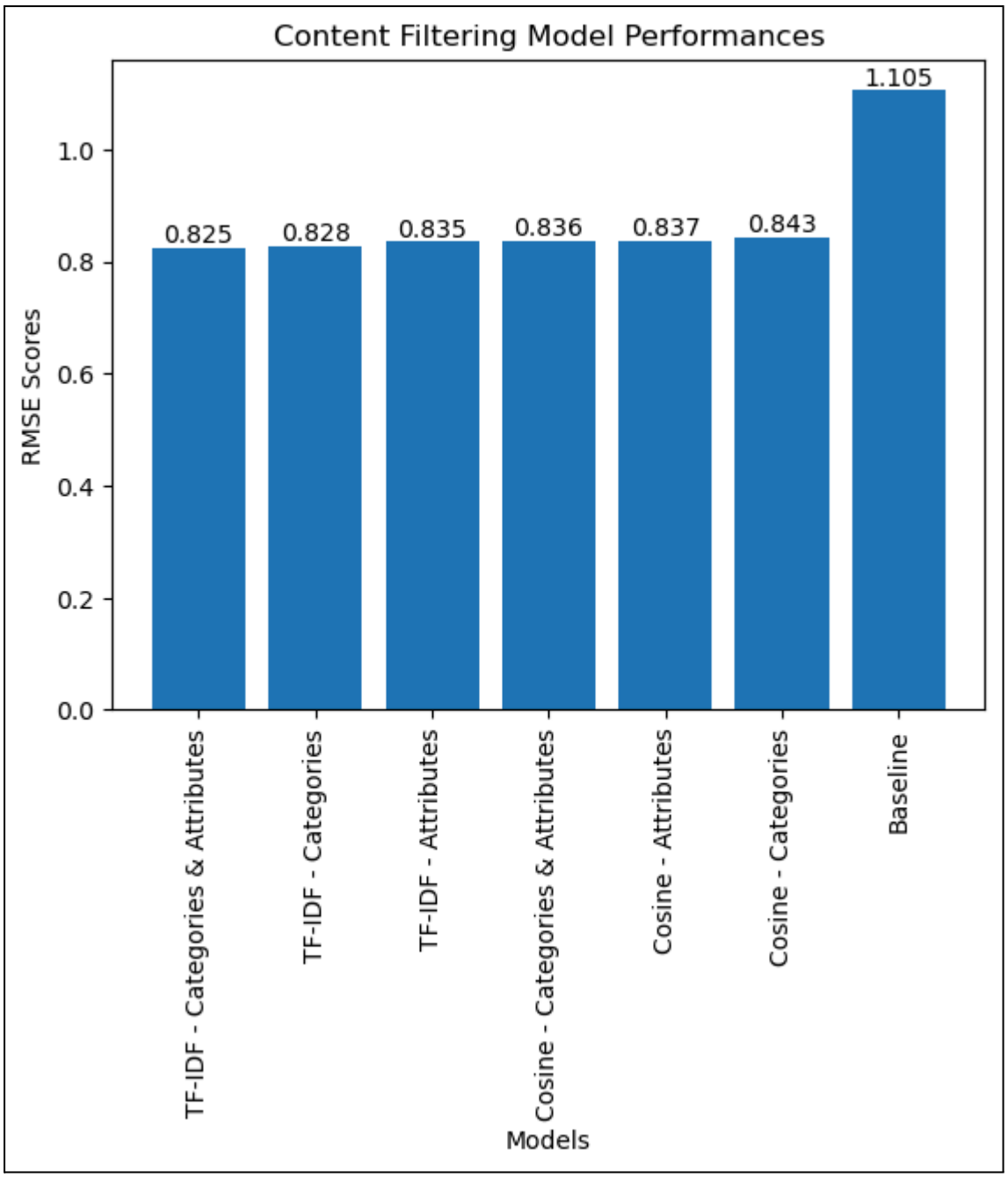

Fig. 70 - Graph comparing models results

6.5.8 Conclusion

In conclusion this testing has allowed us to get a better understanding of which models perform best for both content based approaches and collaborative based approaches. By combining the best performing models, the surprise model for collaborative filtering and the TF-IDF, using the Categories and Attributes, for content based filtering we can achieve the best possible results.

6.6 User Testing

6.6.1 Preparation

User testing is an important part of the evaluation process of our application as it allows us to gain a different perspective on the application through other users. We decided to ask 5 people each (10 total) to gain a wide range of feedback.

When creating the questionnaire we decided to implement the System Usability Scale (SUS) type of questions. This type of questionnaire is quick to administer and cheat to start using. It is also one of the most efficient ways to gather statistically valid data [43]. We also added extra task based questions to the questionnaire to perform some extra functional testing on our system and some general feedback.

When users answer the SUS part of the questionnaire we can calculate a score based on the answer provided. This final score can be used to determine the usability of our application, essentially whether our application needs improvement. The calculation of the SUS score is as follows:

- For each of the odd numbered questions, subtract 1 from the score
- For each of the even numbered questions, subtract their value from 5
- Take the new values and add the total score
- Multiple the score by 2.5

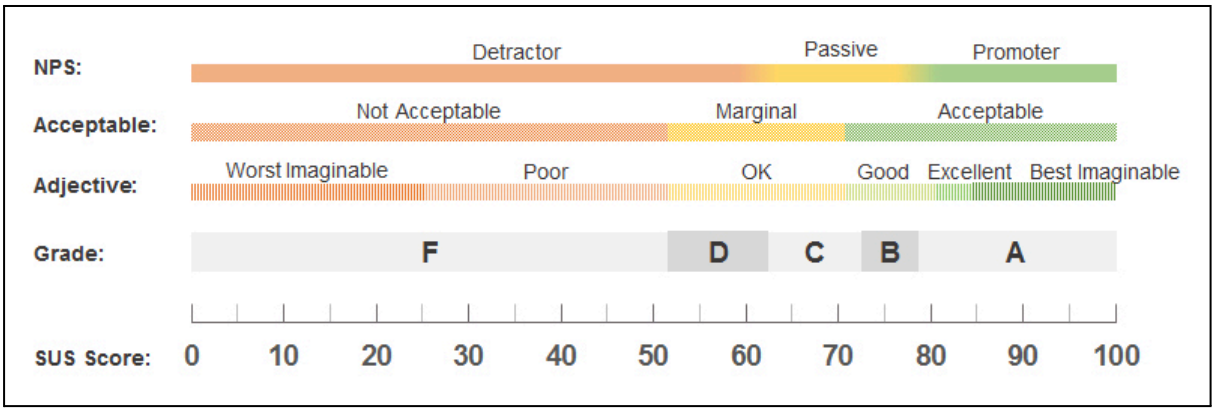

Fig. 71 - SUS Scale [45]

The SUS score is out of 100, but is not a percentage. The average System Usability Scale score is 68, so ideally our application would receive a little bit higher than that [43].

For future user testing and application improvement we could implement two systems, one has some major/minor improvements meanwhile the other continues as the regular version. This can show us if the improvements make our new version of the application better when regards to the usability.

The final part of the questionnaire allows us to gain insights into working functionality and suggested additions. Depending on the complexity of the suggestions we would implement these new suggestions.

6.6.2 Results

After our user testing was conducted we gained valuable insights into the general usability and gained important suggestions throught the responses. Since we implemented a SUS system for a portion of the questionnaire, the average SUS score of our 10 responses was 92.75 (look from Figure 72 to 82). This SUS score shows us that our application is part of the following categories (see Figure 71 for reference):

- NPS: Promoter
- Acceptable: Acceptable
- Adjective: Best Imaginable
- Grade: A

Our testing has shown that our application has really good usability and overall the respondents thought the application was easy to use and navigate. In our user testing document, we show the details of the results. One major impact which user testing has allowed us to do is improving some UI suggestions relating to understandability of the recommendation icon on the navigation bar and changing how the positive or negative sentiment of each review is shown. Below we have all the results from the questionnaire and some deeper insights.

6.6.2.1 SUS Questionnaire

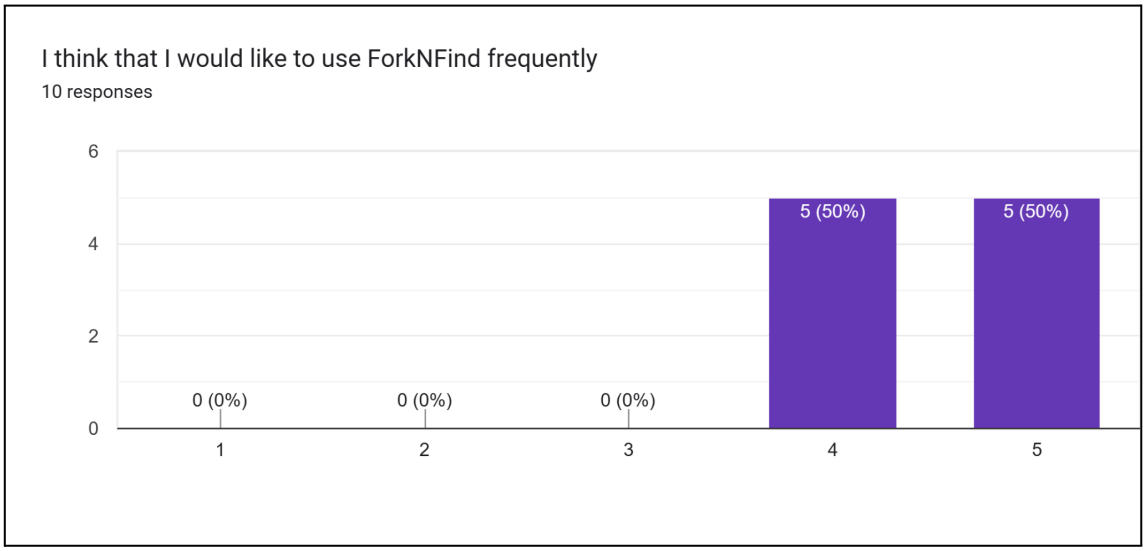

Fig. 72 - SUS frequency

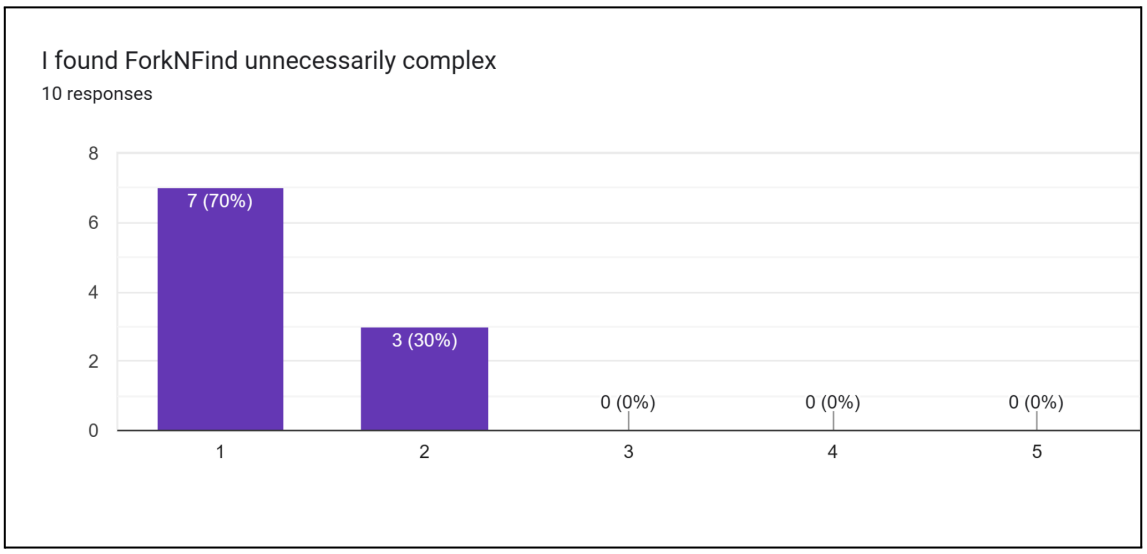

Fig. 73 - SUS complexity

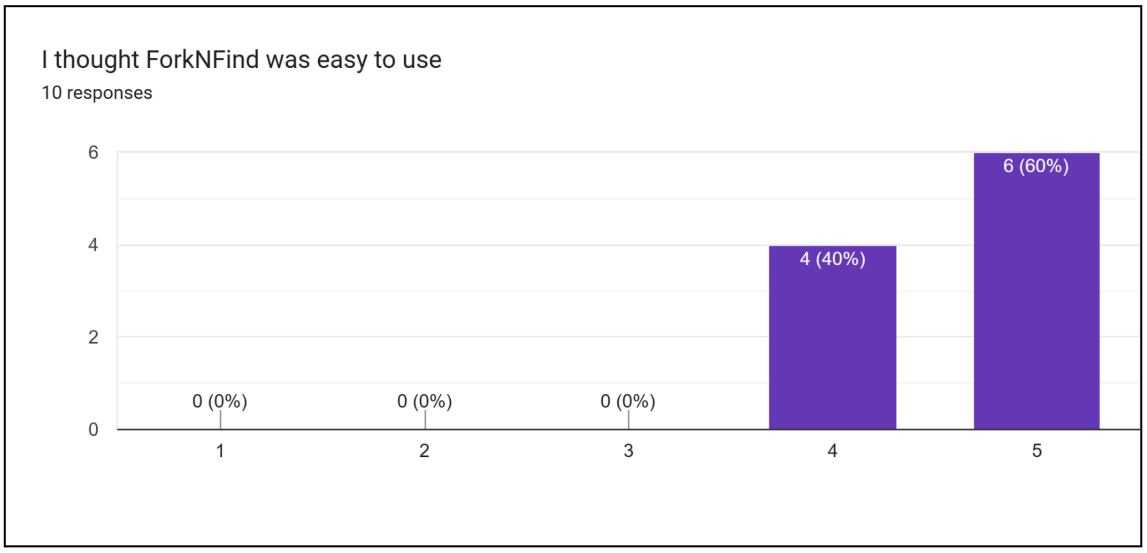

Fig. 74 - SUS ease of use

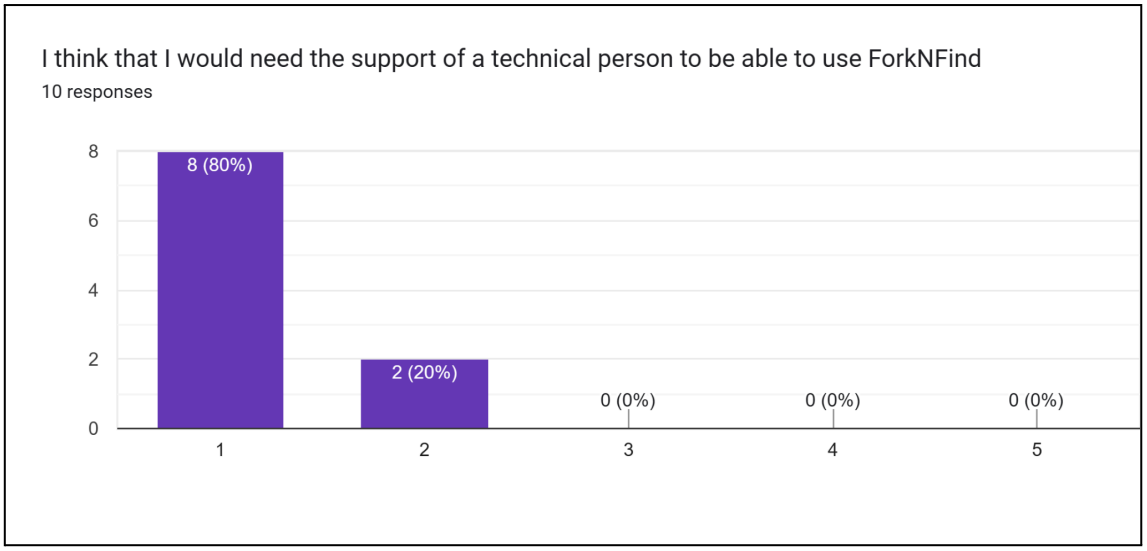

Fig. 75 - SUS need of technical support

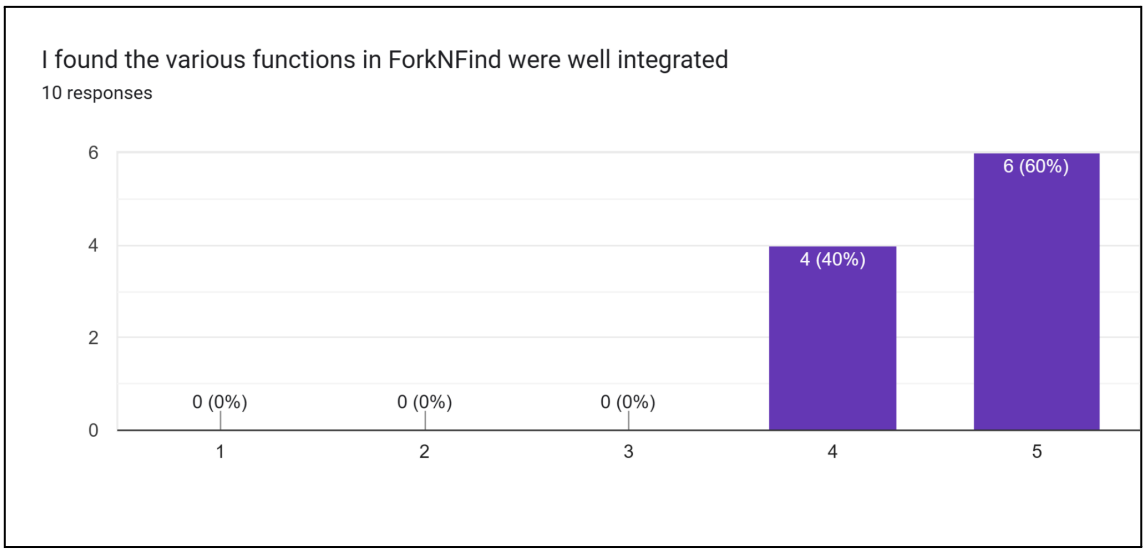

Fig. 76 - SUS well integrated

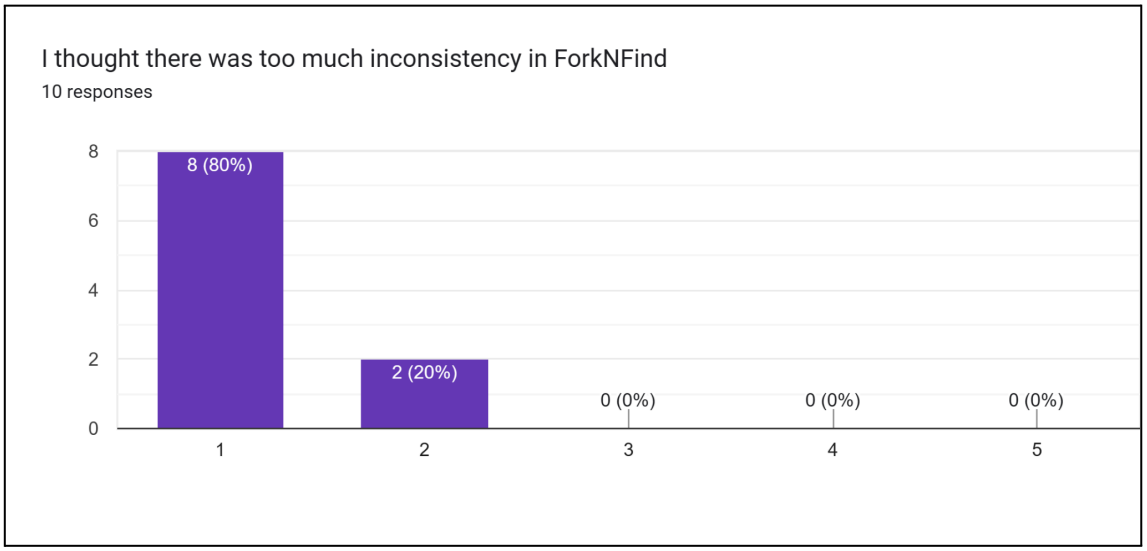

Fig. 77 - SUS inconsistency

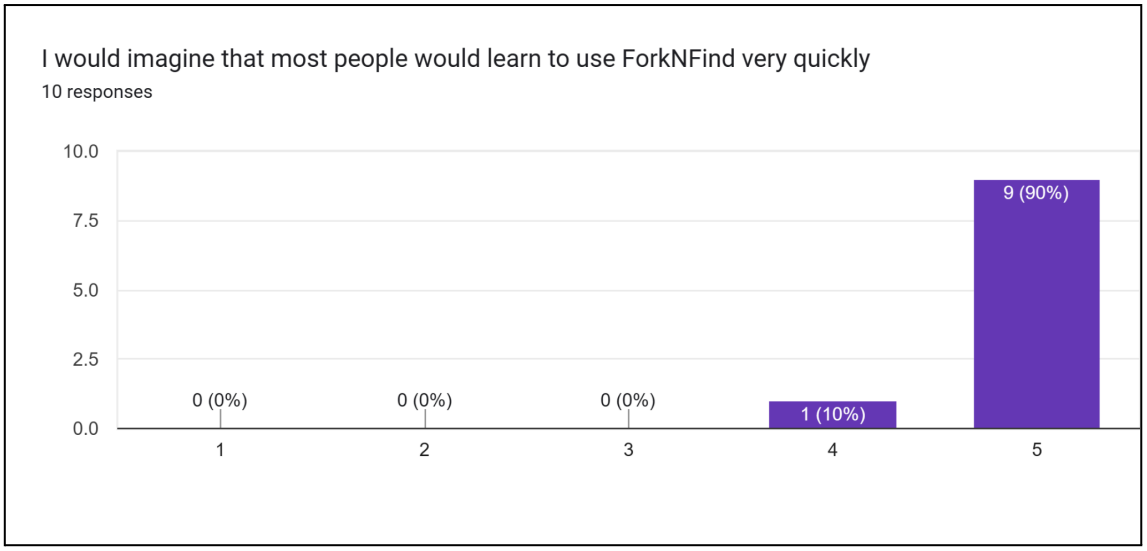

Fig. 78 - SUS learnability

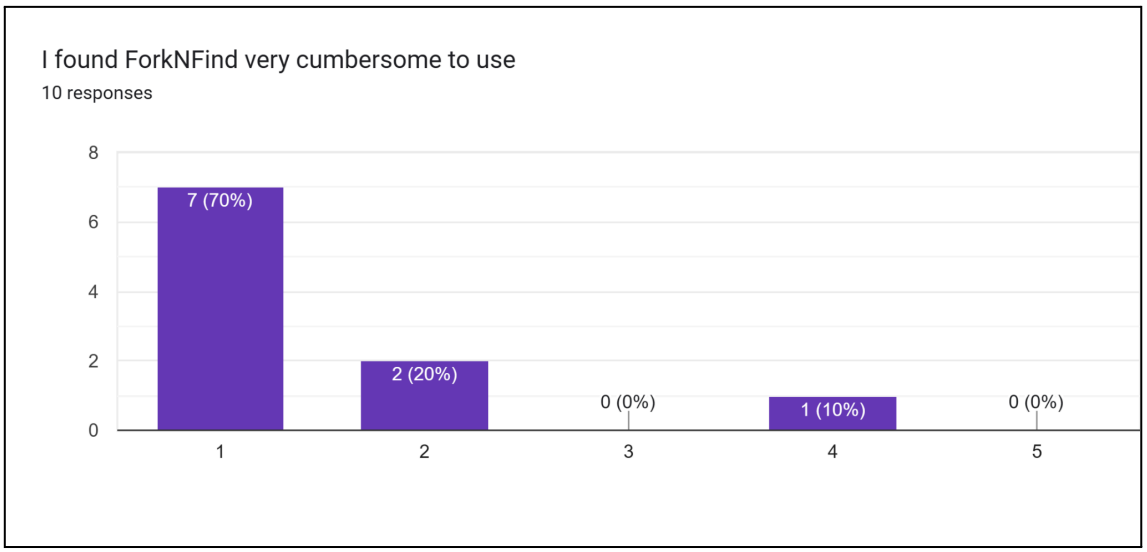

Fig. 79 - SUS use

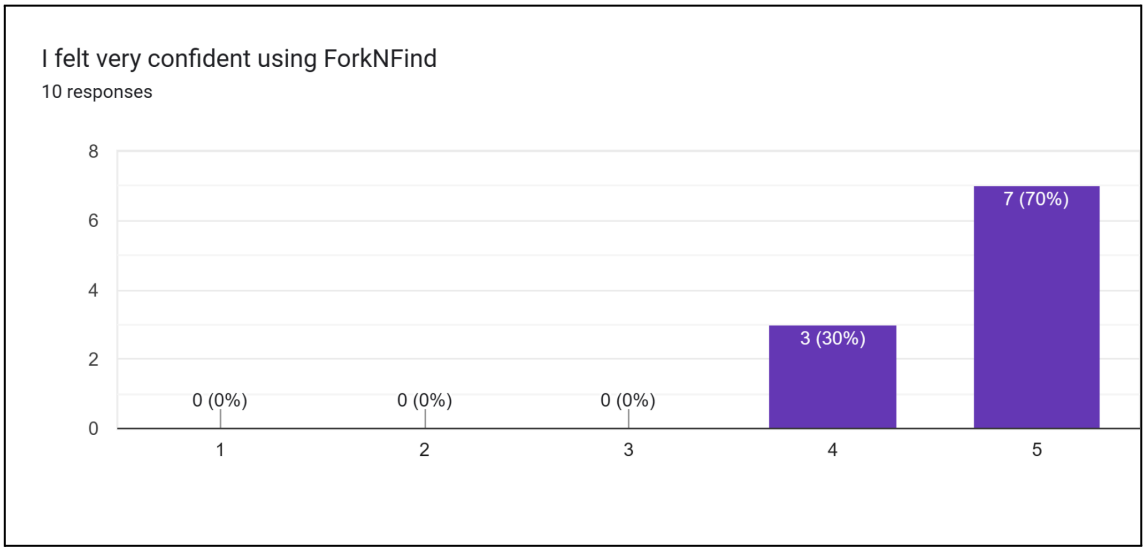

Fig. 80 - SUS confidence

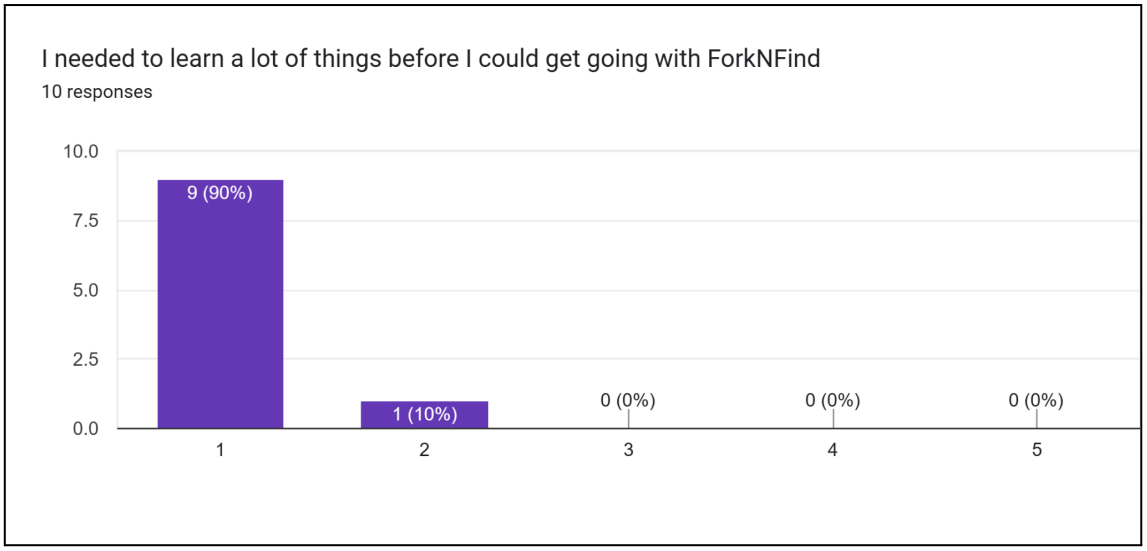

Fig. 81 - SUS learning prerequisite

SUS Raw Score	SUS Final Score
40	100
40	100
35	87.5
39	97.5
36	90
37	92.5
33	82.5
36	90
38	95
37	92.5

Fig.82 - SUS Scores

Above is all the data related to the SUS Scalability Questionnaire.

6.6.2.2 Functionality Testing & Feedback

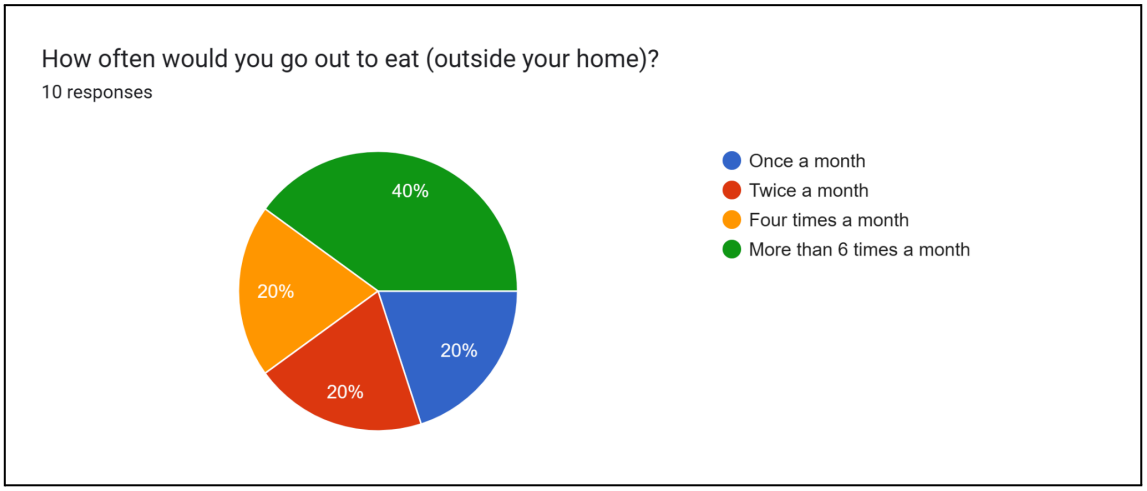

Fig. 83 - Food outings

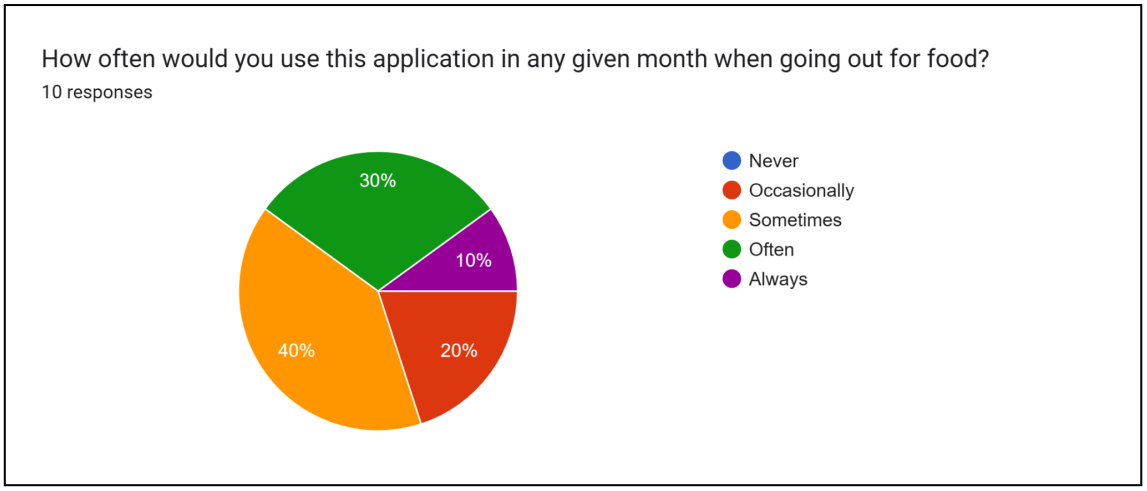

Fig. 84 - App use frequency

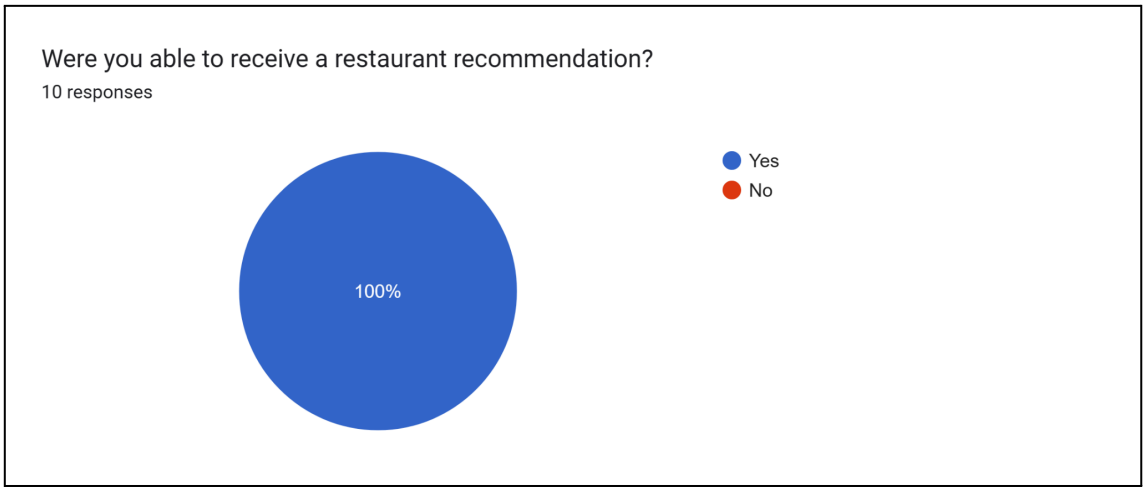

Fig. 85 - Recommendation success

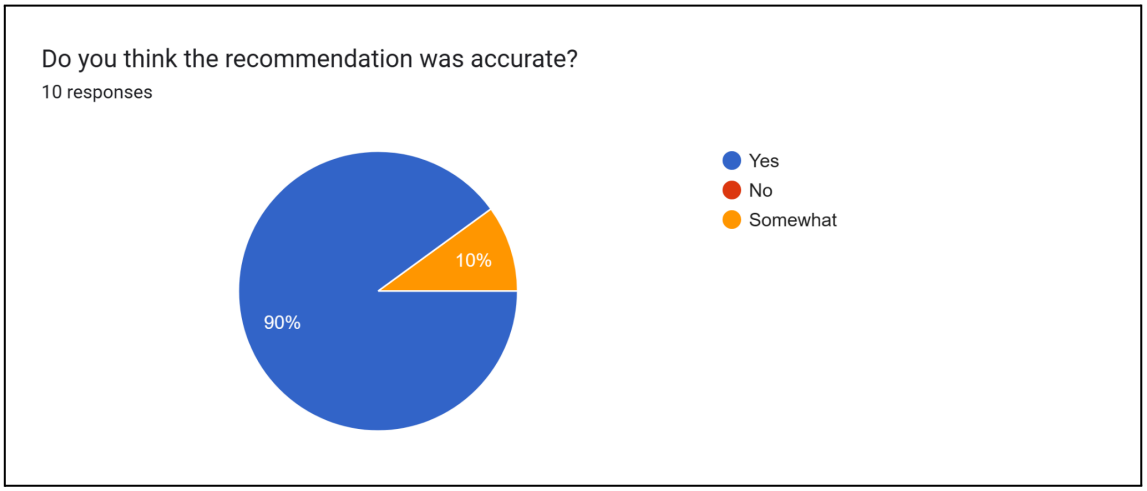

Fig. 86 - Recommendation accuracy

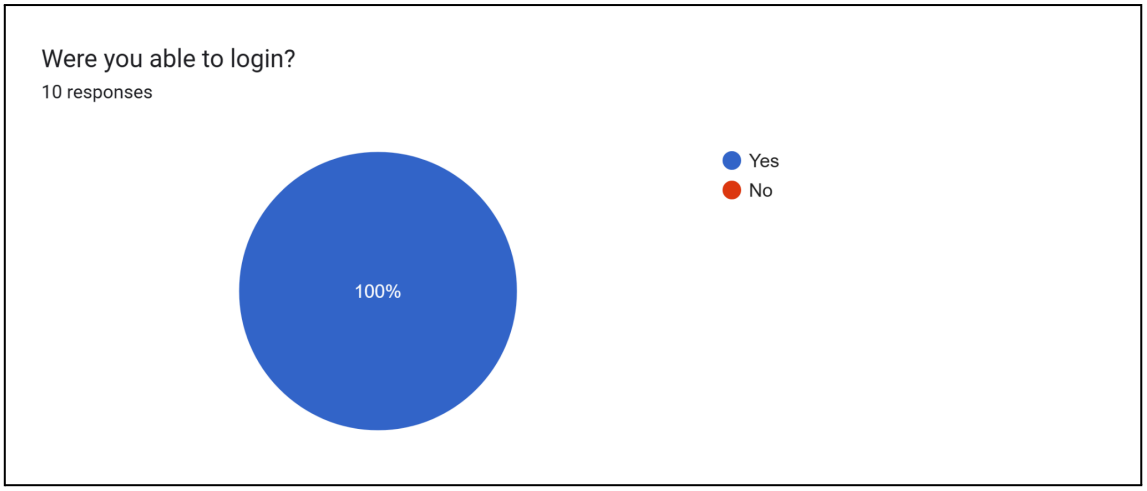

Fig. 87 - Login ability

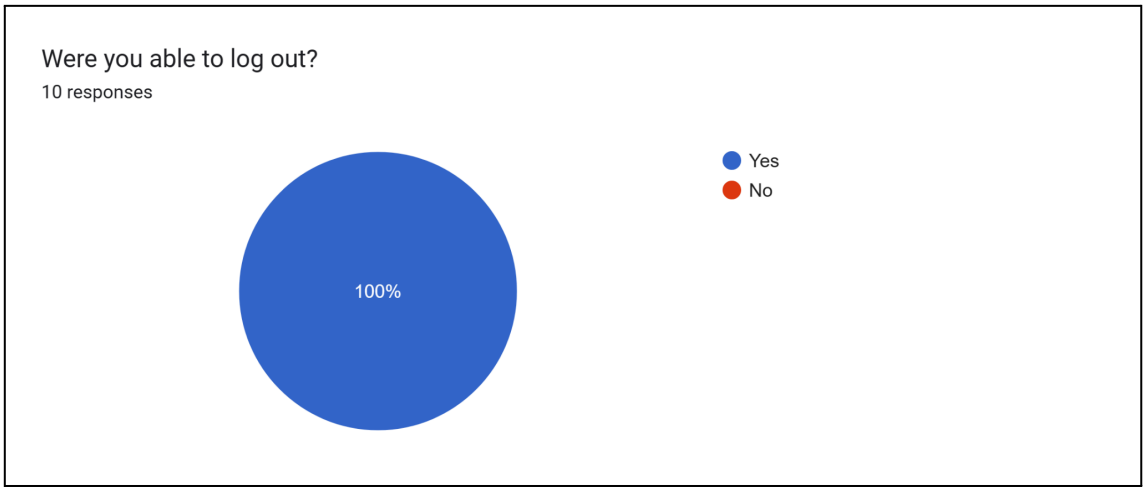

Fig. 88 - Log out ability

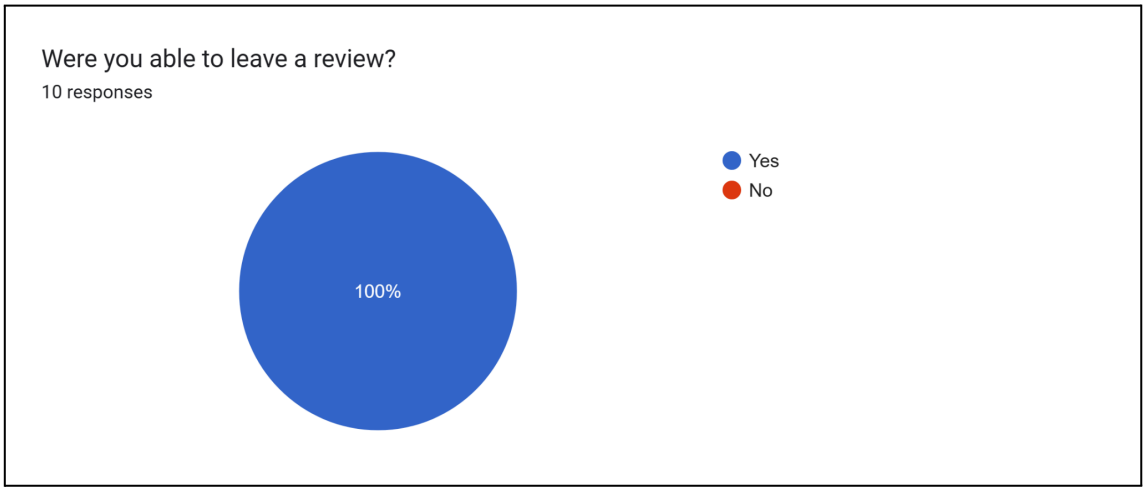

Fig. 89 - Leaving reviews

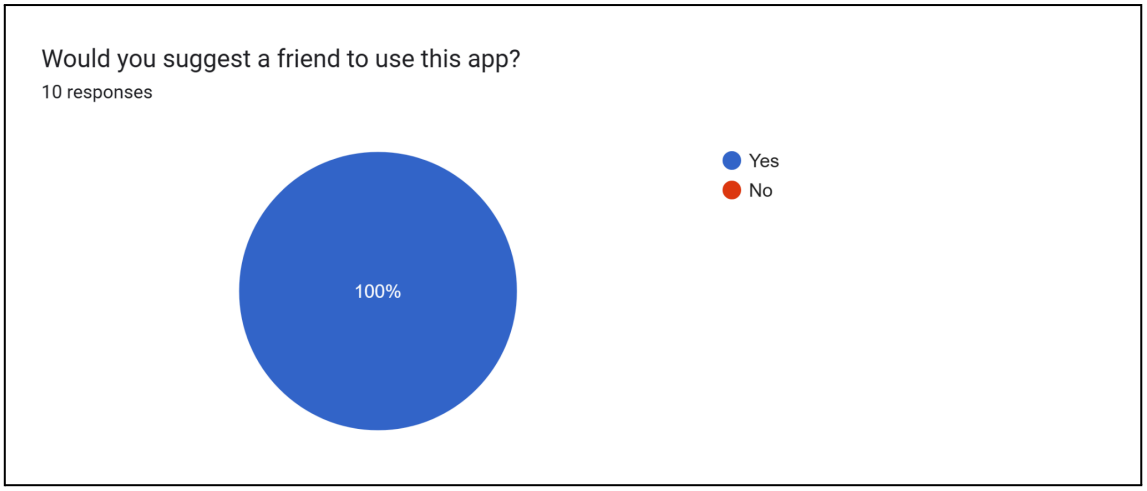

Fig. 90 - Suggesting to a friend

Above is all the data relating to the users habits and the functionality testing they undertook.

Fig. 91 - Must have features

In Figure 91, we see features that users think we should have implemented within the application. Many of these features such as having the ability to show directions to a specific establishment have been added to the future improvements and features.

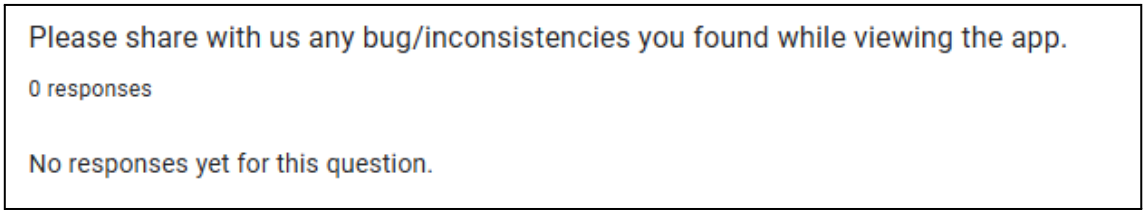

Fig. 92 - Found bug and inconsistencies

In Figure 92, we see from our 10 users, none of them ran into bugs or inconsistencies when navigating through the application.

Fig. 93 - Application feedback

The final application feedback gave us more extremely important and valuable feedback, for example the initial implementation of showing the reviews sentiment was difficult for users with colour blindness to distinguish between a positive and negative review. After gaining this feedback, we changed the method in which we display this information by using icons (smiley and sad emojis) and colour coding the two icons (red for negative and green for positive). Overall we are extremely satisfied with the results of ForkNFinds user testing.

6.7 Manual Testing

Manual testing is an important part of testing an application as it provides us developers with insights on how the application feels and if something looks and feels “off”. Automated testing does not pick these visual issues up [44]. Manual testing allows us to change functionality based on our own perception of the application and specific action.

Below are the manual tests which we undertook to have a working functionality marker for our application. In the table below we have:

- Test - the name of the test
- Actions - The necessary actions to be taken
- Expected outcome - The expected results

- Passed/Failed - Determines the pass or failure of the test

Table 4. Manual Testing

Test	**Actions**	**Expected outcome**	**Passed/ Failed**
Sign Up	1. On the login page, click “Sign Up” 2. Input personal details 3. Click Sign Up	1. The page for creating an account loads 2. The user is redirected to the location page after successfully signing up	Passed
Sign In	1. Automatically on sign in page 2. Input details 3. Click Sign In	1. The user is redirected to the location page	Passed
Choose an establishment on the map	1. If no establishment pins showing 2. Tap on green circles 3. Tap an establishment 4. View details	1. The user will be able to see establishment details	Passed
Change filter for establishment showings	1. Tap on location button placed above navigation bar 2. Choose filter 3. Apply	1. Establishments filter	Passed
Search for an establishment	1. Click on search icon (magnifying glass) 2. Input establishment name in search bar 3. Tap the magnifying glass to search Variations 4. Tap the filter icon 5. Input extra detailed information for the search	1. The user should see possible establishment cards	Passed
Opening Settings	1. Tap the cog icon in navigation bar	1. User should see Account details, Change password, Logout options	Passed
Opening Account	1. Tap Account Details	1. User should see	Passed

Details		First name, Last name, Username and Email	
Opening Change Password	1. Tap on Change Password	1. User should see Current password box, New password box and Retype new password box	Passed
Logout	1. Tap on Logout	2. User will be redirected to the sign in page	Passed
Viewing recommendations	1. Tap on lightbulb icon in navigation bar Variations 2. Click filters 3. Choose filters	1. User should see restaurant recommendations	Passed
View Reviews	1. Tap on the chat box icon for the reviews screen Variations 2. Choose positive, all or negative filter 3. Choose date, relevance or ratings filter	1. If user has created reviews, user will see positive or negative reviews shown with a smiley green icon or sad red icon 2. User can see the filtered results	Passed
View establishment	1. When a establishment card shows up on any page 2. Tap the chosen establishment 3. Scroll to view details Variations 4. Tap similar establishments 5. Tap reviews	1. User should be able to view establishment details	Passed
Create a review	1. User needs to be on an establishments page 2. Tap on Review Now 3. Choose stars 4. Input text review	1. User will be able to see review in the reviews screen	Passed

7. Future Work

When looking at our current sentiment analysis we see that it helps us complete a specific need and role, we think a future implementation and addition to our system would be to implement a prediction of a rating (1-5) given a piece of text in real-time, this could help the user with the suggestion of a rating and allow smoother process within our application. Having a rating prediction system could be implemented with the recommendation system and work together to come up with more accurate recommendations.

Another future addition to our application would be to introduce added functionalities of google maps into our application, this would allow the user to pick or be recommended an establishment by the system and the directions to be taken would show in the location screen. This would allow users to continue their experience within the application instead of leaving the app and using a navigation app.

Another future addition would be to further improve the recommender system by having access to a dataset that makes use of implicit data. Implicit data is where a user has not specifically inputted that they would be interested in a restaurant, but through techniques such as selecting a restaurant and dwell time on a page, we could mark it as implicit interest in the restaurant. This would allow us to better make predictions for users as we would have more information about interests based on what they were clicking.

8. References

[1] Python, R. (2022, August 18). *Build a recommendation engine with collaborative filtering*. https://realpython.com/build-recommendation-engine-collaborative-filtering/

[2] *Collaborative Filtering Advantages & Disadvantages*. (n.d.). Google for Developers. https://developers.google.com/machine-learning/recommendation/collaborative/summary

[3] Grover, P. (2020, July 16). Various implementations of collaborative filtering - towards data science. *Medium*.

https://towardsdatascience.com/various-implementations-of-collaborative-filtering-100385c6dfe0

[4] Rawat, S. (n.d.). *What is Collaborative Filtering? Types, Working, and Case Study | Analytics Steps*. https://www.analyticssteps.com/blogs/what-collaborative-filtering-types-working-and-case-study

[5] GeeksforGeeks. (2023, February 17). *Item-to-Item based collaborative filtering*. GeeksforGeeks. https://www.geeksforgeeks.org/item-to-item-based-collaborative-filtering/

[6] GeeksforGeeks. (2023a, January 22). *User-Based collaborative filtering*. GeeksforGeeks. https://www.geeksforgeeks.org/user-based-collaborative-filtering/

[7] *Content-based Filtering Advantages & Disadvantages*. (n.d.). Google for Developers. https://developers.google.com/machine-learning/recommendation/content-based/summary

[8] Verma, Y. (2024, March 18). *A guide to building hybrid recommendation systems for beginners*. Analytics India Magazine. https://analyticsindiamag.com/a-guide-to-building-hybrid-recommendation-systems-for-beginners/

[9] Haddi, E., Liu, X., & Shi, Y. (2013). The role of text pre-processing in sentiment analysis. *Procedia Computer Science*, *17*, 26–32. https://doi.org/10.1016/j.procs.2013.05.005

[10] Daniel Jurafsky & James H. Martin. *Speech and Language Processing*. Draft of February 3, 2024.

https://web.stanford.edu/~jurafsky/slp3/2.pdf

[11] Germec, M., PhD. (2023, October 7). *Text preprocessing with Natural Language Processing (NLP)*. https://www.linkedin.com/pulse/text-preprocessing-natural-language-processing-nlp-germec-phd#:~:text=Stopword%20removal%20helps%20to%20reduce,text%20more%20clean%20and%20simple

[12] *What are Stop Words & How Do They Impact Digital Content? | Lenovo IE*. (n.d.). https://www.lenovo.com/ie/en/glossary/stop-words/?orgRef=https%253A%252F%252Fwww.google.com%252F

[13] *Sentiment Symposium Tutorial: Stemming*. (n.d.). http://sentiment.christopherpotts.net/stemming.html#:~:text=Stemming%20is%20a%20method%20for,stemmer%2C%20and%20the%20WordNet%20stemmer.

[14] BotPenguin. (2024, February 23). *Stemming: Advantages and Limitations | BotPenguin*. https://botpenguin.com/glossary/stemming

[15] *Sign Up | LinkedIn*. (n.d.). https://www.linkedin.com/authwall?trk=qf&original_referer=&sessionRedirect=https%3A%2F%2Fwww.linkedin.com%2Fpulse%2Fdeep-dive-text-vectorization-techniques-natural-language-amrelle-d0bde%2F

[16] Pradeep. (2024, February 11). Understanding TF-IDF in NLP: A Comprehensive Guide - Pradeep - Medium. *Medium*. https://medium.com/@er.iit.pradeep09/understanding-tf-idf-in-nlp-a-comprehensive-guide-26707db0cec5

[17] Kanade, V. (2022, April 18). *Logistic Regression: Equation, assumptions, types, and best practices - Spiceworks Inc*. Spiceworks Inc. https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-logistic-regression/

[18] Ashfaque, Z. (2023, August 10). Sentiment Analysis with Naive Bayes Algorithm - Zubair Ashfaque - Medium. *Medium*. https://medium.com/@zubairashfaque/sentiment-analysis-with-naive-bayes-algorithm-a31021764fb4#:~:text=Introduction%20to%20Sentiment%20Analysis&text=The%20Naive%20Bayes%20algorithm%20is,text%20is%20positive%20or%20negative

[19] GeeksforGeeks. (2024, March 1). *Naive Bayes classifiers*. GeeksforGeeks. https://www.geeksforgeeks.org/naive-bayes-classifiers/

[20] Kanade, V. (2022b, September 29). *All you need to know about support vector Machines - Spiceworks Inc*. Spiceworks Inc. https://www.spiceworks.com/tech/big-data/articles/what-is-support-vector-machine/

[21] Wang, C. K. (2023). Sentiment analysis using support vector machines, neural networks, and random forests. In *Advances in computer science research* (pp. 23–34). https://doi.org/10.2991/978-94-6463-300-9_4

[22] *What is Random Forest?* | *IBM*. (n.d.). https://www.ibm.com/topics/random-forest#:~:text=Random%20forest%20is%20a%20commonly,both%20classification%20and%20regression%20problems

[23] *Gradient Boosting Machines · UC Business Analytics R Programming Guide*. (n.d.). https://uc-r.github.io/gbm_regression

[24] *Icons as part of a great user experience — Smashing magazine*. (2016, October 20). Smashing Magazine. https://www.smashingmagazine.com/2016/10/icons-as-part-of-a-great-user-experience/

[25] *Unit testing in React native applications — Smashing magazine*. (2020, September 29). Smashing Magazine. https://www.smashingmagazine.com/2020/09/unit-testing-react-native-applications/

[26] Mwaura, W. (2022, May 11). *Snapshot testing React applications with Jest*. CircleCI. https://circleci.com/blog/snapshot-testing-with-jest/#:~:text=Using%20snapshots%20helps%20you%20make,changes%20were%20intended%20or%20not.

[27] Arnold, J. (2018, October 10). A quick intro to React's props.children - codeburst. *Medium*. https://codeburst.io/a-quick-intro-to-reacts-props-children-cb3d2fce4891

[28] *What is CI/CD?* (n.d.). https://www.redhat.com/en/topics/devops/what-is-ci-cd#:~:text=CI%2FCD%20helps%20organizations%20avoid,increase%20efficiency%2C%20and%20streamline%20workflows.

[29] *Yelp Review Sentiment Dataset*. (2020, January 29). Kaggle. https://www.kaggle.com/datasets/ilhamfp31/yelp-review-dataset

[30] Hvilshøj, F. (2023, October 17). *Introduction to balanced and imbalanced datasets in Machine Learning*. https://encord.com/blog/an-introduction-to-balanced-and-imbalanced-datasets-in-machine-learning/

[31] Casati, F. (2022, October 8). Train-Test Data Split - fabio casati - Medium. *Medium*. https://medium.com/@sphoebs/train-test-data-split-99b43492cc56

[32] Galarnyk, M. (2022, July 28). *Understanding train test split*. Built In. https://builtin.com/data-science/train-test-split

[33] Mishra, A. (2021, December 29). Metrics to Evaluate your Machine Learning Algorithm. *Medium*. https://towardsdatascience.com/metrics-to-evaluate-your-machine-learning-algorithm-f10ba6e38234#:~:text=Area%20Under%20Curve(AUC)%20is,a%20randomly%20chosen%20negative%20example

[34] *Yelp Dataset*. (n.d.). https://www.yelp.com/dataset

[35] What is One-hot Encoding | Deepchecks. (2021, August 5). Deepchecks. https://deepchecks.com/glossary/one-hot-encoding/

[36] *fastai - Welcome to fastai*. (n.d.). Fastai. https://docs.fast.ai/

[37] Nyuytiymbiy, K. (2023, March 7). Parameters, hyperparameters, Machine learning | towards data science. *Medium*. https://towardsdatascience.com/parameters-and-hyperparameters-aa609601a9ac

[38] Wikipedia contributors. (2024, January 4). *Hyperparameter optimization*. Wikipedia. https://en.wikipedia.org/wiki/Hyperparameter_optimization

[39] *Welcome to LightFM's documentation! — LightFM 1.16 documentation*. (n.d.). https://making.lyst.com/lightfm/docs/home.html

[40] Wikipedia contributors. (2024b, April 17). *Scikit-learn*. https://en.wikipedia.org/wiki/Scikit-learn

[41] Hug, N. (n.d.). *Home*. Surprise. https://surpriselib.com/

[42] Frost, J. (2023, May 28). *Root Mean square Error (RMSE)*. Statistics by Jim. https://statisticsbyjim.com/regression/root-mean-square-error-rmse/

[43] Thomas, N. (2019, September 13). *How to use the System Usability Scale (SUS) to evaluate the usability of your website*. Usability Geek. https://usabilitygeek.com/how-to-use-the-system-usability-scale-sus-to-evaluate-the-usability-of-your-website/

[44] Wyher, T. (2022, May 18). *5 reasons why manual testing is still very important*. dzone.com. https://dzone.com/articles/5-reasons-why-manual-testing-is-still-very-importa

[45] Sauro, J., PhD. (n.d.). *5 ways to interpret a SUS Score – MeasuringU*. https://measuringu.com/interpret-sus-score/